Understanding and Supporting Software Dependency Management Across the Lifecycle

Jasmine Latendresse

A Thesis

In the Department

of

Computer Science and Software Engineering

Presented in Partial Fulfillment of the Requirements

For the Degree of

Doctor of Philosophy (Software Engineering)

at Concordia University

Montréal, Québec, Canada

September 2026

© Jasmine Latendresse, 2026

CONCORDIA UNIVERSITY
SCHOOL OF GRADUATE STUDIES

This is to certify that the thesis prepared

By: _____

Entitled: _____

_____

_____

and submitted in partial fulfillment of the requirements for the degree of

Doctor Of Philosophy

complies with the regulations of the University and meets the accepted standards with respect to originality and quality.

Signed by the final examining committee:

_____ Chair

_____ External Examiner

_____ Arm's Length Examiner

_____ Examiner

_____ Examiner

_____ Thesis Supervisor (s)

_____

Approved by _____

Chair of Department or Graduate Program Director

_____

Date of Defence

_____

Dean,

Abstract

Understanding and Supporting Software Dependency Management Across the Lifecycle

Jasmine Latendresse, Ph.D.

Concordia University, 2026

Modern software systems rely extensively on third-party packages to accelerate development and reuse established functionality. However, this reliance makes dependency management a persistent challenge across development, production, and maintenance. Developers must determine which dependencies are relevant to deployed software, evaluate packages introduced during development, and update them reliably as projects and ecosystems evolve. These activities are further changing with the increasing use of large language models (LLMs), which can recommend packages, generate imports, and propose dependency-related changes. This thesis empirically investigates how dependencies are introduced, used, and maintained, and examines how existing and emerging development practices affect the reliability of these activities.

We first establish how dependencies introduced through conventional software-development practices are reflected in production. Through an empirical study of 100 JavaScript projects from the npm ecosystem, we determine which installed dependencies are included in production releases. We find that less than 1% of installed dependencies reach production and that conventional runtime and development classifications do not reliably reflect production use. These findings show that manifest-level information alone is insufficient for understanding and prioritizing dependencies once software is deployed. We then examine LLMs as an emerging mechanism through which dependencies are introduced during development. We evaluate six proprietary and open-source LLMs using 112 real-world Python programming problems from Stack Overflow. The models generally recommend mature, popular, and permissively licensed libraries, but some generated imports cannot be mapped automatically to installable packages, and installation guidance is rarely provided. To support the integration of these dependencies, we introduce the Software Librarian, a Visual Studio Code extension that validates package identities and provides information about licensing, dependencies, deprecation, and package health. The tool correctly classifies 99 of 100 packages in its evaluation dataset. Finally, once a dependency has been selected and integrated, it must be maintained as the project and its

packages evolve. We investigate whether LLMs can support this stage by generating dependency-update patches for JavaScript projects. We mine 9,908 dependency-update pull requests from 423 repositories and construct a benchmark of 56 updates from 46 repositories. Although the models frequently identify the intended dependency change, only 50.2% of manifest patches apply to the supplied project state, and 47.1% are both applicable and semantically correct. Providing additional source and test context further reduces manifest-level success, while applicable source-code adaptations remain rare.

This thesis highlights that reliable dependency management requires context beyond manifests, import statements, and update requests. LLMs can support package selection and maintenance reasoning, but they do not eliminate the need for production-aware, ecosystem-aware, and repository-aware tooling. The findings motivate dependency-management workflows that combine LLM capabilities with structured validation, deterministic editing, and project-specific context.

Acknowledgments

I would like to begin by thanking my supervisor, Prof. Emad Shihab, for his continued support throughout my PhD. I feel truly privileged to have been mentored not only by an expert in the field, but also by someone who genuinely cares about his students and continually encourages them to do their best. The skills I have learned under Emads mentorship will remain with me throughout my career and beyond. I am deeply grateful for his trust, guidance, and for the ways in which his mentorship has helped me grow.

When I began my academic journey, I also had the privilege of working with several exceptional postdoctoral researchers. I would like to thank Dr. Diego Elias Costa, Dr. Ahmad Abdellatif, and Dr. SayedHassan Khatoonabadi for sharing their knowledge and taking the time to guide me when I still had so much to learn. As I reach the end of my PhD, I hope to extend the same support to others.

I would also like to thank my committee members, Dr. Nikolaos Tsantalis, Dr. Tse-Hsun (Peter) Chen, Dr. Mohammad Mannan, and Dr. Benoit Baudry for taking the time to read my thesis and for providing valuable feedback throughout my PhD.

I would also like to express my appreciation to my colleagues at the Data-driven Analysis of Software (DAS) Lab. I am especially grateful to Dr. Suhaib Mujahid and Dr. Rabe Abdalkareem for guiding me toward my first publications, and to Dr. Abbas Javan Jafari and Khaled Badran for the many insightful discussions we shared. I would also like to thank everyone who made the lab feel like a family: Lori Akiyama, Samuel Abedu, Musfiqur Rahman, Antonio Collante, Alor Ebube, Chaima Jaziri, Caren Rizk, Mayra Ruiz, Gideon Peters, Sharon Ho, Riya Dutta, Mohamad Salim, Oluwatoyosi Oyelayo, Anthony Forgetta, and Ghada Abushaqra. I will you all the best.

Finalement, j'aimerais remercier mon professeur de mathématiques avancées au Collège Lionel-Groulx, Jean-François Perreault, pour son immense patience. Il a été le premier à croire en mes capacités sur le plan académique et à m'encourager à poursuivre mes études en génie logiciel.

Dedication

À Maman, comment te remercier? Si je parviens à devenir ne serait-ce qu'une fraction de la femme que tu es, je considérerai avoir réussi ma vie. Tu es une femme qui a tant donné et qui continue de donner sans jamais rien demander en retour. Tu es forte, profondément ancrée, et tu représentes pour moi la vie, la stabilité. Tu es mon pilier, ainsi que celui de toute notre famille. Je n'aurais jamais pu accomplir ce diplôme sans ton soutien, mais surtout sans l'exemple que tu m'as donné toute ma vie. Je t'aime, Maman. Merci pour tout.

À Papa, tu m'as toujours fait sentir comme si j'étais la personne la plus intelligente au monde. Tu m'as toujours encouragée et poussée à croire en moi, malgré tous les obstacles que la vie a placés sur mon chemin. Merci pour toutes les fois où tu m'as conduite à l'université, où tu m'as attendue après un cours tard le soir, et où tu as simplement été là, sans jamais hésiter. Ta confiance en moi m'a donné la force d'aller plus loin que je ne l'aurais cru possible. Une part de cette réussite t'appartient. Je t'aime, Papa, et je te remercie pour tout.

À Alexandre, mon frère, mon frère d'âme. Tu as toujours été une présence calme et rassurante dans ma vie. Tu es peut-être mon petit frère, mais ta sagesse me donne parfois l'impression que les rôles sont inversés. Merci d'être toujours là, avec ta douceur, ton écoute et ta façon bien à toi de rendre les choses plus simples. Je t'aime.

To my dear fiancé, Kalle. My home, even from far away. You have been by my side through every high and low, every laugh, and every tear. There were quite a lot of tears, admittedly, but you were always there to wipe them away and make everything feel better and calmer. I would not have made it this far without your love and support. Words could never fully express what you mean to me or how grateful I am to have you beside me. I love you more than I can say.

À Grand-maman et Grand-papa. Vous avez été présents à chaque étape de ma vie : la garderie, la

maternelle, l'école primaire, l'école secondaire, le cégep, les vacances, les pauses dîners et les devoirs. Vous m'avez accompagnée bien avant que je puisse comprendre tout ce que cela représentait. Grand-papa, j'aurais tellement aimé que tu puisses me voir arriver au bout de ce parcours. Je sais que tu me vois, même si, moi, je ne peux plus te voir. Cette réussite est aussi un peu la vôtre.

À toute ma famille, et tout particulièrement à ma cousine Stéphanie, à Léo et à Éllie, ainsi qu'à mes tantes et mes oncles. Vous avez tous contribué, d'une façon ou d'une autre, au chemin qui m'a menée jusqu'ici, que ce soit en m'aidant avec mes devoirs, en m'encourageant ou simplement en étant présents au fil des années. Merci d'avoir cru en moi et d'avoir partagé les petites comme les grandes étapes de ce parcours.

À Frimousse et Popeye, mes chiens d'âme. Frimousse, tu as été à mes côtés durant toutes les grandes transitions de mon parcours académique. Tu as rendu ma vie plus colorée et tu m'as tant appris. Popeye, mon beau chien. J'en ai écrit, des papiers de recherche, pendant que tu dormais sur moi. Je sais que le temps qu'il nous reste est précieux, et je suis si heureuse d'avoir pu écrire cette thèse avec toi à mes côtés. À Mona et Abraham, je vous aime. Et à tous les autres animaux qui ont fait partie de ma vie : Toulouse, Berlioz, Fluffy, Lily et Lady Gaga, mon petit hamster. Une mention toute spéciale à Rufus, Jay, Marshall, Maggie, Kira, Charlie, Doudoune, Whippette, et Choupette.

Table of Contents

List of Figures **xi**

List of Tables **xii**

1 Introduction and Research Statement **1**

1.1 Introduction 1

1.2 Research Statement 3

1.3 Thesis Overview 4

1.4 Thesis Contributions 6

1.5 Related Publications 7

1.6 Thesis Organization 8

2 Background and Related Work **9**

2.1 Background 9

2.2 Related Work 11

2.3 Positioning of the Thesis 23

3 Not All Dependencies are Equal: An Empirical Study on Production Dependencies in NPM **25**

3.1 Chapter Overview 25

3.2 Introduction 26

3.3 Motivation & Background 28

3.4 Methodology 31

3.5 RQ1: How many installed dependencies are production dependencies? 35
3.6 RQ2: What are the characteristics of production dependencies? 38
3.7 RQ3: How often are npm security alerts emitted for production dependencies? . . . 41
3.8 Discussion 43
3.9 Related Work 47
3.10 Threats to Validity 50
3.11 Chapter Summary 50

4 How Robust are LLM-Generated Library Imports? An Empirical Study using Stack Overflow 52

4.1 Chapter Overview 52
4.2 Introduction 53
4.3 Methodology 56
4.4 RQ1: What types of software libraries are recommended by LLMs? 59
4.5 RQ2: What are the characteristics of third-party libraries recommended by LLMs? 61
4.6 RQ3: Why do some libraries recommended by large language models not work out of the box? 65
4.7 Discussion 68
4.8 Related Work 77
4.9 Threats to Validity 79
4.10 Chapter Summary 82

5 The Software Librarian: Python Package Insights for Copilot 84

5.1 Chapter Overview 84
5.2 Introduction 85
5.3 Background 86
5.4 A Walkthrough of the Software Librarian 87
5.5 Evaluation 92
5.6 Chapter Summary 93

6 Evaluating Large Language Models for Dependency-Maintenance Patch Generation 94

6.1 Chapter Overview . . . 94

6.2 Introduction . . . 95

6.3 Background . . . 98

6.4 Study Design . . . 100

6.5 RQ1: How do LLMs perform on real-world dependency-update tasks? . . . 104

6.6 RQ2: What kinds of errors or omissions are common in LLM-generated updates? . 108

6.7 RQ3: How do LLMs perform on manifest updates when source or test context is provided? . . . 113

6.8 Discussion . . . 118

6.9 Related Work . . . 122

6.10 Threats to Validity . . . 128

6.11 Chapter Summary . . . 132

7 Conclusion and Future Work 134

7.1 Conclusion . . . 134

7.2 Future Work . . . 136

Bibliography 140

Appendices 159

A.1 Prompt Templates . . . 159

A.2 Model-Level Prompt-Style Analyses . . . 163

List of Figures

2.1 Example of direct and transitive dependency relationships. 10

3.1 Overview of our approach for filtering projects and collecting dependencies. 32

3.2 Number of production dependencies on the 49 project with one or more production dependencies. 37

4.1 An overview of our approach for filtering and curating our dataset. 56

5.1 An Overview of the Software Librarian 88

5.2 An Example of Output of the Software Librarian for Three Packages. 89

6.1 An overview of our approach to curating our dataset. 100

6.2 An overview of our end-to-end experimental pipeline. 103

List of Tables

3.1 Concepts and definitions. 27

3.2 Descriptive statistics of the selected projects. 32

3.3 Dependency profiles of projects with and without production dependencies. Totals are reported as absolute counts, and medians are computed from the aggregated value per project. Percentages are relative to the number of installed dependencies in each project group. . . . 35

3.4 Characteristics of dependencies in projects with production dependencies. 37

3.5 Frequently installed packages that are both used and not used in production. 41

3.6 Characteristics of vulnerable dependencies reported by npm vulnerability alerts. 42

3.7 Number of vulnerability reports by severity level and the corresponding npm recommended action. 43

3.8 Metrics based on vulnerable dependency (VD) characteristics reported by existing tools and their support for locating vulnerable code (VC) in JavaScript projects. 46

4.1 Overview of the evaluated LLMs. 58

4.2 Classification of imported libraries in LLM-generated code. 60

4.3 Median characteristics of recommended third-party libraries. 63

4.4 Occurrences of alias and module cases per model. 67

4.5 Proportion of unused imports per model, overall and by dependency type. 72

4.6 Frequency with which models import at least one library in their completions. “Any libs” counts questions (out of 112 StackEval Python questions) where the completion imports at least one library. “Libs when GT has none” counts questions (out of the 95 questions whose accepted answer uses no imports) where the model still imports at least one library. 74

4.7 Comparison of imported libraries in model completions vs. accepted Stack Overflow answers for the subset of StackEval Python questions whose ground-truth solution imports at least one library (17 out of 112 questions). Values show the percentage of questions in each category and how often each model fully recovers the ground-truth import set. 74

6.1 Evaluated models and their main characteristics. All models are instruction-tuned. 104
6.2 Overall manifest-level performance on configuration-only dependency-update tasks ($N = 446$). 105
6.3 Manifest-level performance by model on configuration-only dependency-update tasks. 106
6.4 Paired prompt-style comparisons for configuration-only manifest-level outcomes. 106
6.5 Manifest-level failure profile by model for configuration-only dependency-update tasks. Counts and percentages are calculated over eligible outputs. 107
6.6 Failure taxonomy for configuration-only outputs that did not achieve patch success ($n = 236$). 109
6.7 Most frequent failure subtypes among configuration-only outputs that did not achieve patch success ($n = 236$). 110
6.8 Failure taxonomy by model for configuration-only outputs that did not achieve patch success. Percentages are calculated within each model's unsuccessful outputs. 112
6.9 Prompt-style comparisons of unsuccessful configuration-only outputs. category-specific comparisons are exploratory; the overall McNemar and Bowker tests provide the primary inferential results. 112
6.10 Overall manifest-level performance on configuration-code dependency-update tasks. 114
6.11 Manifest-level performance by model on configuration-code dependency-update tasks. . . . 115
6.12 Paired prompt-style comparisons for configuration-code manifest-level outcomes. Here, we apply the correction across the four manifest-level outcomes. 116
6.13 Failure taxonomy for configuration-code outputs that did not achieve manifest-level patch success ($n = 284$). 116
6.14 Most frequent failure subtypes among configuration-code outputs that did not achieve manifest-level patch success ($n = 284$). 117
6.15 Prompt-style comparisons of manifest-level failures in configuration-code outputs. 118
6.16 Structural source-edit and full-patch performance on configuration-code dependency-update tasks. 119

6.17 Model-level applicability of generated source edits and full patches on configuration-code dependency-update tasks. 120
6.18 Paired prompt-style comparisons for configuration-code outputs (n = 209). Holm correction is applied across the six reported outcomes. 122

A.1 Model-level paired prompt-style comparisons for configuration-only manifest outcomes. . . 164
A.2 Model-level prompt-style comparisons of configuration-only failures. 164
A.3 Model-level paired prompt-style comparisons for configuration-code manifest, source-edit, and full-patch outcomes. 165
A.4 Model-level prompt-style comparisons of configuration-code manifest-level failures. 166

Chapter 1

Introduction and Research Statement

1.1 Introduction

Modern software systems are rarely developed entirely from scratch. Instead, developers rely extensively on reusable functionality provided by open-source packages to reduce development effort, accelerate delivery, and avoid repeatedly implementing established solutions [1, 2, 3]. More than 90% of both open-source and proprietary projects depend on such packages [4, 5]. According to the 2024 Open Source Security and Risk Analysis report, approximately 96% of commercial codebases contain open-source software, with open-source components accounting for an average of 77% of the code in the analyzed systems [6, 7]. Consequently, external packages often make up a substantial part of modern software systems.

The reuse of packages creates dependencies between software projects and the external components on which they rely. These dependencies connect applications to large software ecosystems such as the Node Package Manager (npm) [8] and the Python Package Index (PyPI) [9], where packages may themselves rely on many additional packages. A single direct dependency can therefore introduce a large network of transitive dependencies, each maintained, released, and evolved independently. These interconnections enable software reuse at scale, but they also makes dependency management a persistent and complex software-engineering activity.

Managing dependencies remains difficult throughout the software lifecycle. Developers must select packages from large and rapidly evolving ecosystems, assess their compatibility and licensing, and account for the direct and transitive dependencies they introduce. Once adopted, dependencies must be monitored and updated as new versions, security fixes, and breaking changes are released. In practice, updates are

frequently delayed because developers lack the time, information, or confidence needed to determine whether a new version can be adopted safely [10, 11, 12]. Remaining on outdated versions can expose projects to known vulnerabilities and technical lag, while updating too aggressively can introduce incompatibilities, build failures, and unexpected changes in behavior [13].

The scale of modern dependency networks further complicates these decisions. A project may rely on hundreds or thousands of direct and transitive packages, making it difficult to determine which dependencies are relevant, which updates require immediate attention, and which alerts represent meaningful risk. Vulnerability scanners and dependency-management services can produce large volumes of notifications, many of which may concern components that are unused, unreachable, or unlikely to affect the deployed application [14, 15, 16, 17]. Developers must therefore balance security, compatibility, maintenance effort, and software stability while working with incomplete or noisy information.

To reduce this burden, dependency-management activities are increasingly supported by automation [18]. Package managers resolve dependency graphs, security scanners identify known vulnerabilities, and tools such as Dependabot [19] and Renovate [20] monitor releases and generate update pull requests. These tools automate important parts of the workflow, but they typically operate on package metadata, manifests, and predefined rules. They provide limited support for understanding project-specific usage, resolving ambiguous package recommendations, or adapting surrounding code when an update introduces breaking changes. More recently, LLM-based programming assistants have begun to extend automation into tasks that require package recommendation, code generation, and maintenance reasoning [21, 22].

Developers increasingly rely on large language models (LLMs) to streamline their development workflows [23, 24]. Consequently, these systems become a new interface for package selection and integration: rather than searching a software ecosystem directly, a developer may accept a package introduced as part of a generated solution. This can reduce the effort required to identify reusable functionality, but it does not eliminate the underlying dependency-management requirements. Developers must still verify that the suggested package exists, identify its installable distribution, assess its compatibility and licensing, and determine whether additional installation or configuration steps are required.

Prior work has shown that LLM-generated code may contain nonexistent, outdated, or difficult-to-resolve packages and may omit contextual information needed for their reliable integration [25, 26]. These limitations can transfer package-verification and integration work to developers rather than removing it. They also create a need for supporting mechanisms that validate package recommendations and provide current ecosystem

information independently of the generated response.

The literature highlights challenges across several connected stages of dependency management. Developers must understand which dependencies are relevant to production, evaluate packages introduced during development, and maintain those packages as they evolve. At each stage, incomplete context can make apparently simple dependency-management tasks difficult to prioritize or execute reliably. Emerging LLM-based development practices create new opportunities to support these activities, while also introducing new questions about package validity, integration support, repository grounding, and the reliability of generated changes.

To better understand these challenges, this thesis conducts a series of empirical studies spanning development, production, and maintenance. First, it examines the dependencies installed in JavaScript projects and determines which of them are ultimately included in production, providing insight into the limitations of existing dependency classifications and the prioritization of dependency-related activities. Second, it investigates the packages introduced through LLM-generated Python code, including their characteristics and the practical support available for their integration. It then presents a development tool designed to provide additional information about packages introduced through AI-assisted programming. Finally, the thesis studies whether LLMs can support the maintenance of dependencies already integrated into a project by generating applicable and semantically correct dependency-update patches.

Together, these studies examine how dependencies are represented, introduced, used, and modified across the software lifecycle. Our research aims to identify where current practices and tools provide insufficient context, assess how emerging development approaches interact with these limitations, and inform more reliable support for dependency management.

1.2 Research Statement

Motivated by the challenges developers face when introducing, using, and maintaining software dependencies, the goal of this Ph.D. thesis is to better understand dependency management across the software lifecycle. In particular, this thesis examines the relevance of dependencies in production, the packages introduced through LLM-generated code, practical support for their integration, and the reliability of LLM-generated dependency-update patches.

Given the complexity of software dependency management, this thesis aims to understand and support how dependencies are introduced, used, and maintained across the software lifecycle, while determining whether emerging LLM-based development practices alleviate existing challenges or introduce additional risks and effort.

1.3 Thesis Overview

In this section, we provide an overview of the work presented in this thesis and highlight the main results of each work.

Chapter 3: Not All Dependencies are Equal: An Empirical Study on Production Dependencies in NPM

Software projects often install large numbers of direct and transitive dependencies, but not all of these packages are ultimately included in the software released to users. Project manifests distinguish between runtime and development dependencies, yet these classifications do not necessarily reflect how dependencies are actually used in production. This makes it difficult for developers to determine which packages should be prioritized when reviewing updates, vulnerabilities, and other maintenance concerns. To better understand the production relevance of software dependencies, we conduct an empirical study of 100 JavaScript projects from the npm ecosystem. We identify the dependencies installed in each project, determine which of them are included in production releases, and examine whether their declared dependency type accurately represents their production use. We also analyze how production inclusion affects the relevance of dependency-related security alerts. Our findings show that less than 1% of installed dependencies are included in production, while many runtime dependencies are absent from production and some development dependencies are nevertheless included. These results demonstrate that not all dependencies are equally relevant and that production use provides important context for dependency-management and prioritization decisions. This work has been published at the 37th IEEE/ACM International Conference on Automated Software Engineering ([17]).

Chapter 4: How Robust are LLM-Generated Library Imports? An Empirical Study using Stack Overflow

The preceding chapter examined how dependencies already present in software projects differ in their relevance to production. However, the mechanisms through which dependencies enter projects are evolving as developers increasingly rely on LLMs to generate code and recommend libraries. Although these systems can reduce the effort required to identify reusable functionality, their recommendations may still create integration challenges if packages are invalid, difficult to resolve, or introduced without sufficient installation guidance. To examine these challenges, we conduct an empirical study of six proprietary and open-source LLMs using real-world Python programming problems from Stack Overflow. We analyze the libraries imported by the generated solutions, the characteristics of those libraries, and whether the recommendations can be used without additional intervention. Our findings show that LLMs generally recommend mature, popular, and permissively licensed third-party libraries. However, some imports cannot be automatically mapped to their installable packages, and installation guidance is rarely provided. These results show that LLMs can facilitate package discovery while leaving developers responsible for resolving and integrating the resulting dependencies. This work has been submitted to the journal of IEEE Transactions on Software Engineering and is currently under review following a major revision ([25]).

Chapter 5: The Software Librarian: Python Package Insights for Copilot

The preceding chapter showed that LLM-generated code may introduce dependencies without providing sufficient information to support their reliable integration. Developers must still verify whether a recommended package exists, identify its installable distribution, and assess characteristics such as licensing, deprecation status, dependencies, and maintenance health. To support these activities, we introduce the Software Librarian, a Visual Studio Code extension that provides real-time information about Python packages introduced through GitHub Copilot code completions. The tool classifies packages as standard, third-party, or other, resolves cases involving aliases, importable resources, and placeholders, and retrieves relevant package metadata from external sources. We evaluate the Software Librarian using a curated dataset of 100 packages and find that it correctly classifies 99% of them. These results demonstrate how complementary, ecosystem-aware tooling can provide developers with additional context when evaluating and integrating packages introduced through AI-assisted programming. This work has been published in Proceedings of the 2025 IEEE/ACM

47th International Conference on Software Engineering: Companion Proceedings ([27]).

Chapter 6: Evaluating Large Language Models for Dependency-Maintenance Patch Generation

The preceding chapters examined dependencies as they are introduced into software projects and the support needed to integrate them reliably. Once a dependency has been adopted, however, it must be maintained as new versions, security fixes, and breaking changes are released. Dependency updates may require more than changing a version number: the resulting modification must apply to the current project state and may require corresponding adaptations to source code or tests. To evaluate whether LLMs can support this task, we mine 9,908 dependency-update pull requests from 423 JavaScript repositories and construct a paired benchmark of 56 updates from 46 repositories. We evaluate four instruction-tuned LLMs under manifest-only and broader maintenance settings. Our findings show that models often identify the intended dependency update, but frequently fail to express it as an applicable and correctly grounded patch. Providing related source or test files further reduces manifest-level reliability, while applicable source-code adaptations remain rare. These results suggest that LLMs are better suited to supporting targeted maintenance reasoning and source adaptation within staged workflows where repository-aware tools perform structured edits, validation, and final patch construction. This work has been submitted to the journal of ACM Transactions on Software Engineering and Methodology ([28]).

1.4 Thesis Contributions

The main contributions of this thesis are as follows:

- **An empirical characterization of dependencies used in production.** We study 100 JavaScript projects from the npm ecosystem to quantify which installed dependencies are ultimately included in production releases. The study shows that conventional runtime and development classifications do not reliably represent production use and demonstrates how production awareness can support the prioritization of dependency-maintenance activities and security alerts.
- **An empirical evaluation of dependencies introduced through LLM-generated code.** We evaluate six proprietary and open-source LLMs using real-world Python programming problems from Stack

Overflow. We characterize the libraries imported by the generated solutions and assess their usability, including package resolvability and the availability of installation guidance. The study identifies both the strengths of LLM library recommendations and the integration effort that remains for developers.

- **Practical tool support for evaluating LLM-introduced dependencies.** We design and implement the Software Librarian, a Visual Studio Code extension that provides real-time information about Python packages introduced through GitHub Copilot code completions. The tool classifies packages, resolves aliases and importable resources, identifies potential placeholders, and reports information related to package validity, licensing, deprecation, dependencies, and maintenance health. Its package-classification approach achieves 99% accuracy on a curated dataset of 100 packages.

- **A benchmark and empirical evaluation of LLM-generated dependency-update patches.** We mine 9,908 dependency-update pull requests from 423 JavaScript repositories and construct a paired benchmark of 56 updates from 46 repositories. Using this benchmark, we evaluate four instruction-tuned LLMs under two maintenance settings and across two prompting strategies. The study distinguishes semantic understanding of the requested dependency update from patch applicability and repository grounding, identifies common failure modes, and motivates staged workflows that combine LLM reasoning with repository-aware editing and validation tools.

1.5 Related Publications

The work presented in this thesis has been published in or submitted to the following venues:

- **Jasmine Latendresse**, Suhaib Mujahid, Diego Elias Costa, and Emad Shihab. Not All Dependencies Are Equal: An Empirical Study on Production Dependencies in npm, in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 1–12, 2022. DOI:10.1145/3551349.3556896.

- **Jasmine Latendresse**, SayedHassan Khatoonabadi, and Emad Shihab. How Robust Are LLM-Generated Library Imports? An Empirical Study Using Stack Overflow, manuscript under review after a major revision at *IEEE Transactions on Software Engineering*, 2026.

- **Jasmine Latendresse**, Nawres Day, SayedHassan Khatoonabadi, and Emad Shihab. The Software

Librarian: Python Package Insights for Copilot, in *Proceedings of the 2025 IEEE/ACM 47th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, pp. 17–20, IEEE, 2025. DOI:10.1109/ICSE-Companion66252.2025.00014.

- **Jasmine Latendresse** and Emad Shihab. Evaluating Large Language Models for Dependency-Maintenance Patch Generation, manuscript submitted to *ACM Transactions on Software Engineering and Methodology*, 2026.

1.6 Thesis Organization

The rest of the thesis is organized as follows. Chapter 2 reviews the related work to our thesis. Next, Chapter 3, Chapter 4, Chapter 5, and Chapter 6 present the details and results of our empirical studies that investigate the research goal of this thesis. Finally, we summarize our findings and suggest directions for future work in Chapter 7.

Chapter 2

Background and Related Work

In this chapter, we provide the background necessary to understand our work on dependency management. We first describe the concept of software ecosystems and, using the npm ecosystem as a running example, explain how packages are distributed, connected through dependencies, and managed throughout the software lifecycle. We then review the main challenges associated with dependency selection, production use, and maintenance, as well as the tools and practices developed to support these activities. Finally, we introduce large language models and examine their growing role in software development and maintenance, with particular attention to how they recommend, introduce, and modify software dependencies.

2.1 Background

2.1.1 Software Packaging Ecosystems and Dependencies

We study software ecosystems in the context of package libraries for programming languages. In this setting, a software ecosystem consists of reusable software functionality that is developed, shared, and maintained through a common package-management infrastructure [29]. Package managers support activities such as publishing, discovering, installing, updating, and removing packages, while package registries provide the repositories through which these packages are distributed. Popular examples include npm for JavaScript, the Python Package Index (PyPI) for Python, Maven Central for Java, and RubyGems for Ruby.

Each software ecosystem follows its own conventions and practices for declaring dependencies, resolving package versions, organizing releases, and managing updates. Consequently, dependency-management behaviour may differ across programming languages and package managers. In this chapter, we use npm as

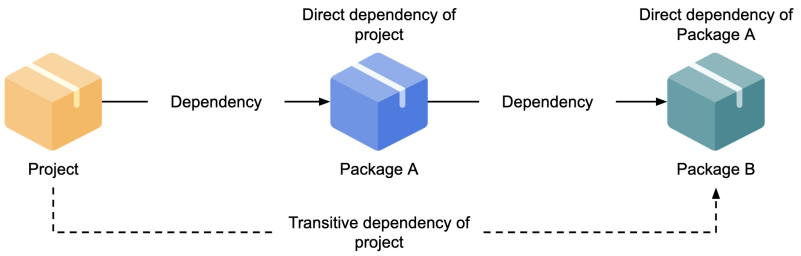


Figure 2.1: Example of direct and transitive dependency relationships.

a running example because it is the world's largest software registry and has historically exhibited one of the highest package-growth rates among software ecosystems [30, 31]. Its scale and highly interconnected dependency network make it a particularly relevant setting for illustrating the dependency-management concepts and challenges examined in this thesis.

2.1.2 Dependency Management

Dependencies evolve independently from the projects that use them. Package maintainers release new versions to introduce features, fix defects and vulnerabilities, or modify existing interfaces, while dependent projects must decide which versions to adopt and when to update them. Dependency management therefore encompasses the practices and mechanisms used to declare, install, configure, monitor, and update the external packages on which a software project relies.

A dependency relationship may be either *direct* or *transitive*. A direct dependency is explicitly selected and declared by a project, whereas a transitive dependency is introduced through another dependency. As illustrated in Figure 2.1, if a project declares package A as a dependency and package A depends on package B, then A is a direct dependency of the project and B is a transitive dependency. The project may also be described as a *downstream* dependent of package A, while package A is an *upstream* dependency of the project. Because transitive dependencies may themselves depend on additional packages, adding a single direct dependency can introduce a substantially larger dependency tree.

In npm projects, dependencies are declared in the `package.json` manifest, which also stores project metadata such as its name, version, and scripts. As shown in Listing 2.1, packages listed under `dependencies` are generally required for the application to execute (runtime), whereas packages under `devDependencies` support development activities such as testing, compilation, and linting. npm also supports `optionalDependencies`, whose installation failure does not prevent the remaining project dependencies from being

installed.

Listing 2.1: Example of dependency declarations in an npm `package.json` manifest.

```
{
  "name": "example-project",
  "version": "1.0.0",
  "dependencies": {
    "express": "^4.21.0"
  },
  "devDependencies": {
    "eslint": "^9.0.0"
  },
  "optionalDependencies": {
    "fsevents": "^2.3.3"
  }
}
```

Each dependency declaration associates a package name with a version constraint that defines the releases npm may select. When the developer runs `npm install`, npm resolves these constraints, retrieves the corresponding direct and transitive dependencies, and constructs the complete dependency tree. The resolved versions are recorded in `package-lock.json`, together with information needed to reproduce the installation. The manifest therefore expresses the project's dependency requirements, while the lockfile records the concrete dependency graph selected for a particular project state.

These mechanisms simplify package installation and reuse, but they also create an ongoing relationship between projects and independently evolving upstream packages. Developers must decide which dependencies to adopt, determine how broadly their versions may change, and respond when new releases introduce fixes, vulnerabilities, or breaking changes [10].

2.2 Related Work

In this section, we review the literature most closely related to this thesis. We first examine prior studies on key dependency-management challenges across software ecosystems. We then review related work on dependency bloat and automation in dependency management. Finally, we explain how this thesis extends

and complements the existing literature.

2.2.1 Challenges in Dependency Management

The rapid growth of software packaging ecosystems has resulted in increasingly large and interconnected dependency networks, making dependencies difficult to understand and manage. Decan et al. [2] compared the evolution of dependency networks across seven software packaging ecosystems and found that these networks grew over time in both size and update activity. Their results also showed that dependency relationships were unevenly distributed: although most packages relied on other packages, a relatively small proportion of packages accounted for the majority of downstream dependents. Moreover, projects commonly had few direct dependencies but substantially larger transitive dependency trees, increasing their exposure to changes and failures originating from upstream packages. In a systematic literature review, Mens and Decan [10] characterize the resulting problems as a broad set of dependency-management challenges, including conflicts, incompatibilities, breaking changes, outdated packages, deprecations, and security vulnerabilities. The studies reviewed by the authors show that these challenges become more difficult to manage as software ecosystems grow and increasingly affect software maintainability, security, and evolution.

One of the most prevalent dependency-management challenges is the continued use of outdated dependencies. Kula et al. [14] found that 81.5% of the examined software projects relied on outdated dependencies, potentially exposing them to security and compatibility issues. The concept of *technical lag* has been introduced to quantify the degree to which dependency networks fall behind the latest available releases, using dimensions such as time lag, version lag, and security lag [32, 33]. This line of research has also been extended beyond package registries. Lauinger et al. [34] found that the majority of websites relying on JavaScript libraries used at least one dependency that was one or more patch releases behind. Studies of Docker images and Helm charts similarly found that more than half of the container images used in Helm charts were outdated and that many remained exposed to known vulnerabilities [35, 36].

Several factors contribute to the persistence of outdated dependencies. Brito et al. [37] and Bogart et al. [38] show that dependency updates may require modifications to surrounding code, extensive testing, and additional effort to address incompatibilities. Developers must therefore balance the benefits of receiving bug fixes, security patches, and new functionality against the risk that an update will introduce breaking changes or disrupt otherwise stable software.

The challenge of outdated dependencies is further compounded by package abandonment. Miller et

al. [39] found that more than 100,000 projects in the npm ecosystem depend on abandoned packages. These dependencies may remain in use because developers are unaware that maintenance has ceased or because replacing them requires substantial migration effort. This problem reflects a broader sustainability challenge in open-source software, where many widely reused packages depend on a small number of volunteer maintainers with limited time, funding, and infrastructure [40]. Zerouali et al. [16] showed that unmaintained dependencies can prolong exposure to known vulnerabilities in ecosystems such as npm and RubyGems because affected packages may no longer receive fixes. In response, Community Package Maintenance Organizations (CPMOs) have emerged as a collaborative model in which volunteers assume responsibility for packages whose original maintainers are no longer able to support them [41].

Reliance on outdated and unmaintained dependencies can leave projects exposed to known vulnerabilities even after fixes become available. Studies across npm, Maven, and PyPI have shown that many projects continue to use vulnerable dependency versions despite the availability of patched releases [15, 34, 42]. Zerouali et al. [16] found that vulnerabilities in npm packages are often fixed relatively quickly by upstream maintainers, while downstream projects remain exposed because they are slow to adopt the corrected versions. Restrictive version constraints, technical lag, and reliance on unmaintained packages can all delay the propagation of these fixes. Similarly, Lauinger et al. [34] observed that 37% of websites using JavaScript libraries included at least one library with a known vulnerability.

Transitive dependencies further complicate vulnerability management because developers may be indirectly exposed to packages they did not select themselves and may not be aware of. Chinthanet et al. [42] showed that transitive dependencies propagate both vulnerabilities and technical lag through dependency networks, making affected components more difficult to identify and update. Liu et al. [43] similarly found that transitive dependencies contribute substantially to the vulnerability exposure of npm projects and that vulnerable packages can affect large numbers of downstream dependents. These findings show that vulnerability management cannot be limited to direct dependencies, but must account for the broader dependency tree and the rate at which security fixes propagate through it.

The scale and interconnectedness of these dependency networks also make software packaging ecosystems attractive targets for supply-chain attacks. Rather than exploiting a vulnerability in an individual application, attackers may compromise a package, maintainer account, or distribution mechanism to reach many downstream projects simultaneously [44, 45, 46]. For example, dependency-confusion attacks exploit differences between internal and public package-resolution mechanisms to cause a malicious public package to be installed in place

of an intended private dependency [47]. Ohm et al. [45] also documented malicious packages in npm, PyPI, and RubyGems that used deceptive names and installation scripts to steal credentials or execute unauthorized code. These attacks illustrate how dependency-management risks can extend beyond outdated software to the mechanisms through which packages are identified, retrieved, and integrated.

To conclude on dependency-management challenges, prior work shows that ecosystem growth increases reliance on transitive dependencies, while outdated and abandoned packages delay the adoption of fixes and expand vulnerability exposure. At the same time, developers often have limited visibility into which dependencies are present, why they were introduced, and how relevant they are to the resulting software. This limited visibility complicates efforts to prioritize updates, vulnerabilities, and other maintenance activities. This thesis complements this work by shifting attention from the presence of dependencies to their relevance: whether dependencies are actually used, whether they contribute to the delivered software, and whether they are introduced or maintained in ways that developers can reason about. This perspective is important because a dependency can increase maintenance effort or vulnerability exposure even when it provides little or no value to the projects actual execution context. The next subsection therefore focuses on dependency bloat, a specific manifestation of this broader visibility problem, where projects retain or distribute dependencies that are unnecessary for their actual use.

2.2.2 Dependency Bloat

The complexity of modern dependency networks discussed in the previous section can result in projects declaring, installing, and maintaining packages that are not required by the software. Jafari et al. [13] examine this problem as part of a broader catalog of dependency smells in JavaScript projects. Among the seven smells identified in their study, unused dependencies describe packages that remain declared despite no longer being referenced by the project. Their analysis of 1,146 JavaScript repositories shows that dependency smells are prevalent and tend to accumulate over time, indicating that unnecessary dependency declarations are part of a broader maintenance problem.

Soto-Valero et al. [48] introduce the concept of *bloated dependencies* to describe libraries that are included in Maven artifacts but are not necessary to build or execute the application. They develop DepClean, a static-analysis tool that examines dependency usage in Java bytecode, and apply it to 9,639 Maven artifacts containing 723,444 dependency relationships. They find that 75.1% of the analyzed relationships are bloated. The authors also submitted dependency-removal patches to open-source projects, with 18 of the 21 answered

pull requests being accepted, suggesting that developers are willing to remove dependencies when provided with sufficient evidence that they are unnecessary. In a longitudinal extension of this work, Soto-Valero et al. [49] examine 48,469 dependencies across 31,515 versions of 435 Java projects. They find that bloated dependencies accumulate over time and that 89.02% of directly bloated dependencies remain bloated in subsequent versions. Furthermore, 22% of dependency updates performed by developers target bloated dependencies, with Dependabot recommending updates at a similar rate. These results show that dependency bloat creates not only larger artifacts but also unnecessary maintenance effort.

Artifact-level analyses are especially important in JavaScript applications, where the dependencies declared in configuration files do not necessarily correspond to the packages that are ultimately shipped to users. Web applications often rely on bundlers that merge first-party code with third-party npm packages into production JavaScript bundles. Rack and Staicu [50] study this phenomenon at scale and show that bundles are common on the web, appearing on 40% of crawled websites. Their work also shows that bundles can be partially reverse engineered and that source maps, when available, expose module names and can reveal the libraries included in bundled artifacts. Using this information, they identify thousands of npm packages in bundled code and show that vulnerable packages can be present in production web artifacts. While their work focuses on supply-chain security rather than dependency bloat, it supports the methodological point that build artifacts provide visibility into which dependencies actually reach deployed software.

More recent work demonstrates that dependency bloat is not limited to Maven. Liu et al. [51] propose a trace-based dynamic analysis for detecting unused dependencies in server-side CommonJS packages. Across 91 packages containing 50,488 dependencies, they identify 50.6% as bloated, with bloat occurring particularly frequently among transitive dependencies. Related studies have also examined fine-grained dependency bloat in Python projects and the continuous integration resources consumed by updates to unused npm dependencies [52, 53].

Overall, these studies show that dependency bloat occurs across ecosystems and can be observed at multiple levels, including dependency declarations, resolved dependency trees, runtime traces, and distributed artifacts. This bloat can increase artifact size, maintenance effort, security exposure, and the volume of unnecessary dependency-update activity.

2.2.3 Automation in Dependency Management

The scale and interconnectedness of software dependency networks make manual dependency management increasingly difficult. The challenges described in the preceding sections have therefore motivated automation throughout the dependency lifecycle, from selecting packages and constructing dependency inventories to identifying vulnerabilities, removing unnecessary dependencies, and generating updates. Early approaches primarily relied on repository metadata, dependency graphs, source-code analysis, and predefined rules. More recently, LLM-based tools have extended automation to tasks that require code generation, natural-language reasoning, and project modification. In the following, we review these approaches and the stages of dependency management they support.

Package Recommendation and Selection Support

Dependency management starts at the very first stage of library selection. The large number of packages available in software ecosystems makes it difficult for developers to identify suitable dependencies for their projects. Researchers have therefore proposed recommender systems that use dependency relationships, library co-usage patterns, source-code similarity, and the choices made by comparable projects to suggest potentially relevant packages. Thung et al. [54] introduced LibRec, which combines association-rule mining and collaborative filtering to recommend libraries based on those already used by a project and the dependencies of similar projects. However, such approaches depend heavily on historical co-usage information, and may struggle when projects have sparse dependency histories or when frequent co-adoption does not necessarily imply contextual suitability.

Subsequent work incorporated additional sources of information into the recommendation process. Ouni et al. [55] proposed LibFinder, a search-based approach that balances library co-usage, semantic similarity between candidate libraries and project source code, and the number of recommended libraries. The approach was evaluated using 6,083 Maven libraries and 32,760 client systems and showed that combining usage history with semantic information can improve package recommendation. Saied et al. [56] later introduced LibCUP, which mines multi-level library co-usage patterns to identify groups of libraries commonly adopted together. Their evaluation covered more than 6,000 Maven libraries and 38,000 GitHub projects. While co-usage patterns can reveal common library stacks, they may also capture correlations caused by templates, trends, or incidental adoption rather than true compatibility or necessity.

Nguyen et al. [57] proposed CrossRec, which uses collaborative filtering to identify projects with similar dependency profiles and recommend additional libraries. Their evaluation showed that CrossRec outperformed LibRec, LibFinder, and LibCUP across several recommendation-quality measures. However, collaborative filtering remains sensitive to the availability and representativeness of comparable projects, and can inherit the biases of existing dependency choices. Other systems address a related selection problem by recommending alternatives to an existing technology. For example, SimilarTech mines developer discussions to identify analogous or competing libraries, including libraries from different programming languages [58]. Such discussion-based approaches can surface useful alternatives, but they depend on the quality and coverage of community discussions and are not necessarily grounded in the concrete dependency context of a specific project.

Overall, this line of work shows that dependency recommendation predates LLM-based programming assistance. However, traditional recommenders generally operate as dedicated tools that rank candidate packages using structured evidence collected from repositories, ecosystem metadata, or developer discussions. Their output is typically a recommendation list that developers inspect before deciding whether and how to adopt a package. LLMs introduce a different interaction model, in which the usual sequence of dependency discovery, ranking, developer evaluation, and adoption can be collapsed into a single generated response. Rather than explicitly presenting a ranked set of candidate libraries, an LLM may introduce a dependency directly through an import statement, installation command, or code example, with the choice learned from patterns during training and by the immediate prompt context. In this thesis, we study this shift by examining not only which packages are suggested by LLMs, but also whether the resulting imports are valid, resolvable, and accompanied by sufficient information for developers to integrate them correctly.

Dependency Representation and Visibility

Package manifests and lockfiles provide the most common representations of a project's dependencies. In npm, the `package.json` manifest records the direct dependencies selected by developers and the version constraints associated with them. The `package-lock.json` file provides a more detailed view by recording the concrete versions selected during dependency resolution, including transitive dependencies. Lockfiles therefore offer substantially greater visibility into the packages installed for a particular project state than manifests alone. However, they describe the result of dependency resolution rather than how the installed packages are ultimately used. Their contents may also vary when the package-manager version, platform,

configuration, or treatment of optional dependencies changes. Consequently, neither manifests nor lockfiles necessarily provide a stable or complete representation of the dependencies that contribute to the resulting software artifact.

This limitations is not only structural, but also practical. Even when package managers provide mechanisms for recording, auditing, and cleaning dependencies, developers do not always adopt them consistently, and the resulting contents of those files can be difficult to interpret or might not reveal relevant, practical information. For example, Kabir et al. [59] studied 841 popular JavaScript applications and found that only 32% specified exact dependency versions, while many projects lacked committed lockfiles or allowed dependencies to vary across installations. They also found that existing tools reported vulnerable, unused, and duplicated dependencies in many applications, but that such reports were not always actionable: automated fixes often failed to remove all issues, and developers sometimes rejected tool outputs as false positives or unimportant. These findings suggest that manifests, lockfiles, and package-manager analyses provide important visibility into dependency declarations and resolved trees, but they remain imperfect proxies for dependency relevance, security impact, and actual use.

Software bills of materials (SBOMs) have been proposed as a more structured mechanism for documenting the components contained in software. An SBOM can record information such as package names, versions, licenses, suppliers, and known vulnerabilities, thereby supporting dependency tracking, security analysis, and compliance activities. Nocera et al. [60] examined SBOM adoption in open-source GitHub projects and found that its use remains limited, although adoption is gradually increasing. Prior work has also shown that SBOM-based information can support vulnerability detection and license-compliance analysis [61]. More recently, Bhuiyan et al. [62] studied the completeness and consistency of GitHub-generated SBOMs across 10,000 repositories from ten programming-language ecosystems, comparing GitHub SBOMs generated by Syft, Trivy, and the Microsoft SBOM Tool. Their results show that GitHub SBOMs provide useful dependency metadata, including component names, identifiers, dependency relationships, authorship metadata, and timestamps, but that no GitHub-generated SBOM fully satisfied the NTIA minimum requirements because supplier information was consistenly missing. They also found that version and license information vary substantially across ecosystems and tools, and that vulnerability tracking depends strongly on the availability of precise identifiers and exact version information. The contents of an SBOM depend on the tool, generation method, build context, and available project metadata. Different tools may therefore produce incomplete or inconsistent inventories for the same project, particularly when transitive, dynamically loaded, or build-time

dependencies are involved. Recent evidence further suggests that SBOM generators can differ in the number of components they report, the metadata they provide, and their usefulness for downstream vulnerability, but they should be understood as partial tool-dependent inventories rather than complete evidence of how dependencies are used by a project. As a result, SBOMs improve dependency visibility, but they should be understood as partial and tool-dependent inventories rather than complete evidence of how dependencies are used by a project.

These representations improve visibility into declared, resolved, or packaged components, but they do not necessarily establish whether a dependency is imported, executed, reachable, or required in production. This distinction limits their ability to support prioritization when developers must determine which dependencies, updates, or vulnerabilities are operationally relevant. As a result, dependency inventories often need to be complemented by static analysis, dynamic analysis, build-artifact inspection, or other forms of project-specific evidence.

Vulnerability and Compatibility Analysis

Software Composition Analysis (SCA) tools automate the identification of third-party components and match their versions against databases of known vulnerabilities. Common examples include OWASP Dependency-Check, Snyk, and Dependabot. Mohayeji et al. [63] investigated the adoption of SCA tools in open-source projects and found that projects using these tools reduced their exposure to vulnerable dependencies. However, vulnerability analysis based primarily on package names and versions may report dependencies that are not exercised by the application or included in its deployed artifact. Developers must therefore interpret the resulting alerts and determine which reported vulnerabilities are relevant to their software.

Prior work has proposed more detailed dependency representations to improve vulnerability analysis and prioritization. Liu et al. [43] introduced DTreme, which constructs dependency-vulnerability graphs to identify how vulnerable packages propagate through direct and transitive dependency relationships. Such approaches provide more context than flat vulnerability inventories by showing which upstream packages introduce a vulnerable component and how broadly the issue may affect downstream projects. Nevertheless, the presence of a vulnerable package in a dependency graph does not necessarily establish that the affected functionality is reachable or used in practice, leaving developers responsible for assessing its operational impact.

Compatibility analysis tools address a related challenge by identifying changes that may disrupt dependent projects. Semantic versioning provides a convention through which package maintainers communicate whether a release introduces major, minor, or patch-level changes. However, prior work has shown that package releases do not always comply with these expectations, and breaking changes may occur outside major releases [64]. Tools such as PyCompat [65], AexPy [66], and API-Diff analyze package interfaces, source code, or test suites to detect potentially incompatible API changes before an update is adopted.

These approaches improve developers' ability to identify vulnerable and incompatible dependencies, but they remain limited by the information available to the analysis. Vulnerability tools depend on the completeness and accuracy of component inventories and vulnerability databases, while compatibility tools may not capture behavioural changes, dynamically used APIs, or project-specific assumptions. Moreover, detecting a vulnerability or breaking change does not perform the corresponding remediation. Developers must still decide whether an update is necessary, modify the relevant dependency declaration, adapt the surrounding code where required, and validate the resulting project state.

Automated Dependency Updates

While vulnerability and compatibility tools identify dependencies that may require attention, automated dependency-update tools perform part of the remediation process. Tools such as Dependabot and Renovate monitor package releases and generate pull requests that modify dependency declarations and lockfiles. Dependabot is integrated into GitHub and can generate both version and security updates, whereas Renovate supports multiple development platforms and package ecosystems and provides extensive configuration options for scheduling, grouping, and managing updates [10, 20].

Alfadel et al. [67] examined 15,243 Dependabot security pull requests submitted to 2,904 JavaScript projects. They found that 65.42% of the pull requests were merged, often within one day, indicating that developers were generally receptive to automated security updates. Their manual analysis showed that most non-merged pull requests were closed because the affected dependencies had been modified concurrently rather than because Dependabot had generated an incorrect update. Only 3.2% of the manually examined pull requests resulted in build failures. However, the severity of the vulnerability and the potential risk of a breaking change were not strongly associated with how quickly an update was merged, suggesting that developer response is also influenced by project- and workflow-specific factors.

He et al. [68] studied a broader set of 502,752 Dependabot pull requests from 1,823 actively maintained

repositories and surveyed 131 developers involved in reviewing them. They found that Dependabot substantially reduced technical lag: within 90 days of its adoption, the average lag decreased from 73.68 to 25.38 days, while the median decreased from 16.27 to 3.62 days. Developers were generally receptive to the generated updates, with more than 70% of pull requests being merged. Nevertheless, many projects later reduced the frequency of update checks or limited the number of simultaneously open pull requests. The authors identified notification fatigue, limited support for grouped updates, and incompatibilities with some package managers as common reasons for discontinuing Dependabot.

The continued adoption of alternative bots indicates that these limitations do not eliminate the perceived value of dependency-update automation. He et al. [68] observed that most projects that discontinued Dependabot migrated to another tool, frequently Renovate. Renovate provides configurable update schedules, dependency grouping, shared configuration presets, and support for updating both manifests and lockfiles. These capabilities allow developers to reduce the volume of individual pull requests and adapt the bot's behaviour to their maintenance workflow [20].

Overall, automated update bots are effective at monitoring releases, reducing technical lag, and modifying structured dependency declarations. However, they can also generate substantial review overhead, and their effectiveness depends on project configuration, available tests, and developer responsiveness. Moreover, these tools provide limited support when an update requires project-specific source-code adaptations or when developers must determine whether the updated dependency is sufficiently relevant to justify the associated review effort.

LLMs in Dependency Management

Traditional dependency-management tools are effective at performing well-defined operations on manifests, lockfiles, release metadata, and vulnerability databases. However, they provide limited support when a dependency-related task requires interpreting natural-language requirements, reasoning about project context, or modifying surrounding source code. The growing use of large language models in software development has therefore motivated research into whether these models can support package selection, integration, and maintenance.

LLMs introduce a new interface through which dependencies can enter software projects. Rather than selecting a package through a dedicated recommendation system or package registry, developers may receive a library recommendation as part of a generated explanation or code completion. Prior work has shown that these

recommendations can be useful, but may also include nonexistent, outdated, or incorrectly named packages and may omit information required for installation or integration [69, 25]. Unlike traditional recommendation systems that rank packages using current ecosystem metadata, LLMs generate recommendations from patterns learned during training and may therefore lack access to the latest package releases, licensing information, maintenance status, or deprecation notices. Consequently, generated code may appear plausible while still requiring developers to validate the identity and suitability of the introduced dependency.

Other studies examine the use of LLMs after a dependency has already been integrated. Almeida et al. [70] investigate the use of LLMs for API-version migration, while Islam et al. [71] evaluate LLMs on real-world migrations between Python libraries. Ziftci et al. [72] further demonstrate an LLM-assisted workflow for large-scale code migrations at Google. Related approaches provide models with compiler errors, API differences, migration documentation, or execution feedback to repair code affected by breaking dependency updates [73, 74]. These studies show that LLMs can support dependency-related reasoning and code transformation, particularly when they receive structured information about the update and its effects. However, the resulting workflow often depends on external tools to retrieve documentation, identify affected code, apply changes, and validate the modified project.

More recently, agentic software-engineering systems have emerged as a distinct way of using LLMs in development workflows. These systems can inspect repositories, retrieve information, modify files, and execute builds or tests iteratively [75, 76]. However, agentic coding is still an emerging paradigm and does not yet represent all, or necessarily most, developer interactions with LLMs. Prior work shows that developers commonly use ChatGPT and similar tools in lightweight settings, such as pull-request and issue discussions, where they request explanations, guidance, or direct code suggestions rather than delegating complete tasks to autonomous agents [77, 78, 79].

Agentic systems may provide richer repository context and opportunities to revise unsuccessful changes through execution feedback, but they also introduce a different set of design and evaluation concerns. Their performance depends not only on the underlying LLM, but also on how tasks are decomposed, which tools are available, how repository actions are performed, and how intermediate outputs are validated. Agentic dependency management should therefore be considered complementary to, rather than a replacement for, evaluations of direct LLM assistance. Characterizing how different agent architectures affect package selection, dependency updates, and source-code adaptation remains an important direction for future work.

2.3 Positioning of the Thesis

Prior work has established that software packaging ecosystems are large, rapidly evolving, and highly interconnected. Researchers have examined dependency selection, technical lag, breaking changes, vulnerabilities, package abandonment, dependency bloat, and automated update tools. More recent studies have also investigated how LLMs recommend libraries, migrate APIs, and assist with software-maintenance tasks. However, these research areas are often considered independently, even though they concern different stages of the same dependency lifecycle.

At the production stage, existing work has shown that projects frequently contain unused or bloated dependencies and has proposed static and dynamic techniques for identifying unnecessary packages. Nevertheless, dependency manifests and resolved dependency graphs do not necessarily indicate which installed packages are ultimately included in released software. This thesis examines this distinction in npm projects by comparing installed dependencies with production bundles and by assessing whether conventional runtime and development classifications accurately represent production use. This provides additional context for understanding dependency relevance and prioritizing maintenance activities and security alerts.

At the development stage, traditional recommendation systems assist developers in selecting libraries using repository data, ecosystem metadata, and package characteristics. LLMs introduce a different interaction model in which dependencies may appear directly within generated explanations or code. Prior work has identified risks such as nonexistent or outdated package recommendations, but less attention has been given to the broader characteristics and immediate usability of the libraries introduced by LLM-generated code. This thesis evaluates the robustness of generated imports, including their resolvability and accompanying installation support, and presents the Software Librarian as an initial mechanism for validating package identities and providing current ecosystem information during integration.

Finally, prior dependency-update bots are effective at detecting new releases and modifying structured dependency declarations, while recent LLM-based approaches have explored API migration and code adaptation using documentation, compiler feedback, or iterative execution. This thesis complements that work by examining an earlier and fundamental step in the maintenance workflow: whether an LLM can generate a dependency-update patch that applies to the supplied project state and performs the requested change correctly. It further evaluates how reliability changes when the task expands beyond manifest editing to include related source or test files.

These studies position dependency management as a continuous activity spanning development, production, and maintenance. The thesis connects established dependency-management challenges with emerging LLM-based development practices and examines where additional production-aware, ecosystem-aware, and repository-aware support is required. In doing so, it contributes both empirical evidence about how dependencies are introduced, used, and updated and practical guidance for designing more reliable dependency-management workflows.

Chapter 3

Not All Dependencies are Equal: An Empirical Study on Production Dependencies in NPM

3.1 Chapter Overview

Third-party packages are widely used to accelerate software development, but their inclusion also creates long-term maintenance and security responsibilities. Before examining how large language models affect dependency management, it is first necessary to understand how dependencies are used in conventional software projects and, in particular, which dependencies ultimately reach production.

This chapter establishes that foundation by examining the production dependency footprint of JavaScript projects. It investigates how many installed dependencies are included in production releases, whether their declared configuration accurately reflects their production use, and how production use affects the interpretation of dependency-related security alerts. The findings show that only a small proportion of installed dependencies are released to production and that commonly used dependency classifications do not reliably distinguish production from non-production dependencies. This demonstrates that dependency management is more nuanced than the contents of a project manifest alone suggest and that dependencies should be prioritized according to their actual role in deployed software.

By establishing the complexity of dependency use before the widespread adoption of LLM-based

programming assistants, this chapter provides the baseline for the remainder of the thesis. The following chapters examine whether LLMs help developers navigate these existing challenges or introduce additional complexity into dependency selection, integration, and maintenance.

3.2 Introduction

The vast majority of modern software systems are built by using modular functionalities provided by open source packages. Reports estimate that more than 90% of open source and proprietary projects rely substantially on reusing open source packages [80, 4]. As a testament to the popularity of open source, popular package managers such as npm, host more than 2 million reusable packages, covering all sorts of software functionalities [2].

While the use of open source packages significantly reduces development time and costs [81, 3, 1], it also exposes software applications to vulnerabilities. In the 2020 State of the Octoverse security report, GitHub reveals that active repositories with a supported package ecosystem have a 59% chance of getting a security alert in the next 12 months [80]. This problem is even more widespread in the JavaScript ecosystem, where nearly 40% of all npm packages rely on code with known vulnerabilities [82]. Software vulnerabilities may lead to significant financial and reputation loss. A popular example is the 2017 Equifax cybersecurity incident caused by a web-server vulnerability in the Apache Struts package. The incident led to a data breach of millions of American citizens, costing Equifax 1.8 billion USD in security upgrades and lawsuits [83].

The problem is that developers struggle to identify what vulnerabilities may affect their software application [14]. Current security scanners report on the severity of a vulnerability, but lack a support to identify if the dependency is 1) used in the code and 2) is part of the production software the project delivers. Developers constantly complain that security alert tools report too many false positives [84, 85], as even the most critical vulnerability may be unexploitable if the vulnerable dependency is never released in the production software.

In this paper, we study how often dependencies are actually part of a production software and their impact on security based on their characteristics, usage, and context. We perform this study on 100 JavaScript projects in npm, the largest and fastest growing software ecosystem to date [86], to answer the following three research questions:

RQ1: How many installed dependencies are production dependencies?

Table 3.1: Concepts and definitions.

Concept	Definition	Example
Runtime dependency	Refers to a dependency declared in the runtime configuration of the `package.json` file and required for the application to function.	`react` is a runtime dependency, as shown in Listing 3.1.
Development dependency	Refers to a dependency declared in the development configuration of the `package.json` file and required to develop the application.	`webpack` is a development dependency, as shown in Listing 3.1.
Installed dependency	Refers to a dependency installed in the project as a result of the `npm install` command.	The dependencies shown in Listing 3.1, along with their own dependencies, are installed dependencies.
Depth	Refers to the level of a dependency in the dependency tree.	The `npm ls` command can be used to obtain the dependency tree.
Direct dependency	Refers to a dependency with a depth of 1.	The dependencies shown in Listing 3.1 are direct dependencies.
Transitive dependency	Refers to a dependency with a depth greater than 1.	The dependencies of the dependencies shown in Listing 3.1 are transitive dependencies.
Usage	Refers to the scope in which a dependency is used.	Listing 3.2 shows that `react-dom` is used in production.
Context	Refers to the context of the application in which a dependency is used.	In the example application, `webpack` is a development dependency used to bundle the application's resources.

RQ2: What are the characteristics of production dependencies?

RQ3: How often are npm security alerts emitted for production dependencies?

Findings show that production dependencies represent a very small fraction of the total number of dependencies in each project. While projects tend to depend on hundreds of dependencies (both direct and transitive), 51 projects did not have any production dependencies, and 49 have a median of 5 production dependencies. Contrary to common assumptions, most dependencies declared as runtime are not shipped to production while some development dependencies are included in the production software. Consequently, we find that dependency usage and context gives better insight at determine if a dependency will be used in production than the nature of a dependency itself. Furthermore, our results show that not all security vulnerabilities reported by npm are an actual threat to the software in production.

3.2.1 Our Contributions

In summary, we make the following key contributions in this chapter:

- To the best of our knowledge, this is the first study to investigate the discrepancy between installed and production dependencies in open source projects.
- We report on results that challenge the assumptions of dependency management and should be revisited by researchers and practitioners.
- We investigate the support of current tools in providing better information for developers regarding the scope and context of vulnerable dependencies.
- We make our dataset of 100 projects available[1], including all scripts used to collect and pre-process data, to facilitate replication and foment more research in the field.

3.2.2 Chapter Organization

The rest of this chapter is organized as follows: we start by motivating our problem with an example in Section 3.3. We describe and justify our methodology in Section 3.4 and explain our results in Section 3.5, Section 3.6, and Section 3.7. Implications of our findings are discussed in Section Section 3.8. We present the related work in Section 3.9, and discuss the limitations to our study in **??**. Finally, we conclude this chapter in Section 3.11.

3.3 Motivation & Background

To motivate our study and illustrate the terminology used in this paper, we walk the reader through the creation of a simple application using `create-react-app` [87]. The terms used in this example and throughout this paper are formally defined in Table 3.1. This example application is a single-page "Hello World" application that is provided by React when initializing a Create React App project. We create our application by simply running the command `npm create-react-app my-app`.

How many dependencies in our project? To achieve this single-page React application without further programming, our generated application reuses several open source packages published in npm. We refer

[1] https://zenodo.org/record/6518765

to each of the packages as a *dependency* of our project. The dependency configuration of our project is stored in the `package.json` file, shown in Listing 3.1. Dependencies are grouped into two groups: *runtime dependencies* ("dependencies") and *development dependencies* ("devDependencies"). Runtime dependencies are dependencies required by the application to function, e.g., as we build a React application, our project depends on `react` version 17.0.2. Development dependencies, on the other hand, are needed to develop the project, e.g., to format the code (prettier 2.5.1), and are not required by the software to run. As it can be seen in Listing 3.1, our small application has 7 runtime dependencies and 9 development dependencies.

Listing 3.1: A snippet of the package.json file listing the dependencies of our example project.

```
"dependencies": {
        "@testing-library/jest-dom": "^5.16.2",
        "@testing-library/react": "^12.1.3",
        "@testing-library/user-event": "^13.5.0",
        "react": "^17.0.2",
        "react-dom": "^17.0.2",
        "react-scripts": "5.0.0",
        "web-vitals": "^2.1.4"
      },
      "devDependencies": {
        "@webpack-cli/generators": "^2.4.2",
        "css-loader": "^6.6.0",
        "html-webpack-plugin": "^5.5.0",
        "prettier": "^2.5.1",
        "style-loader": "^3.3.1",
        "webpack": "^5.69.1",
        "webpack-cli": "^4.9.2",
        "webpack-dev-server": "^4.7.4",
        "workbox-webpack-plugin": "^6.5.0"
      }
```

Once we install these dependencies locally to build and test our application (`npm install`) we may be surprised to see that a total of 1,764 dependencies were installed. The dependencies shown in Listing 3.1 are *direct dependencies* of our project, each of which have dependencies of their own. For instance, the

package `loose-envify` is a dependency of `react`. These are called *transitive dependencies* and represent the vast majority of installed dependencies. As such, `loose-envify` is a transitive dependency of our example application. We use the term *installed dependencies* to refer to all dependencies of a project, both direct/transitive and development/runtime dependencies.

Is our application vulnerable? Security vulnerabilities are a widespread problem in npm [82]. Given that our application depends on 1,764 installed dependencies, is our application affected by vulnerabilities? To verify this, we resort to using a Software Composition Analysis (SCA) tool. SCA tools are used to identify open source components in software codebases to evaluate security, license compliance and overall code quality [88]. In our example, we use *npm audit*, a native tool of npm that reports vulnerabilities affecting software dependencies and maintains its own database of vulnerabilities. If a dependency is affected by one of more vulnerabilities, we refer to the dependency as a *vulnerable dependency*. In our example application, upon running `npm audit`, we receive the report that our simple application contains 6 moderate severity vulnerabilities, 13 high severity vulnerabilities, and 1 critical severity vulnerability. That is, without any further programming, our project already started with an alarming number of vulnerabilities of moderate, high, and critical severity. Examples of the reported high severity and critical severity vulnerabilities include Regular Expression Denial of Service, Template Injection, and Prototype Pollution.

Can reported vulnerabilities really affect our example application in production? Vulnerable dependencies are problematic and may affect the security of our project in multiple ways. However, the risk of vulnerable dependencies reaches its peak when the dependency is needed for the software to run in a production environment. To find which dependencies are part of our production software, i.e., *production dependencies*, we use a module bundler. A *module bundler* is a tool that assists the building process of a software by resolving the software dependencies and pruning the dependencies that are not needed in the production software. The process of pruning dependencies is referred to as *tree shaking*. We use *webpack* [89], a popular JavaScript module bundler, to build our production software and export a list of production dependencies.

Upon building our project with `webpack`, the tool generates a *source map* file, which contains the list of production dependencies of our application. From the 1,764 dependencies in our example project, Listing 3.2 shows that only 6 are released to production: `react`, `object-assign`, `scheduler`, `react-dom`, `style-loader`, and `css- loader`. More so, none of our production dependencies contained any reported vulnerability, thus, our original report of 15 vulnerabilities affected dependencies that would not be present in

the application in production.

The problem: security alert fatigue. Our example showcases an important problem in current software development. Even small applications may depend on thousands of dependencies and vulnerabilities are constantly being reported by the open source community. Developers face the difficult challenge of separating security alerts that are relevant to their application security from the long reports yielded by current SCA tools [84, 85]. In this paper, we evaluate this problem on a scale of 100 popular JavaScript projects.

Listing 3.2: A snippet of the source map generated by building our example project.

```
{
  "version": 3,
  "file": "main.js"
  "mappings": "KAAK,CAACC,EAAOC,GAAI..."
  "sources": ["node_modules/css-loader/dist/runtime/api.js",
  "node_modules/object-assign/index.js",
  "node_modules/react-dom/cjs/react-dom.production.min.js",
  "node_modules/react/cjs/react.production.min.js",
  "node_modules/scheduler/index.js",
  "node_modules/style-loader/injectStylesIntoStyleTag.js"]
```

3.4 Methodology

The goal of the paper is to study how often project dependencies are shipped to production and their impact on the security of software projects. In this section, we describe how we select and curate the set of study projects (Sections 3.4.1 and 3.4.2), and how we identify production dependencies (Section 3.4.3). We provide an overview of our methodology in Figure 4.1.

3.4.1 Dataset of Candidate Projects

The focus of our study is to investigate how active software development JavaScript projects use their dependencies. To this aim, we start by collecting data of a large number of JavaScript repositories as candidate projects for our study. Many studies have used the number of GitHub stargazers as a way to select candidate projects [90, 91**?**]. Thus, we start with 11,860 popular JavaScript projects that were collected on July 27th, 2020 with at least 100 stargazers.

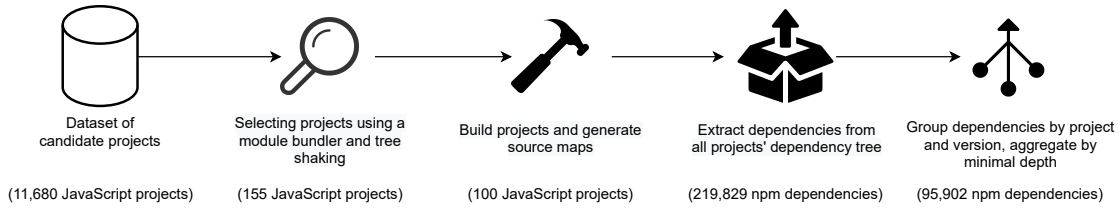


Figure 3.1: Overview of our approach for filtering projects and collecting dependencies.

Finding what dependencies are shipped to production is a very challenging task making it impractical to apply this analysis on a large-scale [92]. In our study, we opt to select projects that already make use of tree shaking (see Section 3.3 for a more in-depth explanation). Specifically, we select projects using either `webpack` or `rollup` [93] because they are two of the most popular module bundlers for JavaScript projects and they have integrated tree shaking support.

To find out which projects use `webpack` and `rollup`, we automatically parse the `package.json` files of the 11,860 projects to identify 1) if any of the bundlers are declared as a dependency and 2) the tree shaking algorithm is enabled for the project. Through this process, we find that 155 JavaScript projects make use of `webpack` or `rollup`, and have tree shaking enabled.

3.4.2 Building Candidate Projects

To assess whether a dependency is used in production, we have to successfully build each project in a production environment with a module bundler. During the build of a project, the module bundler first looks for all of the dependencies in the project and constructs a dependency graph (dependency resolution). The dependency graph is then converted along with source code into a single file (packing) called the bundle. Source maps are then generated after a successful build.

To build the candidate projects, we first clone each of the 155 JavaScript projects locally. We planned to build a framework to automate the build of all the 155 JavaScript projects. However, we soon realized that

Table 3.2: Descriptive statistics of the selected projects.

Metric	Mean	Median	Min.	Max.
Stars	4,827.6	1,224	112	74,201
Commits	1,364.9	496	31	6,188
Contributors	62.3	26	4	401
Age (years)	5.2	5	1	12

many projects require specific building commands and setup to be build successfully. In fact, the majority of the projects did not support the standard build command (`npm run-script build`). Furthermore, the environmental settings varied across projects, e.g., some projects require specific NodeJS versions and identifying this automatically is very challenging. We then proceed to semi-manually build each project using the following methodology:

1. **Read projects documentation.** We read the documentation of all the 100 studied projects to identify the specifics of each project build. The goal of this step is to identify all the steps of the building process: the build commands, supported Node versions, supported package manager (e.g. YAML or npm), and any other specificity of the project building configuration. At this point, we also confirmed that all selected projects are related to software development, i.e., are not personal toy-projects.

2. **Install dependencies.** We install all dependencies specified in the `package.json` file by using the `npm install` command. This generates a `node_modules` folder in every project's home directory which contains all installed dependencies.

3. **Build project.** Following each project's documentation, we build each project in the dataset. The first author manually followed the steps of the building process to ensure the build was successful, the source maps containing the production dependencies was generated, and the yielded artifacts targeted the production environment.

4. **Generate source maps.** Upon the successful completion of the building process, source maps are generated and saved in the project's temporary folder or home directory.

After our careful process, we successfully build and generate source maps for 100 JavaScript projects. From the 55 projects that failed in our process, the main culprit was the generation of the source maps file. In most of the failed cases, projects' configuration did not have the flexibility to generate the source maps file. For example, we found some projects created using the `create-react-app` package that does support module bundlers and tree shaking, but does not have the option to output source maps.

We present descriptive statistics of the 100 projects we successful built and generate source maps in Table 3.2. The projects of our dataset are very popular (median 1,224 stargazers), tend to be mature projects (median of 5 years of development and 496 commits) and are developed by medium-sized team of developers (median of 26 developers).

3.4.3 Identifying Dependencies in Production

To identify production dependencies, we first collect all dependencies found in the source maps of each projects using a mix of source map parser [94] and regular expression (regex). Then, to obtain the version of each dependency found in the source maps, we locate its `package.json` file in the respective project's `node_modules` folder and parse it. This results in a dataset of production dependencies with their corresponding version.

In addition to identifying dependencies used in production, we also want to identify two very important characteristics of all dependencies, as they have an influence on the risk of vulnerabilities [95]: 1) the dependency scope, runtime or development and 2) whether the dependency is a direct or transitive dependency of the project. To classify a dependency into runtime or development, we analyze the `package.json` file of a project, classifying dependencies configured in the "dependency" section as runtime dependencies, and classifying dependencies declared in "devDependency" as development dependencies. Since transitive dependencies are not listed in the `package.json` file, we identify the type of the original dependency which determines the type of the transitive dependency.

To classify installed dependencies into direct or transitive dependencies, using the command `npm list` we generate the *dependency tree*, a hierarchical representation of relationship between dependencies. The `npm list` command lists all installed dependencies in json format, including the name, version, path, and depth of each dependency. From the depth, we identify each dependency as direct or transitive, i.e., direct dependencies have depth = 1, while transitive dependencies have depth > 1.

Our methodology has one limitation, we cannot automatically resolve missing peer dependencies. Peer dependencies are used to decouple dependencies between projects, to ensure a single version of the package is installed for all dependencies. For example, in applications with many npm packages depending on `react`, `react` can be declared as a peer dependency to prevent the installation of multiple (possibly conflicting) versions of `react`. Unlike runtime and development dependencies, peer dependencies are not automatically installed by npm. Instead, they must be included by the code that uses the package as a dependency. We find that 37 projects in our dataset have missing peer dependencies. Since it is not possible to automatically resolve missing peer dependencies for all 37 projects, we exclude the dependencies from our analysis.

Table 3.3: Dependency profiles of projects with and without production dependencies. Totals are reported as absolute counts, and medians are computed from the aggregated value per project. Percentages are relative to the number of installed dependencies in each project group.

	Zero production dependencies		At least one production dependency	
Dependency type	**Total**	**Median**	**Total**	**Median**
Installed	46,031 (100%)	851	53,421 (100%)	1,017
Runtime	1,005 (2.1%)	0	873 (1.6%)	5
Development	45,025 (97.9%)	832	52,542 (98.4%)	1,017
Direct	1,539 (3.4%)	40	2,098 (3.9%)	29
Transitive	44,492 (96.6%)	809	51,307 (96.1%)	963
Production	–	–	497 (0.9%)	5

3.5 RQ1: How many installed dependencies are production dependencies?

Motivation: While reusing packages may reduce development time, developers have to constantly maintain their dependencies to fix bugs in the packages and mitigate the problems of vulnerable dependencies [96**?** , 67]. However, identifying dependencies used in production is not a trivial task making it difficult to prioritize dependency-related maintenance activities [84].

In this research question, we want to assess how often dependencies of the selected projects are actually production dependencies. Answering this question is the first step to understand how often a runtime and development dependency is used in production. It will also help us better understand how dependencies are used in practice and how they impact the security of software.

Approach: To approach this research question, we use the methodology described in Section 3.4.3. That is, we start by installing all dependencies from each project to retrieve the list of installed dependencies and their respective versions. To classify an installed dependency into direct or transitive, we generate the dependency tree of each studied project. Then, to identify production dependencies, we build all software projects with their respective module bundler (`webpack` or `rollup`). This building process was done manually by following the building steps specified in the project documentation, to ensure each project is built correctly and without errors. After building the project, we analyze the yielded source maps to identify the production dependencies. Finally, we cross reference the installed dependencies and production dependencies to classify each project dependency into production/non-production, runtime/development, direct/transitive and report our findings.

Finding 1: Of the 100 projects, 51 projects contain no production dependencies. To make a better sense of our results, we split our dataset of 100 projects into two sets: projects with production dependencies (49 projects) and projects without production dependencies (51 projects). Table 3.3 shows the total number

of dependencies and their characteristics in both sets of projects. The 51 projects with no production dependencies have installed a total of 46 thousand dependencies, including direct and transitive packages, however, none of the installed dependencies are used in production. More interestingly, among the installed dependencies, there were 1,005 packages that were declared to be runtime dependencies, which is supposedly required at the runtime of the the final software, but were not included in the final production artifact.

We also note that the set of 51 projects with no production dependencies have a median of runtime dependencies of zero. We confirm this finding through manual investigation and find that 39 projects in our dataset only declare development dependencies. Most of these projects are libraries meant to be used by other projects as development tools. Examples of such projects are `Vuex`, a state management pattern for Vue.js applications; `three.js`, a popular cross-browser 3D library; and `polished`, a lightweight toolset for writing styles in Javascript. All those library projects have the incentive to depend on little to no runtime dependencies, as the fewer dependencies they have, the less constrained their users may be to rely on their libraries [97, 98].

Finding 2: From the 49 projects with production dependencies, production dependencies represent less than 1% of the installed dependencies. The results show that projects with production dependencies have a total of 53,421 dependencies, of which only 497 dependencies (0.9%) are released to production (see Table 3.3). Analyzing the median number of dependencies per project (see Median column in Table 3.3), we find that projects have in median 5 production dependencies while depending in median over a thousand dependencies. However, we notice that not all runtime dependencies are used in production. The total number of runtime dependencies installed (873) far exceeds the number of production dependencies (497), indicating that many runtime dependencies may be incorrectly configured or not used in the code.

Figure 3.2 shows the distribution of the number of production dependencies per project. We can observe that 15 projects contain a single production dependency and the vast majority of projects (65.3%) have less than 10 dependencies used in production. Still, we found some projects that depend heavily on packages in their production build, with `ProjectMiradormirador` being the project with the most production dependencies in our dataset with 92.

Finding 3: More than half (59%) of the runtime dependencies are not used in production. As shown in the "Production" row of Table 3.4, we find that 510 out of 863 of the total runtime dependencies are not shipped to the production bundle. Runtime dependencies are dependencies (supposedly) required by the application to run. Our results, however, show that in the majority of the cases, dependencies are declared as

Table 3.4: Characteristics of dependencies in projects with production dependencies.

Production status	Dependency type	Direct	Transitive	Total
Production	Development	62	77	139
	Runtime	175	178	353
Non-production	Development	1,809	50,594	52,403
	Runtime	52	458	510
Total		**2,098**	**51,307**	**53,405**

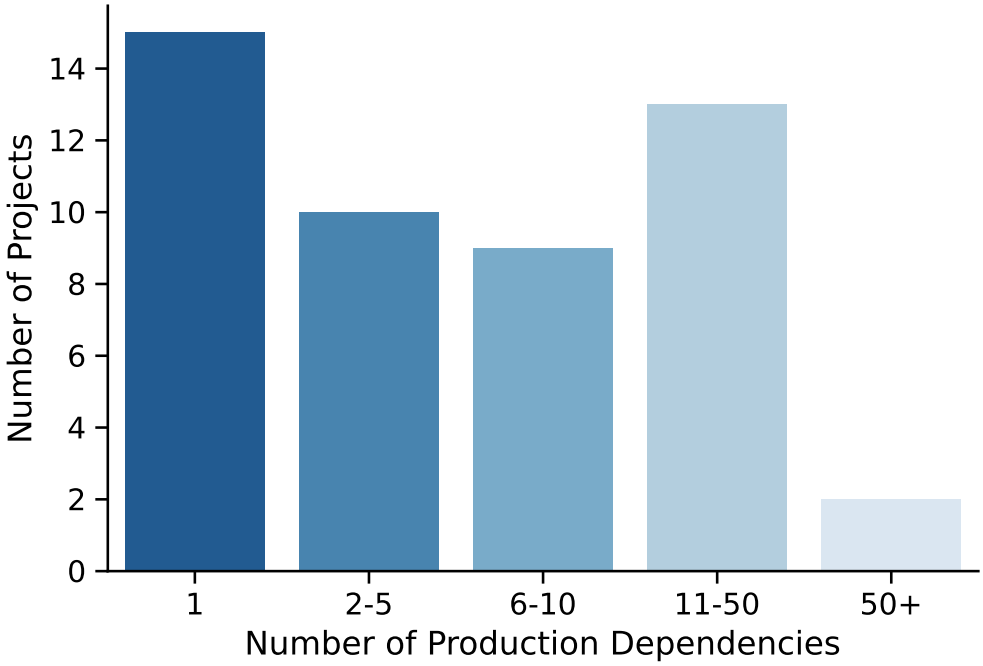


Figure 3.2: Number of production dependencies on the 49 project with one or more production dependencies.

runtime but are not actually used in the code, thus, are excluded by the module bundler during the build. This finding suggests developers mistakenly maintain unused dependencies in their project configuration, which indicates that they lack the necessary information to determine whether a dependency is actually used by the software in production. This is corroborated by related work [13], where authors reported that unused dependencies occur in 80% of studied projects.

Answer to RQ1. 51 out of 100 projects do not use any dependencies in production. The 49 projects that ship dependency to production contain less than 1% of production dependencies. Contrary to common belief, 59% of runtime dependencies are not used in production.

3.6 RQ2: What are the characteristics of production dependencies?

Motivation: Production dependencies are the prime security liability in software systems since they can compromise a running software [92]. Current SCA tools may not distinguish dependency scope (i.e., production, non-production) [84, 14], which may lead to reporting unexploitable vulnerabilities (false positives). They may also only consider direct dependencies although vulnerabilities can be introduced transitively [84, 99, 100]. The problem is that assumptions about production dependencies are not always correct. In fact, RQ1 showed that runtime dependencies are not always in production. In this research question, we study the characteristics production dependencies to establish a practical understanding of how they are used and in what context they are used. Such findings help in improving current SCA tools as they provide insights on how dependencies are used in practice.

Approach: To identify the characteristics of dependencies used in production, we consider the production dependencies identified in RQ1 and classify them based on their scope (runtime, development), depth, and usage. In theory, one can identify the scope of a dependency by looking at the nature of the functionality provided by a package. For instance, packages that provide development utilities should not become production dependencies. To investigate to what extent the nature of the package determines if it will be used as a production dependency, we analyze how packages are released to production across the 100 studied projects. We analyze a total of 1,269 unique packages. We then classify the packages in three categories: 1) packages that are always used in production, 2) packages that are never used in production, and 3) packages that are sometimes used in production.

Finding 4: 28.2% of production dependencies are development dependencies. The first section of Table 3.4 shows the characteristics of production dependencies. We find that 28.2% of production dependencies are development dependencies and the remaining 71.7% are declared as runtime dependencies. It is expected that all dependencies released to production consist of runtime dependencies since they provide the application with specific functionalities to be used by the client. It is then surprising to find that almost 30% of the dependencies released to production are development dependencies since such dependencies are, by default, not included in the production bundle.

While unusual, having a development dependency in production occurs in 37 of the projects in our dataset. To better understand this, we perform an exhaustive inspection of the dependency configuration the 37 projects and deduct two possible causes for a development dependency to be in production. First, the

selected projects use module bundlers, which disregard the configuration of the `package.json` file and use source code analysis to identify what should be a production dependency. Developers may not be as careful to specify their development dependencies as their building process does not depend on a correct specification of development and runtime dependencies [101]. In fact, from the 37 projects with development dependencies in production, 4 (10.8%) projects declared all of their dependencies as development dependencies although all of them have at least 1 dependency in production. Second, it can be that the dependency is initially declared under the ”dependencies” property of the `package.json` file, but is intentionally moved by the developer to ”devDependencies” to get rid of security warnings, as it is explained in a `create-react-app` GitHub issue [102]. The author of the issue explains that `npm audit` reports vulnerabilities for code that never runs in production, but strictly at build time in development. They then suggest to move vulnerable dependencies to ”devDependencies” to get rid of the security warning. We believe development production dependencies are unlikely to happen in projects that do not use module bundlers. By default, npm does not include development dependencies in a production build. This means that a project that requires a development dependency at runtime will not function because of the missing dependency.

Finding 5: The majority of production dependencies (51.8%) are transitive dependencies. Looking at the Transitive columns of Table **??**, we notice that 51.8% of production dependencies are dependencies of their direct project dependencies. These results suggest that developers may not have control over the majority of production dependencies. Naturally, a transitive dependency can only be released to production if the original dependency is also released to production. Hence, developers should be extra careful when selecting production dependencies, preferably by selecting packages that have little to no production dependencies on their own, to reduce the attack surface through vulnerable dependencies.

Finding 6: The 237 production dependencies come from 183 unique npm packages. From these, 43 are sometimes not used in production in other projects. To put things in perspective, we evaluate the number of unique npm packages in our dataset by grouping the packages by name and obtain 1,269 unique npm packages. From this, we find that 1,086 (85.6%) are never used in production since they offer functionalities that are development-only. For example, `eslint` installed in 79 projects, is a static code analysis tool that is used to identify problematic patterns found in JavaScript code, `@babel/core` installed in 70 projects, is a command line interface tool that facilitates working with `babel`, and `rollup` installed in 65 projects, is a build tool for JavaScript projects.

For the rest of the packages, we find that 183 packages are used in production at least once. Taking a

closer look at the production packages, we find that 140 (76.5%) packages are always used in production when installed in a project and that such packages do not occur frequently. In fact, they occur at most in 2 different projects and are installed as runtime dependencies. For example, `is-promise`, a library that tests whether an object is a `promises-a+` promise, `query-string`, a library that parses and stringifies URL query strings, and `react-fast-compare`, a library that provides specific handling of fast deep equality comparison for React, are all installed in 2 projects, and used in production 100% of the time they are installed.

Interestingly, we find that 43 (23.5%) of the 183 production packages are not always shipped to production. This indicates that some packages are used differently (in production and not in production) across projects regardless of their functionalities. We show in Table **??** 10 examples of such packages, and how often they are in production versus how often they are installed. The results show that `react`, a library for building user interfaces, is the most frequently installed package appearing in 40 projects, but is only released to production in 4 projects. In contrast, `react-redux`, a React binding for Redux allowing React components to read data from a Redux store, only appears in 5 projects, but is released to production in 4 out of 5 projects. In only one project (redux-little-router) is `react-redux` not released to production and declared as a development dependency. We further inspect the `package.json` of `redux-little-router` and find that `react-redux` is a peer dependency, thus, it is not included in the production bundle of the project. It is also worth noting that redux-little-router is a lightweight library that provides flexible React bindings and components. Thus, the project makes a conscious effort to mitigate dependency bloat, declaring most of its dependencies as development dependencies, and including `react`, `react-dom`, `react-redux`, and `redux` as peer dependencies.

The main takeaway from this finding is that we cannot identify production dependencies by looking at the functionalities of a package alone. As we have shown, the scope of a dependency can vary based on the context and usage of a package, which means it may differ from project to project. For example, a module bundler may be used in production in one application since it uses some of its functionalities at runtime, but may only be used in development in another application. Thus, it is important for SCA tools to include this scope analysis in their approach so that developers can more easily identify production dependencies based on their own usage and context.

Table 3.5: Frequently installed packages that are both used and not used in production.

Package	Production installations	Total installations	Production share
`react`	4	40	10.0%
`react-dom`	3	37	8.1%
`prop-types`	13	23	56.5%
`@babel/runtime`	10	19	52.6%
`lodash`	4	14	28.6%
`core-js`	5	13	38.5%
`classnames`	5	8	62.5%
`react-is`	1	5	20.0%
`react-redux`	4	5	80.0%

Answer to RQ2. Our findings indicate that 28.2% of production dependencies come from development dependencies and that 51.8% come from transitive dependencies. The functionality of the package alone does not determine if they will be shipped to production: 43 of 183 packages encountered in production in one project are not shipped to production in other projects.

3.7 RQ3: How often are npm security alerts emitted for production dependencies?

Motivation: The observations made in RQ1 suggest that the majority of the dependencies are not used in production. While vulnerabilities in non-production dependencies may affect the development environment (e.g., installing packages with malicious code), it is when a vulnerable dependency is released to production that the threat of exploitation reaches its peak [92]. Developers should constantly run scanners to identify security alerts in their project and prioritize fixes in production dependencies, to avoid having their software compromised. The problem is that tools such as `npm audit` often report many false alerts for deployed code, making vulnerability reports noisy and bloating audit resources [103, 102]. In this research question, we investigate how often security alerts are emitted for production dependencies compared to non-production dependencies and the characteristics of vulnerable dependencies.

Approach: To investigate how often vulnerabilities are encountered in production and non-production dependencies, we first generate the npm vulnerability report of each project by using the `npm audit` tool. Next, to obtain the `npm audit` reports in a parseable csv format, we adapt the `npm-deps-parser` [104], a tool that parses, summarizes, and prints `npm audit` json output to markdown. From this, each vulnerability

Table 3.6: Characteristics of vulnerable dependencies reported by npm vulnerability alerts.

Dependency type	Direct	Transitive	Total
Development	9	410	419
Runtime	1	7	8
Total	**10**	**417**	**427**

report is identified with the project name, the vulnerable dependency and version, the severity, and a unique link to the GitHub Advisory Database (GAD) [105], a database of security advisories affecting the open source world. To obtain the scope and depth of each vulnerable dependency, we cross-reference the set of vulnerable dependencies with the set of production dependencies and installed dependencies for each project. Because of the limitations discussed in Section 3.4.3, we could not identify the scope and depth of 29 vulnerable dependencies and exclude them from further analysis.

Finding 7: A total of 608 security alerts are emitted for dependencies of 32 projects, yet none are related to production dependencies. In our dataset, no security alerts were emitted for 68 projects. The remaining 32 projects reported a total of 608 security alerts for 456 vulnerable dependencies, i.e., the same dependency may issue multiple security alerts. In median, these 32 projects reported 16 security alerts, none related to production dependencies.

There are a few reasons as to why security alerts may have been emitted only to non-production dependencies. First, as seen in RQ1, the vast majority of dependencies are not released to production (99%), the chances of vulnerabilities being encountered in non-production are 99x higher than in production dependencies. Second, developers of the selected projects are likely making the conscious effort of updating production dependencies to mitigate security vulnerabilities, since they may be aware of what dependencies may be used in production (e.g., developers open a PR in the project `InstantSearch` to update a vulnerable dependency [106]). The problem, however, is that tools such as `npm audit` make no distinction whether security alerts are referring to non-production dependencies. Developers have to know themselves which dependencies are released to production to filter out relevant security alerts that need urgent action, making it harder to prioritize management efforts. This is shown in related work [14], where authors reported that 69% of the surveyed developers claimed to be unaware of their vulnerable dependencies and that dependency updates are perceived as extra workload and responsibility.

Finding 8: 98.1% (419) of the vulnerable dependencies are development dependencies. In this analysis, we switch from security alert reports to vulnerable dependencies, as multiple reports may be issued for the

Table 3.7: Number of vulnerability reports by severity level and the corresponding npm recommended action.

Vulnerability severity		Dependency type	
Severity	**Recommended action**	**Runtime**	**Development**
Low	Address at your discretion	3	33
Moderate	Address as time allows	1	226
High	Address as quickly as possible	5	263
Critical	Address immediately	0	45

same dependency under different vulnerabilities. The first row of Table 3.6 shows the number of development and runtime vulnerable dependencies reported by our experiment. The `npm audit` tool reports security alerts from a total of 419 vulnerable development dependencies, representing 98.1% of all vulnerable dependencies identified. From the 419 vulnerable development dependencies, 9 (2.1%) are direct dependencies, and 410 (97.9%) are transitive dependencies. Next, we analyze the severity of the vulnerability reports in relation to the characteristics of vulnerable dependencies as shown in Table 3.7. We find that all critical and most of the high-severity reports are emitted for development dependencies.

In some cases, tools such as `npm audit` allow developers to filter out development dependencies from the security reports, as they are supposedly not released to production [102]. It is dangerous, however, to completely ignore the security maintenance of development dependencies. Some development dependencies are used in production, as seen in RQ2, and hence, have the risk of being exploited in a production environment. Vulnerable transitive dependencies that are released to production are equally dangerous since even if they are reported by npm, developers are not in control of their update.

Answer to RQ3. 32 projects in our dataset reported a total of 608 security alerts, but none of the alerts referred to a production dependency. Projects have in median 16 security alerts, but the vast majority refer to development non-production dependencies which does not represent a threat for their running application.

3.8 Discussion

In this section, we discuss the implications from our work and possible solutions for current source code based tools.

3.8.1 Implications

Tracking production dependencies is very challenging. While our study focuses on a selected number of 100 popular JavaScript projects, the results showcase the difficulties of mapping a project's production dependencies. This difficulty arises primarily because assumptions commonly held by the development community regarding dependency management do not hold in practice:

1. Assumption 1: Runtime dependencies are always shipped to production. Our results showed that the majority of dependencies declared as runtime are not used in production (RQ1). Developers may spend time ineffectively managing runtime dependencies due to security alerts, without confirming that such dependencies are bundled in their delivered software.

2. Assumption 2: Development dependencies are never shipped to production. In projects that use module bundler, dependencies declared as development may be shipped to production (RQ2). In fact, development dependencies represent a third of all production dependencies identified in our study. Developers may disregard all their development dependencies as being irrelevant for security upgrades, when in fact, some vulnerable development dependencies are shipped in their delivered software.

3. Assumption 3: The functionality provided by the package is sufficient to determine if it is a production dependency. Particularly in cases where packages provide runtime utilities, our results show that 43 out of 183 packages are released to production in some projects but not in others. Thus, the package's functionality is not sufficient to determine whether a package is used in production (RQ2).

These assumptions have the potential to affect the security of the delivered software, as developers may wrongly assume what dependencies are sensitive to security exploits.

Not all vulnerabilities in dependencies are a security risk for the software in production. Prior research has shown that not all vulnerabilities are relevant for the software in production [85, 103]. In this paper, we expand on this by studying the relevance of dependencies for the security risk of a software in production. Our findings indicate that, given by the prominence of non-production dependencies (RQ1), the vast majority of security alerts will be emitted for dependencies that do not impact the security of a software in production (RQ3).

To put things in perspective, we analyze the types of vulnerabilities reported by `npm audit`. The most common type of vulnerability identified in the studied projects is Regular Expression Denial of Service

(ReDoS), accounting for 25.3% of all reported vulnerabilities and for 27% of high severity vulnerabilities. While a diligent dependency management is of utmost importance to mitigate security risks, developers should be mindful of the types of alerts they should prioritize. In the case of a ReDoS attack, the performance of an application is compromised if there is a regular expression that, with malicious input, slows it down exponentially. However, we find that 97.3% of the dependencies affected by a ReDoS vulnerability are development-only, which tend not to be part of a production software. Previous research shows that developers tend to ignore security alerts when they receive a lot of them [14]. Our approach allows them to focus on the important ones first (i.e., security alerts for vulnerable dependencies in production).

Source maps and tree shaking can benefit developers beyond client-side applications. In this paper, we use module bundlers to accurately differentiate between installed dependencies and production dependencies. Module bundlers are most commonly used in client-side applications, which are generally defined as libraries or frameworks running in a Web browser (e.g., React, Vue, Angular) to support the development of Web applications. Module bundlers, however, can benefit far beyond just client-side applications by helping developers:

1. Prioritize addressing security alerts on production dependencies. As security alerts are very commonly issued for projects that rely on open source code, developers should prioritize addressing security issues that have the potential to affect their production software, by identifying vulnerabilities affecting their production dependencies.

2. Prioritize maintenance tasks on production dependencies. As projects depend on increasingly high number of software dependencies, updating all dependencies in every release may become increasingly prohibitive. Updating dependencies always have the risk of breaking changes [107], leading to software bugs and mistrust between project maintainers [13]. Hence, developers should prioritize updating production dependencies to focus their maintenance tasks on packages that may affect their delivered software.

3.8.2 Towards Better Tool Support

SCA tools are constantly used by software projects to control the risk related to software dependencies, such as vulnerabilities, and compliance to open source licenses[88]. To understand the support current SCA

Table 3.8: Metrics based on vulnerable dependency (VD) characteristics reported by existing tools and their support for locating vulnerable code (VC) in JavaScript projects.

Tool	Covers all dependencies	Filters security alerts by					Locates dependency code
		Runtime	Development	Direct	Transitive	Production use	
`npm audit`	Yes	Yes	Yes	Yes	Yes	No	No
Snyk	Yes	Yes	Yes	Yes	Yes	No	No
Dependabot	Yes	No	No	No	No	No	No
OWASP Dependency-Check	Yes	No	No	No	No	No	No

tools provide to production dependencies, we investigate four popular tools: npm audit, Snyk, Dependabot, and OSWAP Dependency-Check. We analyze the documentation of the SCA tools, as well as apply them to some of our studied projects to assess their capabilities and limitations.

We present in Table 3.8 an overview of the features related to dependency scope and usage from four popular SCA tools. All SCA tools we assess cover all dependencies of a software project, including both direct and transitive dependencies. Given a project may have thousands of installed dependencies, we now dive into the filtering capabilities of the tools. We note that only `npm audit` and Snyk [108] provide ways of filtering security alerts based on whether the vulnerability affects runtime/development dependencies or direct/transitive dependencies. The filtering of runtime/development dependencies is based on project configuration (e.g., `package.json` file), thus, it is subject to limitations when it comes identifying production versus non-production dependencies. Neither Dependabot nor OSWAP Dependency-Check allow users to filter security alerts based on the scope or depth of their dependencies.

It is worth noting that none of the tools provide a way to differentiate between production versus non-production dependencies. There is no support, for instance, to input source map files in the tools, to help filter out vulnerabilities that concern non-production dependencies. We believe adding source map support to SCA tools would offer developers better insight on their production bundle without relying so much on the dependency configurations that have shown to be inconsistent across different projects.

Finally, we find that none of the tools provide a way to locate where the vulnerable dependency is used in code (column "Locate dep code"). Developers have to rely on their own set of static/dynamic analysis tools to know exactly where the vulnerable dependency is used in the codebase. We believe static analysis tools would benefit from using the features provided by module bundlers that scan the code for `import` statements

to provide the path to the source file in which a dependency is imported and used.

3.9 Related Work

In this section, we discuss the related literature divided into three aspects. First, we discuss works that have focused on the challenges related to the Software Bill of Materials. Then, we discuss works describing the challenges of dependency management in software ecosystems. Finally, we discuss existing tools and approaches to detect vulnerable dependencies.

3.9.1 Software Bill of Materials

The Cybersecurity and Infrastructure Security Agency (CISA) defines the Bill of Materials (BOM) as a nested inventory of components in a piece of software [109]. The process of identifying production dependencies is part of the constructing the BOM of a software. Several studies have proposed approaches to consolidate the BOM of software applications [110, 111, 112]. Zajdel et al. discussed that users of open source softwares tend to arbitrarily download the software into their build systems, but rarely keep track of which versions they use which results in unnecessary software being left in the application, increasing the risk of potential vulnerabilities [113]. Coelho et al. proposed a data-driven approach to measure the level of maintenance activity in GitHub projects [114]. The authors found that 16% of the studied open source projects have become unmaintained over the course of one year. They also reported that software tools such as compilers and editors have the highest maintenance activity over time and proposed that a metric about the level of maintenance activity of GitHub projects can help developers in selecting open source projects.

These prior studies focus on the importance of selecting well-maintained software libraries, and propose approaches to alleviate the challenges related to open source code reuse. However, the challenges of constructing the BOM for dynamic languages like JavaScript are still present. Thus, our study focuses on JavaScript projects and leverages existing approaches (i.e., module bundlers and tree shaking) to help developers maintain their softwares and decrease the risks related to open source code reuse by analyzing the software's dependencies and reporting on the ones that are actually used in the code.

3.9.2 Dependency Studies

Package ecosystems and the presence of vulnerable dependencies have been studied in the literature [13, 2, 97, 115]. Hejderup et al. report that one-third of the npm packages use vulnerable dependencies [116]. Similar to our study, the authors suggest context of usage of a package to be a possible reason for not fixing the vulnerable dependencies. Abdalkareem et al. conduct an empirical analysis on security vulnerabilities in Python packages [97]. They find that the number of vulnerabilities in the `PyPi` ecosystem increases over time and that it takes, on median, more than 3 years to get discovered, regardless of their severity. They emphasize on the need for more effective process to detect vulnerabilities in open source packages since both npm and `PyPi` allows to publish a package release to the registry with no security checks. Lauinger et al. conduct the first large scale of JavaScript open source projects and investigate the relationship between outdated dependencies and dependencies with known vulnerabilities [100]. They report that transitive dependencies are more likely to be vulnerable since developers may not be aware of them and have less control over them, which further corroborates with our findings of RQ3. Similarly, Williams et al. report that 26% of open source Maven packages have known vulnerabilities and refer to a lack of meaningful controls of the components used in the proprietary projects as a partial explanation to this high number of vulnerable dependencies [117].

These prior studies focus on the presence of known vulnerabilities in popular package ecosystems and the reason why the number of vulnerable dependencies is so high. However, their analysis does not consider the scope of dependencies (i.e., they do not distinguish production and non-production dependencies). As a result, the studied vulnerable dependencies may not be exploitable. Zapata et al. investigate vulnerable dependency migrations of npm packages and evaluate the impact of a vulnerability in the `ws` package on 60 JavaScript projects using the vulnerable version of the package [92]. The authors find that up to 73.3% of the dependent applications were safe from the vulnerability since they did not actually used the vulnerable code. The study also highlights that it is not trivial to map vulnerable code to client usage for JavaScript, which further corroborates with our findings in RQ1.

3.9.3 Detecting Vulnerable Dependencies

Alfadel et al. study the use of Dependabot security pull requests in 2,904 JavaScript open source GitHub projects [67]. Results show that the vast majority (65.42%) of the security-related pull requests are often merged within a day and that the severity of the vulnerable dependency or potential risk for breaking changes

are not associated with the merge time. Ponta et al. propose a pragmatic approach to facilitate the assessment of vulnerable dependencies in open source libraries by mapping patch-based changes of vulnerabilities onto the affected components of the application [118]. Sejfia et al. present `Amalfi`, a machine-learning based approach for automatically detecting potentially malicious packages [119]. The authors evaluate their approach on 96,287 npm package versions published over the course of one week and identify 95 previously unknown vulnerabilities. Pashchenko et al. propose Vuln4Real, a methodology that addresses the over-inflation problem of academic and industrial approach for reporting vulnerable dependencies in free and open source software (FOSS) [85]. Vuln4Real extends state-of-the-art approaches to analyzing dependencies by filtering development-only dependencies, grouping dependencies by project, and assessing dead dependencies. Their evaluation of Vuln4Real shows that the methodology significantly reduces the number of false alerts for code in production (i.e., dependencies wrongly flagged as vulnerable). Pashchenko et al's. work is the closest to ours since it considers similar aspects in relationship to the relevance of vulnerable dependencies: exploitability and dependency scope. Our study touches on another aspect that is not discussed in Vuln4Real and that is the context in which a dependency is used versus how it is configured. Our paper shows that there is a discrepancy between the configuration of dependencies and its usage, and that this discrepancy may affect the exploitability of a vulnerability (i.e., its relevance to the application).

Imtiaz et al. [95] present an in-depth case study by comparing the analysis reports of 9 SCA tools on OpenMRS, a large web application composed of Maven and npm projects. The study shows that the tools vary in their vulnerability reporting and that the count of vulnerable dependencies reported for npm projects ranges from 32 to 239. From the 9 studied SCA tools, 4 freely available tools could be applied to npm projects: OWASP Dependency-Check, Snyk, Dependabot, and npm audit. The results show that all 4 tools detect vulnerable dependencies across all scopes and depths and that reported vulnerabilities are mostly introduced through transitive dependencies, except for Dependabot. While the authors of this paper report on the coverage capabilities of SCA tools, our study mainly focuses on the data that is shown to the user. For example, npm audit covers dependencies of all scope when reporting for vulnerabilities, but it is the user's responsibility to filter the vulnerable dependencies by scope (production or development). That is, SCA tools don't explicitly report on the scope of vulnerable dependencies, and when it is done manually by users, this analysis depends on the project's dependency configurations rather than dependency usage.

3.10 Threats to Validity

Threats to internal validity considers the experimenter's bias and errors. Our method of analysis relies on building software projects with their configured module bundler to identify production dependencies, and errors in this process may introduce false positives/negatives in our analysis. We mitigate this threat by 1) only selecting projects that already use module bundlers to minimize any intervention that could introduce bugs in the process, 2) building each project manually by following the projects documentation, 3) manually inspecting the built artifacts (e.g., installed dependencies, source map files), and 4) removing 55 projects that showed evidence of failed builds (e.g., errors, empty source map files). To further confirm the validity of this process, we also sampled 7 projects from our dataset and asked contributors to validate our results, by checking the accuracy of the yielded production dependencies. We received responses from 4 projects and contributors confirmed the yielded classifications, helping us validate the soundness of our methodology.

Threats to eternal validity considers the generalizability of the findings. We purposefully select projects that already use module bundlers which could limit the type of project our findings generalize. First, module bundlers tend to be used primarily by projects that want to minimize their production dependencies, such as client-side packages such as web-applications and libraries. In fact, our finding that development dependencies are shipped to production are unlikely to occur in projects that do not use module bundlers. Second, our dataset is strictly composed of open source JavaScript projects, thus, our results may differ if a study is performed on proprietary projects or projects written in other languages.

3.11 Chapter Summary

This chapter investigates projects dependencies that are released to production and their impact on security and dependency management. We conducted our study on 100 npm projects, one of the largest and fastest growing software ecosystems. Our results showed that production dependencies are rare among the installed dependencies of a project, but are difficult to identify. Commonly held assumptions of dependency management do not hold in practice and context is more important in determining the scope of a dependency as opposed to its configuration. Furthermore, we evaluate how often security alerts are reported for production dependencies, and found that none of the vulnerability reports are emitted for dependencies released to production. Rather, the majority of the alerts are emitted for development, transitive dependencies which has

two main implications: 1) not every vulnerability is a threat to the software in production, and 2) vulnerabilities can be introduced transitively regardless of their scope, which further motivates the need for SCA tools to provide such an analysis.

This chapter outlines directions for future work. Using module bundlers as a way to identify production dependencies may augment current SCA tools to provide better insights on the scope of their dependencies within their project's context and usage. Consequently, module bundlers or similar tools, may benefit far more than just client-side applications and should be part of the build process of projects that extensively rely on open source code.

Chapter 4

How Robust are LLM-Generated Library Imports? An Empirical Study using Stack Overflow

4.1 Chapter Overview

The preceding chapter showed that dependencies differ substantially in their role, configuration, and relevance to production software. As large language models increasingly participate in software-development tasks, they introduce a new interface through which dependencies can enter software projects. Rather than being selected directly by developers, packages may now be recommended or incorporated as part of LLM-generated code.

This chapter examines the dependencies introduced by LLMs during software development. Using real-world Python programming problems, it investigates which libraries different models recommend, the characteristics of those libraries, and whether the resulting dependency recommendations can be used without additional intervention. The findings show that LLMs generally recommend mature, popular, and permissively licensed packages. However, they do not consistently provide the contextual information required to integrate those packages reliably. In particular, structural differences between import names and installable package names can prevent automatic resolution, while installation guidance is rarely provided.

These findings suggest that LLMs can identify plausible libraries but do not necessarily address the

practical integration challenges associated with adopting them. In this sense, LLMs may simplify dependency selection while transferring additional verification and resolution work to developers. The following chapter presents an initial tool-based response to this limitation by providing real-time information about packages introduced through AI-generated code.

4.2 Introduction

Modern software development heavily relies on open source libraries that provide reusable functionalities through well-defined modules, significantly reducing development time and effort [1, 2, 3]. While libraries can help speed up development tasks, they also introduce dependencies (interconnections between code components) that can lead to increased complexity and dependency management challenges [17, 120, 12].

One critical aspect of dependency management is library selection [121]. Choosing the right library impacts factors like code maintainability, performance, and security. Previous studies have explored how developers select libraries, and highlighted primarily ad-hoc processes based on past experiences, expert advice, and online resources [122, 123].

Decisions around dependency adoption are influenced by factors such as functionality, community support, and maintenance compatibility [10, 124, 121]. Previous studies reveal that maintenance tasks like updates and vulnerability migration are often hindered by risks of breaking changes [38, 125, 126] and complexity from transitive dependencies [48, 127, 13, 128]. Automation and tooling (e.g., Dependabot [1]) have been introduced to alleviate these issues, but they frequently fall short because of alert fatigue, false positives, or lack of actionable context [63, 67, 68, 85, 129].

In parallel, the growing adoption of LLMs as programming assistants introduces new possibilities for addressing these challenges. LLMs are increasingly used to assist with code generation, and studies show their potential to enhance productivity through capabilities like code completion and search [130, 131, 132]. However, their impact on software dependencies remains underexplored. In particular, even when LLM-generated code is syntactically correct and the recommended libraries part of the generated code are meaningful, dependency-related issues may still arise at the level of usability and integration. For example, models may introduce unnecessary libraries, omit installation guidance, or recommend imports that don't cleanly map to installable packages. As these models become central in development workflows, it is therefore important to

[1] https://github.com/dependabot

assess not only whether they generate working code, but also whether their library recommendations are easy to integrate in practice.

Our work aims to explore a specific aspect of LLM functionality: their ability to recommend software libraries during code generation. In this paper, we evaluate how six state-of-the-art LLMs recommend software libraries in a conversational coding-assistance setting by conducting an empirical study in which each model generates Python code for a set of real-world coding questions derived from Stack Overflow [133]. We focus specifically on Python and on the conversational question-answering setting, where developers ask programming questions in natural language and receive standalone code solutions. This setting allows us to study how LLMs recommend libraries when they are not constrained by an existing project context or import graph. Our analysis is guided by the following research questions:

RQ1: What types of software libraries are recommended by LLMs? Software libraries play a critical role in the security, performance, and maintainability of software projects. Thus, we examine the types of libraries recommended by LLMs and how frequently they rely on Python's standard library versus third-party. Our results suggest that LLMs tend to recommend third-party libraries more often than standard libraries. Across all models evaluated, 54% of imports were third-party, 41% were standard, and 5% were unresolvable. This suggests a strong reliance on external packages rather than built-in functionality provided by Python.

RQ2: What are the characteristics of third-party libraries recommended by LLMs? Third-party libraries introduce external dependencies that require separate installation and ongoing maintenance, which makes their quality and licenses critical in production contexts. To assess whether the libraries imported in LLM-generated code are correct and sustainable, we evaluate each third-party dependency along three key dimensions: popularity (e.g., GitHub stars, dependents), maintenance (e.g., age, update frequency), and licensing. Our findings show that LLMs tend to favour well-established and permissively licensed libraries that have high community adoption and low maintenance overhead.

RQ3: Why do some libraries recommended by LLMs not work out of the box? Some libraries recommended by LLMs do not work out of the box, which can be disruptive to the development workflow and negatively affect trust in AI-based programming assistants. In this RQ, we investigate the root causes of these unresolved imports by analyzing cases where the recommended library could not be mapped to a known standard or third-party package. We found that most of the unresolved imports

stem from structural conventions in the Python programming language, where the import name differs from the package name. We also identified one case involving a module-level import error. While the code is valid, missing installation instructions limit usability. Only a few models, such as DeepSeek V3, included installation guidance.

Our findings show how effectively LLMs recommend software libraries and the factors that influence their behavior in this context during code generation. This contributes to a better understanding of model capabilities and limitations, and provides actionable insights for developers integrating LLMs into their workflows. We also highlight areas where models consistently produce reliable outputs, as well as scenarios where users should be cautious due to missing context or unresolved dependencies.

4.2.1 Our Contributions

In summary, we make the following key contributions in this chapter:

- An empirical library characterization of LLM-generated Python code across six state-of-the-art models, quantifying how they select, import, and *use* libraries (including over-importing, unresolved and aliased imports, and the popularity, maintenance, and licensing characteristics of recommended dependencies).
- A set of actionable recommendations for developers using LLMs to streamline their workflows, including guidance for potential issues with LLM-suggested libraries.
- A dataset of LLM-generated Python code from six state-of-the-art LLMs that enables a direct comparison of code outputs and library selection practices, available online[2].

4.2.2 Chapter Organization

The remainder of this chapter is organized as follows. Section 4.3 describes our dataset and methodology. Section 4.4, Section 4.5, Section 4.6 present the findings of our three research questions. Section 4.7 discusses the implications of our findings along with our recommendations. Section 4.8 discusses related work. Section 4.9 outlines the threats to the validity of our study. Finally, Section 4.10 concludes this paper.

[2]https://zenodo.org/records/18022086

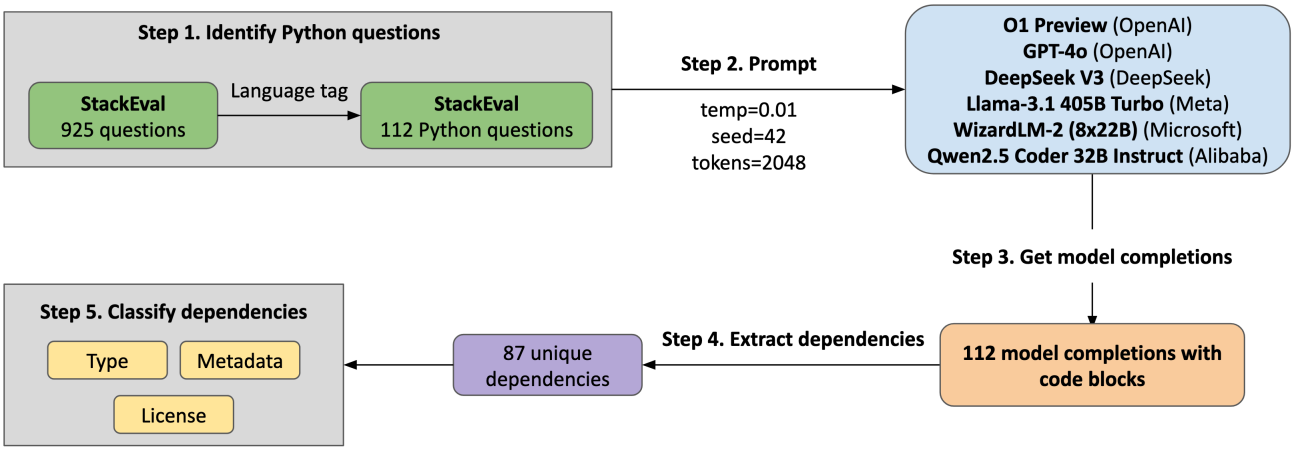


Figure 4.1: An overview of our approach for filtering and curating our dataset.

4.3 Methodology

In this section, we describe the steps to curate the dataset used in our study and the experimental setup.

4.3.1 Dataset

Figure 4.1 presents an overview of our approach for curating the dataset used in this study and analyzing the dependencies generated by LLMs. We use the StackEval benchmark suite curated by Shah et al. [134], which contains questionanswer pairs sourced from Stack Overflow. The benchmark is designed to evaluate LLM performance on coding tasks and includes 925 questions sampled between January 2018 and September 2023. Each question includes a title, body, at least one upvote, and an accepted answer that also received one upvote.

We selected this dataset because Stack Overflow questions reflect real-world programming scenarios where developers seek help with writing, fixing, or improving code. These questions may involve software libraries either explicitly (e.g., *Difference between numpy power and ** for certain values*) or implicitly through the broader coding task. This allows us to examine how LLMs introduce and use libraries in a practical question-answering setting, rather than through tasks artificially constructed around dependency selection. In addition, StackEval matches a common interaction mode for LLM assistants, namely chat-style programming questions, and each item includes an accepted human-written answer that we use as a reference point in our ground-truth comparison (Section 4.7). Because accepted answers vary in whether they contain imports, we use the full Python subset when analyzing whether and how models introduce libraries in their

own completions. When comparing model-generated imports against accepted answers, however, we restrict that comparison to the subset of questions whose accepted answers contain imports.

At the same time, this dataset is not a benchmark in which every task explicitly asks the model to select a library. In many questions, library choice is secondary to the broader programming problem. We therefore do not treat StackEval as a comprehensive benchmark of library selection in isolation. Instead, we use it to study library-related behavior in a realistic conversational coding-assistance setting, where dependency decisions may arise implicitly but still affect usability, installation effort, and integration burden.

More broadly, we acknowledge that benchmark-based LLM evaluations have known limitations, including possible contamination, benchmark-specific overfitting, and uncertainty about whether strong benchmark performance reflects true generalization. Prior work has raised these concerns both for LLM benchmarks in general and for code-generation benchmarks specifically [135, 136]. Although our methodology could be adapted to such benchmarks to study other usage modes, in this work we focus specifically on StackEvals Stack Overflow-style Q&A setting.

We filtered StackEval to retain Python-related questions using the language tags. Python was chosen due to its popularity, rich ecosystem of third-party libraries, and the strong performance of LLMs in generating and understanding Python code [137, 138, 139, 21]. We deliberately restrict our analysis to a single language and ecosystem for methodological reasons. This allows us to isolate model behavior more clearly, rather than conflating it with differences across language ecosystems, package registries, and import conventions. Including another ecosystem such as npm would require separate parsers, heuristics, and metrics, and would make it difficult to disentangle differences in model behavior from differences in ecosystem structure, package managers, and import conventions. We therefore treat Python as a representative ecosystem and leave cross-language comparisons to future work. Thus, our final dataset consists of 112 Python programming questions.

To better understand the kinds of problems represented in this subset, we inspected the question-level metadata provided by StackEval (task type and difficulty). The 112 Python questions span four task types: debugging (49 question, 43.8%), implementation (43, 38.4%), conceptual understanding (14, 12.5%), and optimization (6, 5.4%). Most questions are labelled as intermediate (87, 77.7%), with the remainder split between beginner (16, 14.3%), and advanced (9, 8%). This yields 11 distinct (type, level) combinations). Question texts are also non-trivial in length, and range from 143 to 3,845 characters (median 790) and from 5 to 116 lines (median 32). Overall, while the Python subset is small in comparison to the overall set, it

Table 4.1: Overview of the evaluated LLMs.

Model	Provider	Parameters	Released	Cutoff	Key capabilities
DeepSeek V3	DeepSeek AI	685B	Dec. 2024	July 2024	Mixture-of-experts; general-purpose; code; reasoning
GPT-4o	OpenAI	~1.8T	Apr. 2024	Dec. 2023	Multimodal; general-purpose; code; reasoning
Llama 3.1	Meta	405B	July 2024	Dec. 2023	Instruction-tuned; general-purpose; text; code
O1 Preview	OpenAI	~175B	Sept. 2024	Oct. 2023	Reasoning-optimized
Qwen2.5	Alibaba	32B	Sept. 2024	Mar. 2024	Instruction-tuned; strong code support
WizardLM-2	Microsoft	141B	Apr. 2024	Oct. 2023	Mixture-of-experts; instruction-tuned; code

covers a diverse mix of realistic Python debugging, implementation, and conceptual tasks at predominantly intermediate complexity.

4.3.2 Experiment Setup

We evaluate six state-of-the-art LLMs, including two proprietary models and four open-source models. The proprietary models are O1 Preview [140] and GPT-4o [141], both developed by OpenAI. The open-source models include DeepSeek V3 [142] from DeepSeek AI, Llama 3.1 405B Turbo [143] from Meta, WizardLM-2 8x22B [144] from Microsoft, and Qwen 2.5 Coder 32B Instruct [145] from Alibaba. Table 4.1 provides an overview of their key characteristics and capabilities. All model inference was performed through the TogetherAI platform. This applies to both the proprietary and open-weight models evaluated in our study.

“‘latex “‘

To collect model completions, we follow the approach of Shah et al. [134]. Each model is prompted with the selected 112 Python questions from StackEval using only the question title and body, without additional instructions or examples. In line with Shah et al., we intentionally avoid any further prompt engineering beyond what is present in the original Stack Overflow question: we do not prepend meta-instructions such as “include installation instructions“ or “explain your imports“. This setup reflects realistic usage, where developers typically pose natural language questions without carefully crafted prompts [23], and allows us to evaluate model behavior in conditions that are minimally guided and standardized across models. The parameter configuration has a near-deterministic temperature (0.01), a fixed seed (42), and outputs limited to 2,048 tokens. These settings follow the configuration used in the original StackEval benchmark [134], and are chosen to minimize the randomness of the output and ensure reproducibility. While higher temperatures

and multiple samples are sometimes used to explore the diversity of generated code, our goal in this study is to characterize each model's most likely completion for a given question, analogous to the single answer a developer would typically inspect. We therefore generate one low-temperature completion per question and leave the analysis of full output distributions under higher temperatures as complementary future work.

We emphasize that this setup is intended to characterize model behavior in a conversational coding-assistance workflow, not to measure the maximum achievable performance of tool-augmented or agentic LLM systems. In particular, we do not provide execution feedback, package-resolution tools, linters, or iterative repair loops. We chose this design to isolate the models own library recommendation behavior under a minimally engineered prompt, and because single-turn question-answering remains a common way developers interact with LLM assistants in practice. Evaluating how the same dependency-related behaviors change under agentic scaffolding is an important direction for future work.

After generating responses, we first extract code blocks from each completion using a regular expression over fenced code (e.g., ```` ```python ```` blocks). We then parse each code block with Python's `ast` module and identify all `import` and `from X import Y` statements and normalize them to top-level names (e.g., `pandas.core.frame` → `pandas`). This allows us to focus on high-level usage patterns of libraries rather than individual submodules. Across the 112 questions and all models, this process yields 87 unique libraries. In our analysis, we treat each library as an individual case, regardless of how many times it appears across questions or models. For per-model analysis, the same library may appear in multiple completions, whereas for aggregated analysis we focus on the set of unique libraries recommended across all completions. Finally, we classify each library by type (standard, third-party, unresolved) and collect metadata such as its ecosystem and license; this classification procedure is detailed in RQ1.

4.4 RQ1: What types of software libraries are recommended by LLMs?

Motivation. As developers increasingly turn to AI-based programming assistants to streamline their development workflows, the choice of libraries included in AI-generated code becomes critical. These libraries directly impact the resulting code's performance, security, and functionality [17, 11, 146]. This research question aims to examine the types of libraries included in LLM-generated code and how frequently models rely on Python's built-in standard libraries versus third-party or unidentifiable libraries. Understanding this helps evaluate the reliability and integration complexity of LLM-generated library imports in real-world

development settings.

Approach. We classify each imported library as one of the following: (1) *standard*, meaning it is bundled with the default Python installation, (2) *third-party*, meaning it is installable via the Python Package Index (PyPI), or (3) *unresolved*, meaning it cannot be mapped to a known standard or third-party library. To perform this classification, we use the `stdlib-list` package to identify standard libraries (for Python 3) and query the PyPI API for the rest. Libraries that are not found in either are labeled as *unresolved*.

Results. Table 4.2 presents the distribution of dependency types. The first row shows the results for all combined models. We identified a total of 87 distinct dependencies of which 47 (54.0%) were *third-party* libraries, and 36 (41.4%) were *standard* libraries. This suggests that models are more likely to generate code that imports external packages rather than relying on built-in Python functionality. Moreover, four (4.6%) of the libraries were classified as *unresolved*, indicating these libraries are neither identified as *standard* nor *third-party*. We further investigate this category and its implications in RQ2.

Table 4.2: Classification of imported libraries in LLM-generated code.

Model	**Std.**	**3rd-Party**	**Unresolved**	**Total**
All models	36 (41.4%)	47 (54%)	4 (4.6%)	87
DeepSeek V3	24 (42.1%)	30 (52.6%)	3 (5.3%)	57
GPT-4o	26 (40.6%)	35 (54.7%)	3 (4.7%)	64
Llama 3.1	22 (40.7%)	28 (51.9%)	4 (7.4%)	54
O1 Preview	18 (39.1%)	26 (56.5%)	2 (4.3%)	46
Qwen2.5	21 (36.2%)	34 (58.6%)	3 (5.2%)	58
WizardLM-2	19 (38.8%)	28 (57.1%)	2 (4.1%)	49

Looking at the results per model, we observe a consistent pattern across all six LLMs: third-party libraries account for the largest proportion of imports. GPT-4o recommends the most unique libraries overall (64), with *standard* libraries accounting for 40.6%, third-party accounting for 54.7%, and three instances of unresolved imports. Llama 3.1 has the highest proportion of *unresolved* libraries (5), though this is only marginally higher than the other models, which range between 2 and 4. This small variation is not substantial enough to indicate systematic differences between models. Nonetheless, its proportions of *standard* and *third-party* imports remain comparable to other models. O1 Preview has the lowest total number of distinct dependencies (46), but maintains a similar overall trend, with a majority of third-party recommendations (56.5%). Qwen2.5 has the highest proportion of *third-party* dependencies (58.6%), which could have implications for integration complexity and maintenance. WizardLM-2 has the smallest proportion of unresolved (4.1%), but also shows

a high third-party usage rate (57.1%). Overall, we can see that all models tend to use *third-party* libraries more often than *standard* with varying degrees (e.g., Qwen2.5, WizardLM-2).

Answer to RQ1. Our analysis shows that all six LLMs generate code that predominantly relies on third-party libraries, ranging between 51.9% and 58.6% of imports across models. Out of 87 distinct libraries identified across all completions, 54.0% were third-party, 41.4% were standard, and 4.6% were unresolved. GPT-4o generated the broadest set of libraries, while Llama 3.1 has the highest proportion of unresolved.

4.5 RQ2: What are the characteristics of third-party libraries recommended by LLMs?

Motivation. Unlike standard libraries, third-party libraries introduce external dependencies that must be installed and maintained separately. Their characteristics, such as popularity, maintenance activity, and licensing, are known to significantly affect the long-term stability and legal compliance of software projects [10]. Thus, in this research question, we aim to evaluate whether LLMs tend to recommend third-party libraries that are mature, reliable, and safe for production.

Approach. We analyze each third-party library along three key dimensions based on prior work [11, 124]: popularity, maintenance, and licensing. For popularity, we examine the number of GitHub forks, stars, and dependent projects. For maintenance, we look at the number of dependencies, age (calculated from the library's first release date to the metadata collection date (May 2024)), version release frequency (releases per month from first release to our May 2024 snapshot, using libraries.io version timestamps), and SourceRank [3] score (provided by libraries.io). Finally, we assess licensing information by assigning SPDX license identifiers and grouping them into categories such as permissive, copyleft, and weak copyleft.

Results. In this section, we analyze the characteristics of third-party libraries recommended by LLMs. As mentioned in RQ1, across all models, we observe 47 distinct packages and 343 third-party import occurrences. The distribution is skewed towards widely used packages: the five most frequent libraries (`pandas`, `numpy`, `sklearn`, `torch`, and `django`) account for 51.9% of all third-party import occurrences, and the top ten libraries (adding `matplotlib`, `boto3`, `xgboost`, `imblearn`, and `pyspark`) account for 63.8%.

[3] https://github.com/librariesio/libraries.io/issues/1916

The remaining 36.2% of occurrences are spread across 36 additional libraries that each appear between one and six times. We provide the complete list of libraries and their frequencies in our replication package to make this distribution fully transparent. Thus, the results discussed below should be interpreted as indicative of those libraries rather than exhaustive characterizations of the entire Python ecosystem. Table 4.3 presents the median values of key dependency characteristics for each model as well at the aggregated results. Below, we discuss the results in more detail.

Popularity. The popularity metrics (forks, stars, and dependents) show how widely adopted and trusted a library is within the Python ecosystem. They reflect developer interest (e.g., stars), collaborative engagement (e.g., forks), and real-world usage (e.g., dependents). The aggregated medians across models show that LLMs generally recommend libraries that are well-established and commonly used, with a median number of forks of 1,473, 6,962 stars, and 1,140 dependents.

At the per-model level, we observed that the median number of stars ranges from 5,471 to 8,528. This indicates that all models recommend libraries with a notable level of visibility and level of adoption. As shown in Table 4.3, some models fall toward the high end of this range (e.g., Qwen2.5 and Llama 3.1), and some fall toward the lower end (e.g., WizardLM-2 and DeepSeek V3), suggesting that they might recommend more specialized or context-specific libraries. Other models have popularity metrics that are close to the overall medians (e.g., GPT-4o on all popularity metrics and O1 Preview on forks and dependents), which suggests that their recommendations are still broadly visible and adopted, but not skewed towards highly popular or niche options.

Maintenance. In terms of maintenance, the results show that most models recommend libraries that are mature and self-contained. The median number of dependencies across all models is low (1.0) and never exceeds 3.0 at the per-model level. This indicates that the recommended libraries are lightweight in terms of transitive dependencies, reducing integration risk and potential for version conflicts in downstream projects [10].

The SourceRank metric, which reflects a library's overall quality and development activity, is relatively consistent across models, ranging from 22 to 24, with an overall median of 23 (out of a maximum of 30). This indicates that LLMs tend to select libraries that are well-maintained and well-regarded in the community. The age of recommended libraries is consistently high, with median values across models above 7 years, which suggests LLMs tend to recommend stable, long-standing libraries over newer ones. From a developer's perspective, this behaviour may promote trust and ensure compatibility, especially in production environments

Table 4.3: Median characteristics of recommended third-party libraries.

	Popularity			Maintenance			
Model	**Forks**	**Stars**	**Dependents**	**Dependencies**	**SourceRank**	**Age (months)**	**Version frequency**
All models	1,473	6,962	1,140	1.0	23	95.0	0.64
DeepSeek V3	1,292	5,607	1,068	0.5	22.5	94.9	0.62
GPT-4o	1,473	6,962	1,114	2.0	23	97.7	0.58
Llama 3.1	1,682	8,528	1,068	2.5	22	92.9	0.62
O1 Preview	1,682	8,073	1,287	0.0	22.5	87.3	0.77
Qwen2.5	1,430	8,073	2,080	1.5	24	99.5	0.77
WizardLM-2	1,292	5,471	1,287	1.5	22	104.1	0.77

where stability and maturity are important.

Version release frequency remains below one release per month for all models, with medians ranging from 0.58 to 0.77 (˜17 to 23 days). These values imply that most libraries receive regular but not overly frequent updates, which is often preferred by developers who want to avoid breaking changes, frequent manual interventions, or unexpected regressions [103]. The observed range for version frequency does not appear to correlate directly with the training cutoff dates. For instance, O1 Preview (cutoff: October 2023) and Qwen2.5 (cutoff: March 2024) both recommend libraries with identical median version frequency per month (0.77). This suggests that other factors like training data composition, domain coverage, or fine-tuning strategies may have a greater influence on the kinds of libraries selected by each model.

License. We next examine the licenses associated with the third-party libraries recommended by each model. Of the 181 license instances across all models, 164 (91%) fall into the permissive category, primarily MIT, BSD, and Apache-2.0 (we consolidate BSD-2-clause and BSD-3-clause into a single BSD category). Permissive licenses impose minimal obligations such as attribution and allow use in proprietary software, so they are generally preferred in industrial settings. In contrast, copyleft licenses (e.g., GPL family) require derivative works to remain open source, while weak-copyleft licenses (e.g., PSF-2.0, MPL-2.0) sit in between by allowing proprietary use but imposing conditions on modifications to the licensed component. DeepSeek V3 is the only model that recommended a copyleft-licensed library (`certifi`), and each model recommended a weak-copyleft PSF-2.0 library (`matplotlib`).

However, each model consistently recommends the same three libraries for which the license is not explicitly declared on PyPI (*unresolved*): `myapp`, `sklearn`, and `imblearn`. For `myapp`, the documentation

indicates that it is a test library [4], which could explain why no license is explicitly declared. We observed that the models use this name as a placeholder instead of referring to the actual library. We discuss this occurrence in more detail in RQ2.

In the case of `sklearn`, the corresponding PyPI page indicates that the library is deprecated [5], which led us to classify it as having no declared license. However, we found that `sklearn` is an import alias for the actively maintained `scikit-learn` library. This means that while developers install the library using the name `scikit-learn`, it is imported in Python as `sklearn`. As a result, querying `sklearn` directly on PyPI leads to a deprecated library. In reality, `scikit-learn` is distributed under a permissive type of license (BSD). We discuss the implications of such aliasing further in RQ2. Finally, for `imblearn`, the PyPI documentation recommends using `imbalanced-learn` instead [6], yet no explicit deprecation notice is shown and unlike `scikit-learn`, we found no evidence of aliasing for this case in our dataset. Thus, the results show that LLMs recommend libraries that are widely permissive, which is ideal in policy-sensitive production settings, but there are a few cases where the license can be more restrictive (e.g., copyleft) or not explicitly declared, which can lead to potential legal challenges. Practitioners should actively verify the license of the libraries recommended by LLMs they integrate into their codebases.

Answer to RQ2. Our findings show that LLMs predominantly generate dependencies on third-party libraries that are popular, well-established, and permissively licensed. While these characteristics are generally reassuring, they also indicate that LLM-generated code often expands the external dependency surface, making downstream usability and integration considerations important. Across all models, the recommended libraries are stable (median version frequency of 0.64), have minimal transitive dependencies (median of 1 dependency), and are well-regarded by the community (median SourceRank of 23).

[4] https://pypi.org/project/myapp/
[5] https://pypi.org/project/sklearn/
[6] https://pypi.org/project/imblearn/

4.6 RQ3: Why do some libraries recommended by large language models not work out of the box?

Motivation. In RQ1, we observed that 4.6% of libraries in LLM-generated code do not function out of the box, meaning that they are neither available on PyPI (i.e., *third-party* library) nor included in the standard Python distribution (i.e., *standard* library). In a baseline conversational workflow, this can create friction, hinder execution, and reduce trust in the output [139]. Thus, in this RQ, we examine the underlying reasons why such libraries are included in LLM-generated code. Understanding this is important for improving the reliability of LLMs in programming tasks and their practical utility in real-world settings. RQ3 therefore aims to identify and characterize the common sources of unresolved library recommendations in LLM outputs, and to examine their practical implications for dependency resolution.

Approach. To understand why some libraries recommended by LLMs do not work out of the box, we analyzed each case identified as *unresolved* in RQ1 (i.e., neither *third-party* nor *standard*). Specifically, we qualitatively examined the context in which each library was recommended by analyzing the associated Stack Overflow question used in the prompt, including its body, code snippets, and accepted answer. This process allowed us to infer whether the recommendation stemmed from a misunderstanding of naming conventions, structural issues in the import statement, or other contextual factors. Based on this, we developed a set of mutually exclusive categories that characterize the underlying reasons why certain libraries in LLM-generated code fail to resolve correctly. Since this categorization relied on concrete, factual properties, the first author performed the initial labeling, and the results were reviewed and validated by the second author. This led to the identification of two primary categories:

- **Alias**: The extracted library name corresponds to an alias used in Python import statements, rather than the actual name of the library used for installation. For example, `sklearn` is used as an alias for the `scikit-learn` library. While the model's suggestion is functionally correct, alias-based imports may lead to errors during dependency resolution.

- **Module**: The extracted name refers to an internal module or subcomponent within a third-party library. These cases occur when the model suggests imports from a specific file using non-standard or path-based import patterns. These cases also happen with implicit imports, which is when a module is imported directly without referencing the parent library. Since the name used in the import does not

match the actual installable library, it can lead to a failure in dependency resolution if the parent library is not installed.

Results. We first present the underlying causes of unresolved imports, and then discuss their practical consequences in terms of installation guidance and user burden. Table 4.4 shows the *unresolved* libraries identified in RQ1, categorized into the two underlying causes discussed above: *alias* and *module*. The *alias* category accounts for most of the *unresolved* cases across all six models, with a total of 24 occurrences (3 unique). The most frequent alias is `cv2` used for the `opencv-python` library, which appears in all models. Similarly, `dateutil` is used as an alias for `python-dateutil`, also appearing in all models. The last alias, `yaml`, used for the library `PyYAML`, appears in half of the models (GPT-4o, Llama 3.1, and Qwen2.5).

These imports are functionally correct, but can lead to confusion since the alias used in the import statement does not match the actual installable library name. To further investigate the practical consequences of this, we manually reviewed the responses generated by the models for each of the 24 alias cases. We found that only 5 of these completions included an installation step (e.g., `pip install opencv-python`), which occurred in outputs from DeepSeek V3, GPT-4o, and Qwen2.5. This indicates that models may often assume implicit knowledge about the correct library to install and places the burden on developers to manually infer the library before they execute the code. This is especially important in educational contexts, where users may lack the knowledge to resolve missing dependencies.

In contrast to *alias* cases, we identified a single instance categorized as *module*. This occurred in a response generated by Llama 3.1, where the model produced the statement `from client import EmailageClient`, which resulted in the extraction of `client` as the top-level dependency. However, `client` is not a standalone installable library; it is a submodule nested within the `emailage` library. Thus, the import statement is not incorrect in itself, but attempting to install `client` on its own would fail. This is a challenge that occurs with implicit imports: the module is imported without an explicit reference to its parent library. In this particular case, Llama 3.1 did not provide additional installation steps in the completion, leaving it to the user to manually infer the library to install. It could be argued that the user might possess the necessary knowledge given that they formulated the original question, but this assumption is problematic. Similar to the alias cases, it places the burden of dependency resolution on the user and undermines the practical reliability of LLMs in real-world settings.

Practical implications and model differences. Although all models produce some *unresolved* libraries,

Table 4.4: Occurrences of alias and module cases per model.

Model	# Alias Occ.	Alias instances	# Module Occ.	Module instances
DeepSeek V3	4	`cv2` (1), `dateutil` (3)	0	–
GPT-4o	5	`cv2` (2), `dateutil` (2), `yaml` (1)	0	–
Llama 3.1	5	`cv2` (1), `dateutil` (2), `yaml` (1)	1	`client` (1)
O1 Preview	3	`cv2` (1), `dateutil` (2)	0	–
Qwen2.5	5	`cv2` (1), `dateutil` (3), `yaml` (1)	0	–
WizardLM-2	2	`cv2` (1), `dateutil` (1)	0	–
Total	24		1	

their behavior varies slightly. For instance, GPT-4o, Qwen2.5, and Llama 3.1 tend to rely more heavily on alias imports. However, these aliases (e.g., `cv2`, `dateutil`, `yaml`) are widely adopted and well-established libraries. Interestingly, only these three models used the `yaml` alias, despite all models receiving the same input, which suggests that they may have a more accurate understanding of the `PyYAML` library to recommend it for the same use case. In contrast, WizardLM-2 and O1 Preview produced the fewest *unresolved*, which could suggest they are more conservative about importing libraries. This is reflected in RQ1, where these models produced the fewest unique imports overall.

DeepSeek V3, which produced a moderate number of aliases (4), was one of the few models that included installation guidance (with GPT-4o and Qwen2.5), indicating that the model can both produce correct imports and correctly map aliases to their installable libraries. This may indicate that models which include installation guidance in their completion anticipate potential friction during code execution and proactively address it. While this does not guarantee robustness, it suggests a model-level difference in how much contextual support is embedded model's output, a feature that enhances usability and trust.

Our results show that all models are capable of generating technically correct code, but some models may place a greater burden on the user to interpret and resolve dependencies, especially when explicit installation instructions are omitted from the output.

Answer to RQ3. Our findings show that the majority of unresolved library references stem from aliasing, where the import statement refers to a library name that differs from its installable package name. We also identified one case involving an implicit module import, in which a submodule of a library is referenced without its parent library. We conclude that while the code produced is syntactically correct, it lacks sufficient context to ensure seamless execution and places a burden on users to resolve dependencies manually. Only a few models, such as DeepSeek V3, offered installation guidance, highlighting key differences in practical usability across models.

4.7 Discussion

In this section, we discuss the implications of our work and propose recommendations for practitioners and researchers in the context of LLMs and dependencies.

4.7.1 How do LLMs perform on unseen questions?

A central threat to validity in studies using LLM-generated code is the possibility of data leakage (i.e., models having been exposed to the same or similar questions during training). To address this, we repeated our analysis on StackUnseen, a separate dataset specifically curated to reduce this risk [134]. This dataset complements StackEval by focusing on more recent Stack Overflow questions (as of October 2024), which are less likely to have been present in the models' training corpora. From the 494 questions in StackUnseen, we extracted 71 Python-related questions and applied the same methodology as used in the StackEval analysis, which yielded a set of 66 unique libraries. Among these 66 libraries, 38 are third-party, 25 are standard-library modules, and 3 are unresolved. As a reminder for comparison, StackEval contains 87 distinct libraries (47 third-party, 36 standard, and 4 unresolved).

Thus, in this analysis, we examine whether the characteristics of the third-party libraries differ between StackEval and StackUnseen. In particular, we ask whether model behavior changes when faced with unseen questions, as reflected in the characteristics of the libraries they recommend. A substantial shift in these characteristics could indicate a reliance on previously seen data and would raise concerns about the generalizability of our results.

To achieve this, we compared StackEval and StackUnseen at the *per-model* level. For each model, we

considered unique third-party libraries (deduplicated by `(model, dep)`) and compared the distributions of library characteristics between the two datasets using MannWhitney U tests. We repeated this test for each characteristic (number of stars, forks, dependents, dependencies, SourceRank, age in months, and version frequency) and applied a Holm correction for multiple comparisons across all modelmetric combinations.

Across all models and characteristics, we found no statistically significant differences between StackEval and StackUnseen (all Holm-corrected $p \geq 0.05$). While individual medians can vary slightly between the two datasets, these shifts are small and not statistically reliable after correction. In other words, for a given model, the libraries recommended on unseen questions exhibit similar popularity and maintainability characteristics as those recommended on historical questions. However, aggregate characteristics do not capture how often recommendations fail to work out of the box (i.e., unresolved). Thus, in addition to evaluating the characteristics of recommended libraries (RQ1), we examined the subset of libraries that were identified as *unresolved* (i.e., not clearly identifiable as *standard* or *third-party*). These represent edge cases where model recommendations may not work out of the box and require manual resolution from the user. As seen in RQ2, in StackEval, we identified 24 such cases (4.3% of the recommendations), primarily due to aliasing and module-level imports.

In contrast, StackUnseen yielded only 4 *unresolved* cases (1.2%). Aliasing accounts for 50% of these cases (`browsermobproxy`, `seleniumrequests`), while placeholder usage accounts for the other 50% (`some_module`). In this context, a placeholder refers to a stand-in or generic example used by the model that is meant to be replaced by the user with a real library. This reduction in `unresolved` cases may be due to models defaulting to more popular or generic imports when faced with truly unseen data. It could also be partially influenced by the smaller sample size in StackUnseen, which reduced the likelihood of observing edge cases. Therefore, it is not possible to draw firm conclusions about the influence of unseen questions on this outcome.

Our findings have practical implications for developers relying on LLM-based code generation tools. First, the consistency in library characteristics between StackEval and StackUnseen shows models exhibit relatively robust behavior in their library recommendations, even when answering questions unlikely to have been seen during training. This reinforces the reliability of our findings and supports the use of StackEval as a benchmark for future evaluations. Second, although rare, the presence of *unresolved* cases, even in unseen data, highlights the need for caution when integrating LLM-generated code into software projects. Developers should be aware that LLMs may suggest libraries that do not install cleanly or require specific domain knowledge to resolve. Incorporating mechanisms that prompt LLMs to clarify or justify their library

usage could improve usability in real-world development workflows.

4.7.2 Are LLMs Importing Libraries They Do Not Use?

To better understand whether the imported libraries are actually used by the generated code, we perform a static analysis over all model completions. Building on the import-extraction pipeline described in Section 4.3.2, we take each completion, inspect the AST to locate import statements, and then check whether the bound name is ever read in the body of the generated code block (e.g., used in an attribute access such as `pandas.DataFrame` or as a bare function or class imported via `from X import f`). We then classify each import as used or unused (i.e., dead import) and aggregate the results per model.

Across all models, the median number of imports per completion is two, indicating that the generated snippets are not heavy in terms of declared dependencies. Table 4.5 summarizes how often these imports remain unused. As shown in the column ”All imports”, the majority of the models leave around 5-7% of their imports unused, with WizardLM-2 at the highest proportion with 12%. These results suggest that the majority of imported names are used in the corresponding generated code blocks.

Table 4.5 also breaks down unused imports by dependency type (standard, third-party, unresolved). Unused standard-library imports range, for the majority, between 2 and 6%, and even 0 in the case of O1 Preview. Third-party imports have higher dead-import rates, with values between 4 and 15% depending on the model. For example, WizardLM-2 leaves about 14.5% of its third-party imports unused, compared to 5.6% for standard-library imports. This indicates that, in general, when models ”over-import”, they tend to do so with external libraries rather than standard libraries, which is more concerning from a dependency-management perspective. Finally, the ”unresolved” column corresponds to libraries that could not be mapped to standard or third-party libraries. This group shows higher unused ratios for some models (e.g., O1 Preview and Llama 3.1) but represent a small subset of imports, so these ratios should be interpreted with caution.

Our analysis correctly marks the top-level `import pymongo` as unused, since the body of the code only ever refers to the exception class imported via `from pymongo.errors import BulkWriteError`. From a runtime perspective, the package is loaded via the second import, so the first line is redundant. This pattern is consistent with several generation behaviours: the model ”stitches together” a generic import template (`import pymongo`) with a more specific error-handling idiom (`from pymongo.errors import BulkWriteError`) [147]. This pattern mirrors common tutorial code that imports both the parent package and one of its submodules, even when only the submodule is needed. *Our StackEval vs. StackUnseen results*

in Section 4.7.1 suggest that this behaviour is better explained by models learning common import patterns from many similar examples in their training data than by direct memorization of a specific historical solution, although some overlap with training data cannot be entirely ruled out. Thus, when lacking project-wide context, the model tends to err on the side of importing more than necessary rather than risk a missing symbol [148]. In all of these cases, the resulting snippet is syntactically correct, but it leaves behind imports that do not contribute to the behavior of the solution. This reveals a tendency for models towards "template-like" code generation that can inflate the dependency surface, which is a form of dependency bloat that practitioners and tool builders should monitor or mitigate.

Overall, our analysis shows that the import statistics are not dominated by unused imports. Most imported libraries are actually referenced in the generated code, which indicates that models are generally not adding dependencies arbitrarily. This strengthens our interpretation of the dependency-level findings. At the same time, the presence of unused imports (especially among third-party libraries) still matters from a dependency-management perspective, because even a small number of unnecessary external dependencies can increase integration efforts, maintenance, and add friction overall. We therefore interpret these findings not as evidence of widespread dependency bloat, but as evidence of a more targeted form of integration friction that arises when external libraries are introduced without clear necessity.

Listing 4.1: A simplified example of an unused import in a generated snippet.

```
import pymongo
from pymongo.errors import BulkWriteError


try:
    results = collection.insert_many(documents, ordered=False)
except BulkWriteError as e:
    for error in e.details["writeErrors"]:
        if error["code"] == 11000:  # duplicate key error
            pass  # ignore duplicate key
```

4.7.3 Library-Level Correctness Related to Ground Truth

In Section 4.7.2, we showed that the libraries imported by LLMs are, in most cases, actually used in the generated code. We now compare these imports against the accepted Stack Overflow answers to assess how

Table 4.5: Proportion of unused imports per model, overall and by dependency type.

Model	All imports	Std.	Third-party	Unresolved
DeepSeek V3	6.0	5.4	4.5	17.6
GPT-4o	5.7	1.9	5.4	23.1
Llama 3.1	6.7	2.0	6.0	28.6
O1 Preview	5.7	0.0	1.6	41.7
Qwen2.5	5.9	2.0	8.3	0.0
WizardLM-2	12.1	5.6	14.5	7.7

closely model-generated libraries align with the benchmark ground truth. In other words, we want to see how often the Python tasks actually require any libraries, when models introduce libraries even though the ground truth does not, and how closely the libraries they choose align with those used in the ground-truth solution. In this analysis, we use the accepted answer from the corresponding Stack Overflow question as a proxy for the ground truth.

To achieve this, we perform a post-hoc comparison between the imports in each model completion and those in the accepted Stack Overflow answer. Using the same code-block extraction and AST-based import identification as in Section 4.3.2, we extract, for every question, (1) the set of modules imported by the accepted answer and (2) the set of modules imported by each model. For each question–model pair, we compare these two sets and assign one of four labels: *ExactMatch* (the model imports exactly the same modules as the accepted answer), *Superset* (all ground-truth imports plus additional ones), *Subset* (only a subset of the ground-truth imports), or *Different* (neither set contains the other). For questions where the accepted answer imports at least one library, we also compute precision and recall of the models import set relative to the accepted answer.

How many StackEval Python tasks actually require libraries? Among the 112 Python questions in our StackEval dataset, only 17 (15%) have accepted answers that import at least one library. The remaining 95 questions are solved without any imports in the ground-truth solution.

How often do models import libraries, even when the ground truth does not? Table 4.6 summarises how frequently models import at least one library in their completions. Across all 112 questions, models import libraries in 30 to 48% of their answers (34–54 questions per model), even though only 17 questions actually use imports in the accepted answer. Focusing on the 95 questions whose accepted answer has no imports, model still introduce at least one library in 28–42% of those questions (27–40 questions per model). In other words, all models sometimes over-import libraries on tasks that the ground-truth solution solves

without any dependencies, with Qwen2.5 and WizardLM being the biggest "over-importer" in our experiment. Thus, LLMs can propose libraries even when they are not strictly required by the task.

How closely do models match the ground-truth libraries when libraries are required? Table 4.7 focuses on the 17 questions whose accepted answers import at least one library and compares each model's library set to the ground-truth set. DeepSeek V3 and WizardLM-2 exactly reproduce the ground-truth import set in 52.9% of these questions, while GPT-4o and Qwen2.5 do so in 47.1% of cases. For these four models, the remaining outcomes are split between omitting some ground-truth libraries (*Subset*: 29.4% for DeepSeek V3 and 23.5% for GPT-4o, Qwen2.5, and WizardLM-2), adding extras on top of the ground truth (*Superset*: 5.9% for DeepSeek V3, 17.6% for GPT-4o and Qwen2.5, and 11.8% for WizardLM-2), and 11.8% of questions falling into the *Different* category. Overall, these models fully recover the ground-truth import set in 58.8% of the library-requiring questions for DeepSeek V3 and in 64.7% for GPT-4o, Qwen2.5, and WizardLM-2 (*ExactMatch* + *Superset*), which suggests that when libraries are required, they often identify the appropriate dependencies, even if they sometimes add extra imports.

In contrast, Llama 3.1 and O1 Preview omit the libraries used in the accepted answer more often, with *Subset* rates of 64.7% and 58.8%, respectively. Their exact-match rates are also lower, at 23.5% for Llama 3.1 and 29.4% for O1 Preview, and they fully recover the ground-truth import set in only 35.3% and 29.4% of cases, respectively. Combining this with the over-import behavior on tasks where the ground truth uses no libraries, our analysis suggests that models differ not only in which libraries they propose, but also in when they decide to introduce any libraries at all. While this analysis does not guarantee functional correctness, it provides a concrete approximation of correctness at the dependency level and clarifies how often models over- or under-approximate the libraries used in ground-truth solutions. At the same time, this comparison is based on a relatively small subset of 17 questions whose accepted answers contain imports. We therefore interpret these results as complementary evidence about dependency-level correctness rather than as a basis for broad generalization on their own.

Overall, these findings refine the interpretation of our results by showing that the key issue is not broad failure, but a more specific form of dependency-related friction. Most StackEval Python questions do not require imports in the accepted answer, and when libraries are required, several models often select reasonable ones. At the same time, the results show that models may still introduce libraries when none are needed in the ground-truth solution. Therefore, the practical concern is less about universal model failure and more about the tendency to introduce external dependencies in cases where a dependency-free solution would be enough,

Table 4.6: Frequency with which models import at least one library in their completions. "Any libs" counts questions (out of 112 StackEval Python questions) where the completion imports at least one library. "Libs when GT has none" counts questions (out of the 95 questions whose accepted answer uses no imports) where the model still imports at least one library.

Model	**Any libs (112 questions)**		**Libs when GT has none (95 questions)**	
	Occurrence	Percentage	Occurrence	Percentage
DeepSeek V3	49	43.8	35	36.8
GPT-4o	44	39.3	31	32.6
Llama 3.1	44	39.3	36	37.9
O1 Preview	34	30.4	27	28.4
Qwen2.5	54	48.2	40	42.1
WizardLM-2	50	44.6	35	36.8

Table 4.7: Comparison of imported libraries in model completions vs. accepted Stack Overflow answers for the subset of StackEval Python questions whose ground-truth solution imports at least one library (17 out of 112 questions). Values show the percentage of questions in each category and how often each model fully recovers the ground-truth import set.

Model	ExactMatch	Superset	Subset	Different	Fully recovered
DeepSeek V3	52.9	5.9	29.4	11.8	58.8
GPT-4o	47.1	17.6	23.5	11.8	64.7
Llama 3.1	23.5	11.8	64.7	0.0	35.3
O1 Preview	29.4	0.0	58.8	11.8	29.4
Qwen2.5	47.1	17.6	23.5	11.8	64.7
WizardLM-2	52.9	11.8	23.5	11.8	64.7

thereby creating avoidable integration and maintenance burden.

4.7.4 Practical Considerations for LLMs in the Context of Dependencies

Our study provides several insights for both practitioners and researchers who integrate LLMs into software development workflows. Importantly, the value of RQ1 and RQ2 is not that models recommend third-party or popular libraries per se, but that they characterize the dependency profile of LLM-generated code in a conversational coding-assistance setting. In particular, these results make visible the trade-offs associated with the dependencies that models introduce, including their installation burden, maintenance implications, licensing properties, and practical usability.

Taken in isolation, the findings that LLMs often recommend third-party libraries and tend to select popular ones are not necessarily surprising or negative. Their practical value lies in what they reveal about the kinds of dependencies models bring into generated code. Recommending third-party libraries more

often than standard-library ones increases the external dependency surface of generated solutions, while recommending popular and mature libraries is generally reassuring but does not guarantee friction-free adoption. These descriptive results therefore provide the context needed to interpret the usability, installation, and dependency-resolution issues discussed later in this section.

Licensing awareness and policy checks. Our analysis in RQ2 shows that most libraries recommended by the models use permissive licenses (91% of license instances, primarily MIT, BSD, and Apache-2.0), which aligns well with typical industrial policies. However, we also found (i) one copyleft-licensed recommendation and (ii) a small but recurring set of imports whose license could not be resolved on PyPI due to aliasing or deprecation (e.g., `sklearn`, `imblearn`). This suggests that organizations should not treat LLM recommendations as automatically policy-compliant. Instead, LLM-integrated tools should surface basic license metadata (e.g., SPDX identifier and category) alongside each suggested library and flag libraries that are missing or ambiguous in package registries. At the project level, teams can maintain an allowlist of acceptable license categories and automatically warn when an LLM suggests a library outside that set.

Resolving imports, not just syntax. RQ3 shows that 4.6% of library references in LLM-generated code do not map directly to an installable package, even though the surrounding code is often plausible. Many of these cases are due to aliases (`sklearn` vs. `scikit-learn`) or module-level imports within larger packages, and only 5 out of 24 alias cases included explicit installation guidance. This means that even when the model knows the right library, the burden of translating the import into a working installation command is still placed on the user. For practitioners, this implies that LLM suggestions should be treated as incomplete until the import has been resolved to a concrete package name and successfully installed. For tool builders, a practical direction is to couple every suggested import with (i) an explicit installation command (e.g., `pip install scikit-learn`) and (ii) an automatic check against the target package registry; if the check fails or returns a deprecated package, the interface should prompt the user to confirm or correct the library choice.

Over-importing and dependency surface. Our usage analysis reveals that LLMs not only introduce third-party libraries more often than the Python standard library, but they also tend to *over-import*. First, models frequently import libraries even when the accepted Stack Overflow answer solves the task without any imports: for the 95 StackEval questions whose ground-truth solution uses no libraries, models still import at least one library in 2842% of completions (Table 4.6). Second, when we examine whether imported libraries are actually referenced, we find that unused imports are more common for third-party libraries (415% of third-party imports, depending on the model) than for standard-library imports (06%). In other words,

when models add unnecessary code, they are more likely to introduce an unnecessary *external* dependency than an unused standard module. For practitioners, this means that accepting LLM code as is can quietly inflate a projects dependency surface with libraries that neither the task nor the ground-truth solution requires. It is therefore important to run basic static checks (e.g., dead-import detection) and to question whether a suggested third-party library is actually needed or whether the task can be solved with existing dependencies or standard-library functionality. For researchers and tool builders, these results point to concrete opportunities for LLM-aware linters and IDE integrations that (i) flag unused or low-value imports and (ii) prefer suggestions that minimize the number and scope of new dependencies (e.g., importing a specific submodule rather than an entire framework).

Security implications of non-installable imports. We did not observe hallucinated package names in our dataset, but we did find non-installable imports such as placeholders (e.g., `some_module`) and alias-based names that resolve to deprecated or empty PyPI entries (e.g., `sklearn`). Prior work has shown that such gaps can be exploited via dependency confusion attacks, where an attacker registers a package under an expected name [69]. In practice, this means that even harmless placeholders in LLM-generated code can become attack surfaces if they are later published or installed without verification. For practitioners, a simple but effective guideline is to treat any package that is missing from the official registry, recently created, or maintained by an unknown publisher as suspicious until vetted. For researchers, our taxonomy of unresolved imports provides concrete categoriesaliases, internal modules, placeholdersthat future security-focused evaluations can target when measuring models susceptibility to hallucination-driven supply-chain attacks.

More generally, our results should be interpreted as identifying friction points in a baseline conversational workflow, not as evidence that such issues are unavoidable in tool-augmented environments. In agentic settings, some of these problems may be mitigated through package-resolution checks, installation support, static analysis, or execution-based feedback. At the same time, those mitigations depend on the surrounding system being designed to detect and resolve dependency-related issues correctly. Taken together, these findings suggest that LLMs are most effective when embedded in tooling that exposes dependency-level metadata and performs basic validation, rather than used as stand-alone code generators.

4.8 Related Work

In this section, we discuss the related literature divided into two aspects. First, we discuss the works that have focused on dependency management and related challenges. Second, we discuss the works that report on the use of large language models as programming assistants.

4.8.1 Dependency Management Challenges

While open source software libraries significantly reduce development time and costs [1, 2, 3], depending on numerous libraries introduces complexity and potential dependency management challenges [17, 120, 12]. One such challenge is highlighted by Mujahid et al. [121] who identify library selection as a critical aspect of dependency management. In their work, they surveyed developers from the npm ecosystem to qualitatively understand the characteristics of highly-selected libraries. Their results show that JavaScript developers believe that such libraries are well-documented, popular, and free of vulnerabilities. Building upon this work, our study leverages those same characteristics to categorize libraries used by various LLMs. Hauge et al. [122] show that organizational library selection is an ad-hoc process that often relies on a combination of past experiences, expert advice, and online resources. This is further discussed in the work of Haenni et al. [123] where the authors surveyed developers about their decision-making when selecting a library to integrate into their application. Their findings show that, in general, developers do not apply a clear rationale when selecting libraries. Alternatively, developers opted for libraries that fulfilled the immediate task requirements. In this context, LLMs are increasingly being adopted to assist in programming tasks, but little is known about the characteristics of the libraries included in LLM-generated code. Our work empirically analyzes the types of libraries recommended by several state-of-the-art LLMs and assesses their popularity, maintenance, and licensing properties.

Dependency hell is a concept discussed in several studies [149, 150, 97] and refers to a situation where a project has an excessive number of dependencies, and managing these dependencies becomes difficult and error-prone. Chen et al. [98] discuss the impact of ”trivial packages,” referring to libraries implementing simple functionalities, on the npm ecosystem. Their survey highlights that developers struggle with the multiple dependencies introduced by these libraries, contributing to dependency hell. For instance, a developer reported the cascading effect of patching a deeply nested dependency, requiring updates throughout the dependency tree. Jafari et al. [11] investigate the relationship between npm library characteristics and the

dependency update strategies adopted by its dependents. The authors report that the release status, the number of dependents, and the age of a library are the most important indicators of the dependency update strategy. Raemaekers et al. [151] discuss the risks associated with the usage of third-party libraries. They identify key library attributes that could serve as risk indicators. Notably, they report that more popular libraries may be updated more frequently, which increases the chance for new bugs to get introduced into the codebase. These findings support our claim that LLMs could better support developers by providing critical information about the libraries they recommend, such as maintenance metrics (i.e., the number of dependencies, version update frequency, project age). Such transparency can help developers anticipate and manage potential maintenance challenges associated with the increased complexity of dependencies.

4.8.2 LLMs in Software Development Workflows

LLMs are increasingly used by software engineering practitioners to perform various tasks, such as code generation, and have shown the potential to improve developer productivity [131, 132, 22]. However, some studies have raised concerns about the reliability of LLM-generated code. For instance, Zhong et al. [152] report common API misuse patterns found in popular LLMs. The study reveals that in the case of GPT-4, 62% of the generated code contained API misuses. The authors argue that this is particularly problematic given that users of LLM code generation are generally not familiar with the APIs that LLMs generate code for, and cannot tell whether the provided code is correct. Similarly, a Vulcan Cyber article claims that ChatGPT hallucinated in almost 40% of the programming questions it was asked [69].

Our findings diverge from those reported in the aforementioned studies. This discrepancy might be attributed to differences in experimental design. First, Zhong et al. employed ”one-shot” or ”few-shot” approaches, providing either irrelevant or relevant examples alongside the prompt. The Vulcan Cyber article describes a scenario where ChatGPT was instructed to generate code for a specific library and then repeatedly prompted for alternatives, potentially leading to hallucinated libraries. Second, we conducted our study on multiple more recent and capable LLMs and aimed to more closely simulate realistic developer-LLM interactions by issuing a single prompt per question. This approach aligns with findings from Jin et al. [153], whose empirical study reveals that developers request code regeneration in only 3% of conversations with ChatGPT.

While LLMs are making significant advancements in software development, concerns regarding reliability and the potential for hallucinations remain. However, studies also highlight the potential of LLMs to improve

developer productivity through functionalities like code completion and search. Ross et al. [131] developed an LLM-based programmer's assistant and evaluated their system on 42 participants. Their results reveal that participants were mostly positive about the assistant's potential for improving their productivity. Heitz et al. [132] evaluate and compare the performance of OpenAI's ChatGPT and Google's Gemini on programming code. They find that while the premium version of the models offers enhanced performance, their free counterparts remain highly relevant for a wide range of users in the context of software development. Also, the authors report that these models can significantly accelerate coding tasks and improve productivity, but necessitate a rigorous review, especially if the generated code is used in critical areas. Despite this, there is a gap in understanding how LLMs perform in the context of dependencies.

Our study directly addresses this gap by investigating the effectiveness of LLMs in recommending software libraries. We use real-world developer questions from Stack Overflow to analyze the libraries suggested by several recent LLMs (as of 2025) in a setting that reflects practical usage scenarios. This approach avoids the potential bias of controlled experimental prompts. By evaluating factors like library popularity and maintenance, licensing, and potential dependency challenges, our work aims to inform the development of more robust LLMs as programming assistants, and provide developers with recommendations and considerations when using such tools to streamline their workflows in the context of dependencies.

4.9 Threats to Validity

Internal validity considers the experimenter's bias and errors. A potential threat to the validity of our study is the manual classification of libraries and import failures. To mitigate this, all cases were reviewed collaboratively by the authors, and discrepancies were resolved through discussion. We did not employ multiple independent coders or inter-rater agreement measures (e.g., Cohen's kappa), as the classification relied on factual criteria (e.g., `cv2` is an alias for `opencv-python`), not subjective judgment. While this may still introduce some bias, we reduced the likelihood through validation by multiple authors.

For our import-usage analysis, we mark an import as used if its bound name appears in a read context in the snippet's AST and as unused otherwise. This approach may miss dynamic uses or usages in surrounding code that is not part of the completion, and it may classify some imports as "used" even if they are not exercised at runtime. We therefore interpret dead-import rates as conservative, approximate indicators of redundancy rather than exact measures of semantic necessity.

Moreover, the stochastic nature of LLMs means they can generate different outputs for the same input. To reduce this variability, we used a low temperature value (0.01) to produce near-deterministic completions. Finally, prompt formatting and parameter settings can affect LLM outputs. We used default parameters where applicable (except for temperature) and used the Stack Overflow questions for prompting as-is to mimic typical usage and ensure consistency across models

Our findings are also conditioned on a minimally engineered, single-turn prompting setup. As a result, they should not be interpreted as measuring the full potential of agentic LLM systems that can call tools, execute code, invoke package managers, or refine outputs based on feedback. Such systems may mitigate some of the dependency-resolution issues observed in our study. Our results should therefore be understood as a baseline characterization of library-related behavior in conversational coding assistance, while richer agentic evaluations remain outside the scope of this work.

External validity concerns the generalizability of our findings. One external threat relates to the time gap between the Stack Overflow questions and the current software ecosystem. Some libraries may have changed since the questions were posted. To control this, we used the latest available metadata (as of September 2025) when analyzing library characteristics.

Another potential threat is data leakage, where questions from Stack Overflow may have been seen during model training. To assess this, we repeated our analysis on StackUnseen, a companion dataset to StackEval, designed to reduce overlap with LLM training data. For each model, we compared the distributions of recommended-library characteristics (e.g., stars, forks, dependents, dependencies, SourceRank, age, and version frequency) between StackEval and StackUnseen using MannWhitney U tests with Holm correction for multiple comparisons. Across all models and metrics, we found no statistically significant differences (all Holm-corrected $p \geq 0.05$), which suggests that potential data leakage does not materially bias our library-level conclusions. At the same time, StackUnseen can only reduce, not eliminate, the possibility that models were exposed to overlapping or near-duplicate content from other sources. Our results on StackUnseen should therefore be interpreted as approximate lower bounds on data contamination rather than definitive evidence that models fully generalize to entirely unseen questions.

Another threat to external validity is that StackEval is not a benchmark explicitly constructed around library-selection tasks. In many Stack Overflow questions, selecting a library is only one possible means of solving the broader problem, and in some cases it is auxiliary rather than central to the task. As a result, our findings should not be interpreted as measuring LLMs general ability to choose the optimal library

in all software engineering settings. Rather, they characterize model behavior in a Stack Overflow-style conversational Q&A setting, where libraries may be introduced implicitly as part of a generated solution. More generally, benchmark-based LLM evaluations are subject to concerns such as contamination and benchmark-specific overfitting, which can weaken generalization claims [136, 135]. Although our use of StackUnseen helps mitigate some of these concerns, it does not eliminate them entirely.

Our findings are also conditioned on a specific Stack Overflow-style prompt format in which the original question title and body are passed to the model without additional meta-instructions or examples. Prior work shows that LLM outputs can be sensitive to small prompt variations, and prompt sensitivity indices such as POSIX [154] have been proposed to quantify this effect. In this study, we deliberately fix the prompt protocol (StackEval title+body) to 1) mimic typical, minimally engineered Q&A usage, 2) avoid introducing prompt-engineering degrees of freedom, and 3) remain comparable to Shah et al.'s StackEval setup [134]. As a consequence, we interpret our results as characterizing model behaviour under this realistic baseline prompt rather than claiming prompt invariance. Systematically exploring how alternative prompts (e.g., explicitly requesting installation instructions) or prompt-sensitivity metrics (e.g., POSIX) affect library imports is an important direction for future work, but falls outside the scope of this work.

More broadly, our study focuses on a single-turn conversational assistance setting without external tools or iterative feedback. As a result, our findings should not be interpreted as measuring the full potential of agentic LLM systems that can execute code, call package managers, consult linters, or refine outputs based on oracles. Such systems may reduce some of the dependency-resolution issues observed in our study. Our results therefore provide a baseline view of library-related behavior in minimally scaffolded interactions, while richer agentic evaluations remain an important avenue for future work.

Moreover, our analysis focuses on code generated in response to stand-alone questions, without access to an existing project codebase. In practice, LLMs are also used within a repository context (e.g., to add a new function or test to an existing file), where many imports may simply mirror or extend the projects current dependency set rather than introduce entirely new libraries. Studying this scenario would require repository-level benchmarks or logs, together with an analysis pipeline that distinguishes between reused imports and genuinely new dependencies at the project level. In this work, we deliberately stay away from project-specific context to characterize how models select libraries when they are unconstrained by an existing import graph. Our findings should therefore be interpreted as a baseline for library-selection behaviour in conversational coding assistants, and complementary studies on in-repository usage are left to future work.

Our dataset contains 112 Python questions from the StackEval benchmark. While each question is answered by six models, yielding 672 completions and 87 distinct imported libraries, the number of underlying tasks is still small compared to the full extent of Python usage on the web. In addition, the post-hoc comparison against accepted Stack Overflow answers is limited to the subset of questions whose ground-truth solution contains at least one import (17 out of 112 questions). As such, these results should be interpreted as indicative rather than definitive, and broader benchmark coverage would be needed to support stronger generalization. Moreover, the distribution of imported libraries is skewed towards popular third-party packages. Thus, our library-level statistics are most representative of how models handle widely used packages and less informative for niche libraries that appear rarely or not at all in our sample. As a result our quantitative results (e.g., fraction of imports from popular libraries or prevalence of installation guidance) should be interpreted as indicative of model behavior on this curated benchmark rather than exhaustive characterizations of the entire Python ecosystem, as results may differ in other ecosystems, languages, models, or when using different prompts.

4.10 Chapter Summary

In this chapter, we assessed how effectively LLMs recommend and use software libraries during code generation. Our findings show that LLMs are generally capable of generating valid and often appropriate library imports across a diverse set of Python programming questions. Across models, the recommended libraries were often well-established, actively maintained, and permissively licensed. We also found that most imports appearing in generated code were actually used, which suggests that library recommendations are usually meaningful rather than arbitrary. At the same time, our results show that models may still introduce external libraries in cases where the ground-truth solution does not require them, pointing to a more subtle form of dependency-related friction in practice.

This friction becomes particularly visible when the recommended import is not straightforward to translate into a working dependency. A non-trivial portion of recommended imports involved cases where the installable package name differed from the import name (e.g., `cv2` for `opencv-python`), or where module-level imports were used without referencing the parent library. In these situations, models often failed to provide installation guidance, leaving developers to manually resolve dependencies. While technically correct, these omissions nonetheless reduce the practical usability of LLM outputs, especially for novice users

or in automated workflows.

From a practical standpoint, developers using LLMs for code generation should remain vigilant about dependency validation and license compliance. While most recommended libraries were permissively licensed, we also observed cases where license information was not straightforward to infer from the model completion alone, as well as one case involving a copyleft license, a more restrictive type of license. Thus, although LLM-generated code may accelerate software development, it can still require post-processing to ensure technical and legal compatibility. Tool builders and IDE designers could help close this gap by integrating dependency resolution checks, license-awareness features, and automatic install-command generation directly into LLM-assisted environments. Such support may become even more important in emerging agentic systems, where models can take a more active role in dependency-related actions within the development workflow.

Future work should explore techniques for making LLM outputs more transparent and execution-aware, such as prompting strategies that elicit installation instructions or LLM pipelines augmented with static or runtime analysis. It would also be valuable to study how these issues manifest in agentic development settings, where models may move beyond suggesting code to taking actions that affect project dependencies. Since our study is limited to Python, it remains an open question whether similar patterns hold across other ecosystems such as JavaScript (npm), Java (Maven), or Rust (Cargo).Ultimately, improving the traceability, trustworthiness, and contextual completeness of LLM-generated code in the context of dependencies is critical for its safe integration into software projects.

Chapter 5

The Software Librarian: Python Package Insights for Copilot

5.1 Chapter Overview

The preceding chapter showed that LLMs can recommend plausible software packages but often provide insufficient information to support their reliable integration. Developers may still need to determine whether a recommended package exists, identify its installable distribution, assess its licensing and maintenance characteristics, and resolve differences between import names and package names.

This chapter presents the Software Librarian, a Visual Studio Code extension developed as an initial practical response to these challenges. The tool analyzes Python packages introduced through AI-generated code completions and provides information about their validity, classification, licensing, dependency relationships, maintenance status, and documentation. It also attempts to distinguish standard and third-party packages from aliases, importable resources, and placeholders.

Within the broader thesis, the Software Librarian illustrates how complementary tooling can mitigate some of the dependency-integration challenges introduced by LLM-generated code. Rather than relying on the model alone to provide complete and current package information, the tool combines LLM-based development assistance with package-specific data retrieved from external sources.

5.2 Introduction

Open source software packages are the cornerstone of modern software development as they significantly reduce development time and effort [2**?**]. However, software packages introduce the concept of dependencies, which are interconnections between code components, increasing complexity and dependency management challenges [17, 120, 12].

Large Language Models (LLMs) emerging as powerful programming assistants [130] that offer potential to simplify and streamline these processes, but they lack real-time critical insights into the packages they import in their code completions, such as license information, deprecation status, or even whether the package still exists [25]. This poses significant risks to software projects, as developers may unknowingly integrate outdated, legally restrictive, or unsupported packages into their codebase. To address these challenges, we introduce the **Software Librarian**, a Visual Studio Code (VS Code) [1] extension that provides developers with real-time analysis of Python packages recommended by GitHub Copilot [2]. Our tool provides insights into the key aspects of packages including license details, dependency data, and deprecation status based on the package's type (i.e., third-party or standard).

To evaluate the Software Librarian, we curated a dataset of third-party and standard packages, as well as packages that do not exist. We focused on the tool's ability to accurately classify these packages into their respective categories- standard, third-party, or invalid. The tool achieved 99% accuracy, which shows its potential to enhance AI-based programming assistants like Copilot. This paper details the tool's design, evaluation, and future improvements to streamline dependency management in the context of LLMs.

5.2.1 Chapter Organization

The remainder of this chapter is organized as follows. Section 5.3 presents the background, terminology, and concepts relative to this work. Section 5.4 presents the implementation details of the tool. Section 5.5 discusses the evaluation of tool Section 5.6 concludes this chapter.

[1] https://code.visualstudio.com/
[2] https://github.com/features/copilot

5.3 Background

In this section, we introduce key concepts that are necessary to understanding the rest of the paper. To make these concepts more intuitive, we will use the ”file directory” analogy, which closely mirrors how packages, modules, and subpackages are organized in Python. We will also define concepts like package aliases, placeholders, and package types.

Package, Module, and Subpackage: In Python, packages and modules are organized like folders and files in a directory:

- **Package**: A folder that contains a collection of modules and/or subpackages. It is defined by the presence of an `__init__.py` file in the directory.
- **Module**: A single Python file within a package, containing functions, classes, and other Python code.
- **Subpackage**: A package contained within another package, similar to a subfolder within a directory.

For the rest of this paper, a subpackage, module, function, or class, will be referred to as an **importable resource**.

Package Alias: In Python, the name used to import a package in code sometimes differs from the package's distribution name (the name under which the package is published and distributed). This difference is known as an **alias**. For example, the package `opencv-python` is imported in Python as `cv2`. This can lead to confusion, as developers might not know immediately that `cv2` refers to the distribution name `opencv-python`.

Placeholder: A placeholder in the context of code refers to a generic or illustrative name used as an example. They are often used to demonstrate concepts without referring to actual packages, acting as temporary stand-ins meant to be replaced. However, it can be confusing if developers mistakenly interpret a placeholder as an actual library or vice versa, especially in the case where a placeholder also happens to be a published package (e.g., `my_module`).

Package Types: Python packages fall into two broad categories: standard and third-party packages.

- **Standard Packages**: These are packages that come pre-installed with Python and are developed and maintained by the Python core team.
- **Third-Party Packages**: These packages are developed and maintained by the broader open-source community and must be installed separately, typically from PyPi.

5.4 A Walkthrough of the Software Librarian

The Software Librarian is a VS Code extension designed to analyze the software packages generated by GitHub Copilot. Specifically, it assists developers in managing package recommendations made by GitHub Copilot during code completions by verifying their correctness and by providing key metadata such as licensing and maintainability metrics. Figure 5.1 shows an overview of the Software Librarian, which has two main components: 1) a frontend integrated into the VS Code editor to view the results of the package analysis and 2) a backend for analyzing the package from Copilot code completions.

5.4.1 Frontend of the Software Librarian

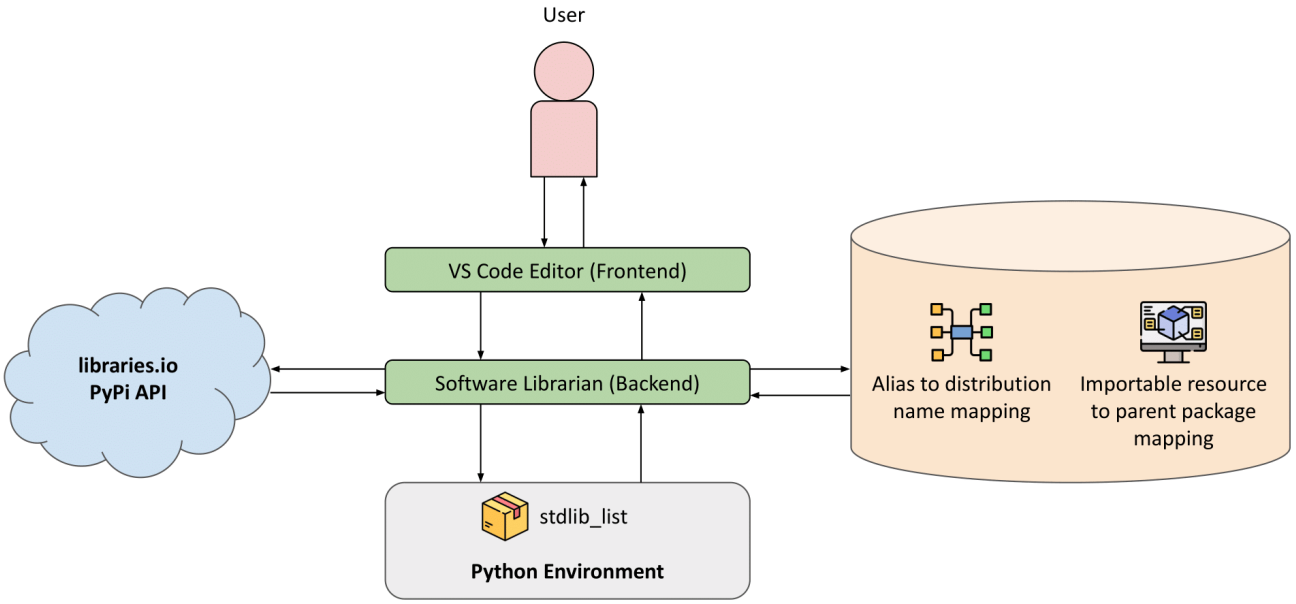
The Software Librarian has a dedicated tab located on the side of the VS Code editor to display the results of a package analysis. Below, we discuss the four main features in the Software Librarian UI.

1. Package Information Display. As shown in Figure 5.2, each analyzed package appears as a folder-like entry in the Software Librarian tab. These entries feature a dropdown menu that allows the user to expand or collapse the detailed information for each package. When expanded, the dropdown shows the collected metadata for the package.

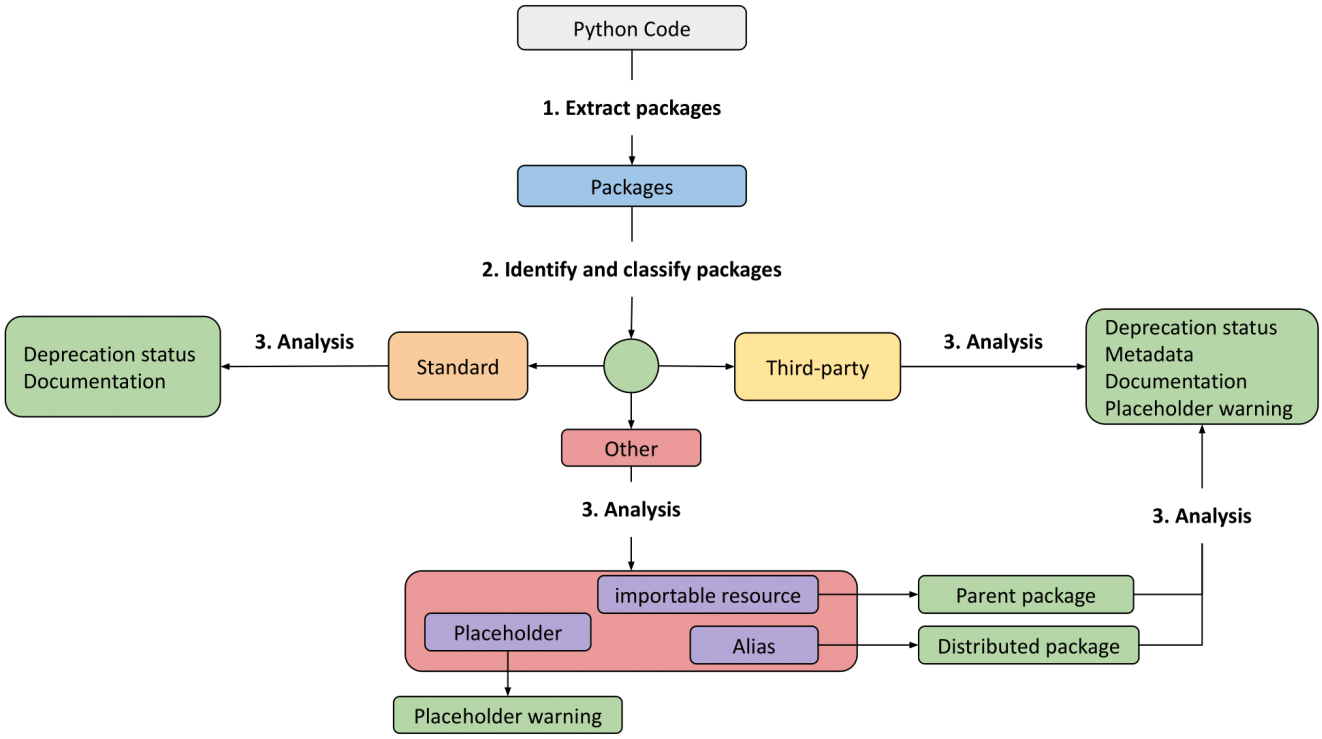
2. Exporting and Managing Package Information. Users can manage package information with several options. For indivudal packages, they can download the analysis results as a JSON file by clicking the ”Download” button located within each package’s dropdown, or remove the results by clicking the ”Clear” button. For bulk actions, users can download results for all analyzed packages at once by pressing the ”Download All” button at the top of the extension tab, or clear all results by selecting the ”Clear All” button.

3. Status Bar Indicators and Notifications. When the extension is actively analyzing a package, a progress indicator appears in the status bar of the VS Code editor. Once the analysis is complete, a completion message pops up to inform the user that the results are ready to be viewed in the extension tab.

4. Manual Trigger for Analysis. In addition to the automatic analysis triggered by Copilot code completions, the user can also manually initiate an analysis using the ”Analyze” button located in the status bar by highlighting any portion of code and press ”Analyze”. This triggers an analysis of the packages imported within the highlighted section.



(a) Frontend and Backend Interaction



(b) Backend Workflow

Figure 5.1: An Overview of the Software Librarian

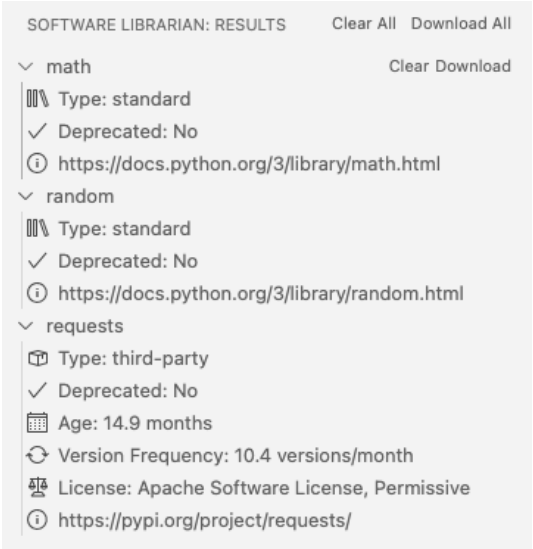


Figure 5.2: An Example of Output of the Software Librarian for Three Packages.

5.4.2 Backend of the Software Librarian

To provide information on the software packages from GitHub Copilot code completions, the Software Librarian performs three sequential steps as illustrated in Figure 5.1: 1) obtaining packages from Copilot code completions, 2) package identification and classification, and 3) package analysis. The Software Librarian's backend component receives an import statement from a Copilot code completion and extracts the imported package name. The backend then interacts with the `stdliblist` [3] package and the PyPi API [4] to determine the type of the package (standard Python or third-party) and consequently, the existence of the package. In case the package is a standard or third-party package, the backend interacts with the Python documentation API for metadata on the standard package and the libraries.io API [5] for the third-party package metadata. In case the package is neither standard nor third-party (classified as *other*), the software librarian performs additional analysis to determine the source of the Copilot suggestion and potential explanation for such a case. The results of the analysis are then forwarded to the UI where they are organized for each individual package. We describe each step below.

Step 1: Obtaining packages from Copilot Code Completions. The first step in the backend is to detect and collect packages that are generated in Copilot Code Completions. To achieve this, the Software Librarian monitors the user's interactions with the code editor and detects lines of code that are 1) auto-completed,

[3] https://pypi.org/project/stdlib-list/
[4] https://pypi.org/
[5] https://libraries.io/api

and 2) contain an `import` statement. When the developer accepts the code completion that includes an `import` statement (triggered by pressing the "Tab" key followed by the "Enter" key to go to the next line), the extension retrieves the `import` statement, extracts the package, and analyzes it.

Step 2: Identifying and Classifying packages. Once an `import` statement is detected, the extension identifies the package and classifies it into one of the types described in Section 5.3: *standard* or *third-party*. To identify *standard* packages, we use the `stdliblist` Python module, which provides a list of all standard packages included in each Python version since 2.6. For packages that are not classified as *standard*, we use the PyPi API to verify whether the package is published in the PyPi repository, in which case it is classified as *third-party*. When a package cannot be classified as either *standard* or *third-party*, it falls under the *other* category, which indicates that the package could potentially lead to installation errors.

Step 3. Package Analysis. The Software Librarian provides different types of information based on the classification of each package into one of the three aforementioned categories (*standard*, *third-party*, or *other*). Below, we discuss how the tool responds for each category.

1. **Third-Party Package Analysis**: For *third-party* packages, the Software Librarian provides different metadata related to the package's maintenance, legal status, and health.

    - **Metadata Retrieval**: The tool retrieves metadata from external sources namely libraries.io API and PyPi. The following metadata is returned for each third-party package:
        - **Number of dependencies**: The number of packages a given package depends on.
        - **Number of dependents**: The number of packages that depend on a given package.
        - **License information**: The type of license associated with a given package.
        - **Version frequency**: How often new versions of a given package are released in a month.
        - **Package age**: How long the package has been available since its first release (in months).
        - **Source rank**: The SourceRank metric of a given package as provided by libraries.io which is an indicator of its general health status.
    - **Deprecation Status**: The deprecation status indicates whether a package is outdated or no longer supported in the current version of Python. For this, the Software Librarian creates a temporary virtual environment, installs, and imports the package. Then, it scans logs during the installation and import processes for any deprecation warnings.

- **Documentation**: The tool provides the link to the official PyPi documentation of the package.

2. **Standard Package Analysis**: For *standard* Python packages, the focus is primarily on checking the deprecation status, as detailed metadata like number of dependencies does not apply in this context.

   - **Deprecation Status**: The Software Librarian determines the deprecation status on *standard* packages by querying the official Python documentation. If a package has been renamed in Python 3 (e.g., `urllib2` to `urllib.request`), it is flagged as deprecated. Additionally, if a package is only found in the Python 2 documentation and not in Python 3, it is also marked as deprecated.
   - **Documentation**: The tool provides the link to the official Python documentation of the package.

3. **Other Package Analysis**: The *other* classification is assigned when the package cannot be identified as either *standard* or *third-party*. These packages fall under one of the following subcategories: aliases, importable resources, and placeholders, which are described in Section 5.3.

   - **Alias Detection**: To detect if a package was imported using an alias, the Software Librarian leverages an internal database to map aliases to their corresponding distribution names. The data for this was collected using the `johnnydep` Python package. Upon identifying an alias, the Software Librarian retrieves its associated distribution package and performs a regular `third-party` analysis. For example, for the alias `cv2`, the Software Librarian would retrieve its distribution `opencv-python` [6] and return its metadata.
   - **Importable Resource Mapping**: The Software Librarian checks whether the package is an importable resource (as described in Section 5.3). To do this, the tool uses an internal database that maps modules, subpackages, and any other item to its parent package. This database is constructed by listing all items defined in the `__init__.py` files of Python packages using the built-in Python `dir()` function. Upon retrieving the parent package, the tool performs a regular *third-party* package analysis. For example, if the tool receives the subpackage `webapp2_extras`, it will retrieve the parent package `webapp2` [7] and return its metadata to the user.
   - **Placeholder Detection**: As discussed in Section 5.3, a placeholder is a generic name that is not meant to represent a real package. To identify placeholders, the Software Librarian uses a

[6] https://pypi.org/project/opencv-python/
[7] https://pypi.org/project/webapp2/

predefined list of common placeholder patterns, curated from Stack Overflow, and augmented with data from prompting large language models such as ChatGPT and Gemini (e.g., my_package, module1). If a suspected placeholder corresponds to a real package (e.g., my_module [8], the extension checks the package's number of downloads. If this number is below 1,000, the extension alerts the developer that, while the package exists, it may have been intended as a placeholder in this context, while still providing the package's metadata.

5.5 Evaluation

In this section, we present the evaluation of the Software Librarian's ability to correctly identify and classify Python packages extracted from Copilot code completions. The goal of this evaluation is to assess the tool's accuracy in distinguishing between types of packages, such as *standard*, *third-party*, and packages that we define as *other* (i.e., aliases, importable resources, and placeholders) when they do not fall into either of the first two categories. We did not evaluate the correctness of the metadata retrieved by the Software Librarian since that information is dynamically fetched from external sources in real-time.

To perform our evaluation, we manually curated a dataset consisting of representative mix of 100 packages commonly encountered in Python development, including edge cases that could lead to installation issues. Specifically, the dataset contains 35 third-party packages, 38 standard packages, 16 placeholders, 9 aliases, and 2 importable resources. Then, we ran the Software Librarian on these input packages and measured its performance in accurately identifying each package type. This also includes flagging incorrectly named packages as placeholders, aliases, or importable resources.

The tool achieved a high level of accuracy, with a total of 99 correct package classifications and only one incorrect classification. The single misclassification occurred with the input `webapp2_extras`, which was incorrectly classified as an alias, while in reality, it is a subpackage of the `webapp2` package. This edge case can be linked to the `johnnydep` package, which we used to map package aliases to their distribution names. In certain cases, `johnnydep` lists importable subpackages or components, which may lead to misclassifications like the one observed.

Overall, the Software Librarian achieved an accuracy rate of 99%, which demonstrates that it can reliably identify and classify Python packages correctly.

[8] https://pypi.org/project/my_module/

5.6 Chapter Summary

In this paper, we introduced the Software Librarian, a Visual Studio Code extension designed to identify and classify Python packages in Copilot code completions. The tools provides developers with information about these packages, including licensing details, number of dependencies, and deprecation status. Our tool bridges the gap between the capabilities of LLMs and the practical needs of developers when integrating LLM-generated code into their codebases. Our evaluation demonstrated that the Software Librarian is highly effective in classifying packages, achieving a 99% accuracy rate in distinguishing between standard and third-party packages, aliases, importable resources, and placeholders. In future work, we plan to further refine the logic for distinguishing between aliases and importable resources. One potential enhancement involves accessing the user's workspace to inspect the contents of installed packages and their internal structure directly.

Chapter 6

Evaluating Large Language Models for Dependency-Maintenance Patch Generation

6.1 Chapter Overview

The preceding chapters examined dependencies during software development: first as components selected and integrated into projects, and then as packages introduced through LLM-generated code. However, dependency management continues after a package has been adopted. Dependencies evolve, new versions are released, vulnerabilities are disclosed, and updates may require changes to project manifests, source code, or tests. Dependency maintenance therefore represents a later and distinct stage of the software lifecycle.

This chapter investigates whether LLMs can support dependency maintenance by generating patches for dependency updates. It examines whether models identify the intended version change, whether they express that change as a patch that applies to the current project state and whether they can produce the related source-code adaptations that may be required. The results show that models frequently understand the requested dependency update, but often fail to construct an applicable and correctly grounded patch. Providing additional source and test context does not consistently resolve these limitations and can reduce manifest-level reliability while producing few applicable source edits.

These findings distinguish maintenance reasoning from maintenance execution. LLMs may assist in interpreting update requirements and proposing related code changes, but they do not reliably manage structured edits, repository state, or final patch construction on their own. The chapter therefore motivates a

staged maintenance workflow in which repository-aware tools perform deterministic operations and validation, while LLMs support targeted reasoning and source adaptation.

Together with the preceding chapters, this study completes the thesis's examination of dependency management across development, production, and maintenance. Across these stages, the findings show that dependency-related challenges arise from limited contextual information, mismatches between declared and actual use, and the difficulty of producing reliable project changes.

6.2 Introduction

Software projects heavily depend on third-party packages to reuse functionality and reduce development effort [1, 2, 3]. These packages become part of the software's implementation and directly affect its security, reliability, and compatibility. A project's behavior therefore depends partly on code maintained outside the project itself. Problems in a dependency, such as a vulnerability, defect, or compatibility issue, can consequently affect every project that relies on it.

Dependencies are also not static. New releases provide security and bug fixes, support for evolving platforms and development tools, and other improvements. Delaying updates can leave known problems unresolved, create incompatibilities with the surrounding software environment, and make later upgrades more difficult as the project falls further behind [14, 12, 11]. Updating is not risk-free either. A new version may introduce breaking changes that cause build failures, test failures, or incorrect runtime behavior and may require changes to the client project's source code [155, 151, 103, 10]. Dependency maintenance is therefore a continuous task that requires projects to balance the risks of remaining on an outdated version against the risks of adopting a new one.

Because dependency updates are frequent and often repetitive, tools increasingly automate parts of this work. Dependency-management bots can monitor package releases and open pull requests that update version specifications. However, generating an update pull request does not complete the maintenance task. The change must still be correctly applied, reviewed, and validated, and some updates require corresponding changes to source files or tests. Prior work on Dependabot shows that dependency-update automation reduces the effort required to identify and initiate updates, but does not remove the work needed to assess and integrate them [68, 63, 67].

Large language models (LLMs) extend software automation beyond fixed update rules. They can interpret

maintenance requests, inspect repository files, generate code changes, and, within agentic workflows, use development tools and respond to execution feedback [156, 157, 158]. This creates an opportunity to use LLMs not only to identify a dependency update, but also to generate the corresponding project changes. However, dependency updating is a precise editing task. The model must modify the correct manifest entry, preserve unrelated content, and produce an edit that applies to the supplied project state. A patch may describe the intended update while still being unusable because its structure, file context, or target path is incorrect.

Existing dependency-specific LLM studies mainly examine how models repair client code after an update has already produced an observable compilation or build failure [74, 73]. These studies show the value of error messages, API differences, and execution feedback for guiding source adaptation, but they do not determine whether an LLM can reliably generate the dependency-update patch itself. Before any compatibility repair can occur, the model must first perform the required manifest change. It is also unclear whether providing related source files helps the model complete this operation or makes the resulting patch less reliable.

We address this gap through an empirical study of LLM-generated patches for real-world JavaScript dependency updates. We first mine 9,908 dependency-update pull requests from 423 repositories and use this corpus to construct a paired evaluation benchmark of 56 pull requests from 46 repositories. Each update is evaluated under two settings. In the configuration-only setting, the model receives the requested dependency update and the base `package.json`. In the configuration-code setting, the model additionally receives related source or test files and is instructed to modify them when necessary. We evaluate four instruction-tuned models using natural and surgical prompting strategies. Across the three research questions, we evaluate the manifest component using patch usability, applicability, semantic correctness, and patch success. We then use the configuration-code outputs in the Discussion to examine the structural applicability of generated source edits and complete multi-file patches.

Our study is guided by the following research questions:

RQ1: How do LLMs perform on real-world dependency-update tasks? When asked to update only the project manifest, models produce a usable manifest patch in 98.9% of eligible outputs. However, only 50.2% produce an applicable manifest patch, 64.6% perform the intended dependency update, and 47.1% satisfy both applicability and semantic correctness. Among applicable patches, 93.8% produce the expected manifest, showing that constructing a patch grounded in the supplied file is a greater limitation than identifying the requested dependency and target version.

RQ2: What kinds of errors or omissions are common in LLM-generated updates? Patch-construction

and grounding problems dominate unsuccessful configuration-only outputs. Inconsistent hunk content, incorrect hunk locations, and malformed diff structure account for 77.1% of failures, whereas wrong edit semantics accounts for 18.2%. The most frequent concrete defects involve context absent from the base manifest, duplicated or impossible old-side lines, missing hunk coordinates, and hallucinated old-side content.

RQ3: How do LLMs perform on manifest updates when source or test context is provided? Under configuration-code prompts, 83.9% of outputs contain a usable manifest patch, but only 37.8% produce an applicable patch and 33.8% achieve patch success. Semantic success reaches 52.2%, indicating that models often identify the intended dependency update without expressing it as a patch that applies to the supplied manifest. Surgical prompting produces moderate numerical improvements, but none of the manifest-level differences is statistically significant after correction. Most unsuccessful outputs contain malformed diff structure, manifest content that does not match the supplied `package.json`, or edits directed toward another provided file.

6.2.1 Our Contributions

This chapter makes the following contributions:

- A paired benchmark of 56 real-world npm dependency updates from 46 JavaScript repositories, instantiated under configuration-only and configuration-code task settings and two prompting strategies that we make available online [1].
- A controlled evaluation of four instruction-tuned LLMs that distinguishes patch usability, applicability to the supplied manifest, semantic correctness, and manifest patch success.
- A taxonomy of recurring LLM dependency-update failures, including malformed diffs, inconsistent or hallucinated hunk content, incorrect hunk locations, wrong target artifacts, and incorrect dependency edits.
- An additional structural analysis of the source edits and complete patches produced under configuration-code prompts. Only 7.5% of outputs produce an applicable in-scope source edit, and 35.4% produce a complete patch that applies across all edited files. Based on these findings, we propose a staged workflow that separates manifest editing, project validation, source adaptation, and final patch construction.

[1] https://zenodo.org/records/21084173

Listing 6.1: Excerpt of a `package.json` manifest.

```
{
    "dependencies": {
    "lodash": "^4.17.20"
    },
    "devDependencies": {
    "eslint": "^8.57.0"
    }
}
```

6.2.2 Chapter Organization

The remainder of this chapter is organized as follows. Section 6.3 presents the key concepts used throughout the paper. Section 6.4 describes our dataset and methodology. Section 6.5, Section 6.6, and Section 6.7 present the findings for our three research questions. Section 6.8 discusses the implications of our findings, presents the additional source-edit and full-patch analyses, and develops our recommendations. Section 6.9 discusses related work. Section 6.10 outlines the threats to the validity of our study. Finally, **??** concludes the paper.

6.3 Background

In this section, we introduce the concepts used throughout our study, including software dependencies and project manifests, unified diff patches and patch application, and the distinction between applicability and semantic correctness.

6.3.1 Dependency Updates and Project Manifests

Software projects commonly rely on external packages, referred to as *dependencies*, to reuse existing functionality. In JavaScript projects that use npm, direct dependencies are declared in a `package.json` manifest. Packages required at runtime are generally listed under `dependencies`, while packages used for development or testing are listed under `devDependencies` [159]. Each declaration maps a package name to a version specification, as illustrated in Listing 6.1.

A dependency update changes the declared specification of a package from its current value to a target value. For example, updating `lodash` in Listing 6.1 to `^4.17.21` requires replacing its existing version specification while preserving the remaining manifest content. Some updates require only this manifest change,

whereas others also require source or test changes to accommodate differences in the updated package [155].

JavaScript projects may also contain a lockfile, such as `package-lock.json`, which records the exact resolved dependency tree to support reproducible installation [160]. In this study, we focus on the requested manifest operation in `package.json`; lockfile regeneration and dependency installation are outside the evaluated generation task.

6.3.2 Diffs, Patches, and Patch Application

A *diff* is a textual representation of the differences between two versions of one or more files. When this representation is used to modify an existing file, it serves as a *patch*. In this study, models are instructed to produce patches in the unified diff format, a common format that groups changes by file and by *hunk* [161].

Listing 6.2 shows a patch that updates the dependency introduced in Listing 6.1. The lines beginning with `---` and `+++` identify the old and new file, respectively. The hunk header beginning with `@@` describes the affected line ranges. Within the hunk, lines beginning with `-` are removed, lines beginning with `+` are added, and unchanged lines provide context for locating the edit.

Applying a patch involves matching its target file paths and old-side hunk content to the corresponding base files, then performing the specified additions and removals [162]. In this study, a patch is considered *applicable* when it can be cleanly applied to the base files supplied in the task and produces valid JSON. A patch may express the intended change but still be inapplicable if, for example, its context does not match the base file, its hunk structure is incomplete, or it targets the wrong file. A multi-file patch is applicable only when each of its file sections can be successfully applied to the corresponding base file.

Listing 6.2: Unified diff patch that updates a dependency in `package.json`.

```
--- a/package.json
+++ b/package.json
@@ -1,5 +1,5 @@
    {
    "dependencies": {
        -"lodash": "^4.17.20"
        +"lodash": "^4.17.21"
        }
    }
```

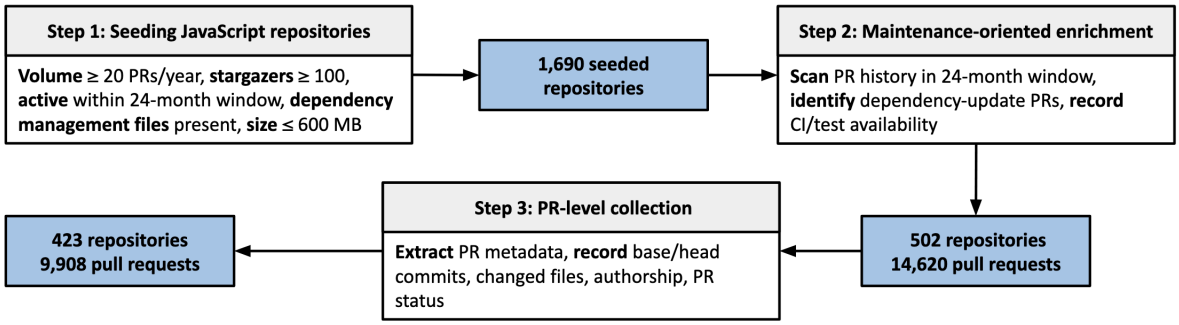


Figure 6.1: An overview of our approach to curating our dataset.

6.4 Study Design

In this section, we describe the steps to curate the dataset used in our study and the experimental setup.

6.4.1 Dataset Construction

Figure 6.1 presents an overview of our approach for curating the dataset used in this study and analyzing the dependency updates generated by LLMs. We constructed our dataset from real-world JavaScript pull requests. To reduce the risk of data leakage, we began by mining candidate repositories using a 24-month lookback window as of January 1, 2026. The purpose of this window was to lower the likelihood that the exact benchmark instances had already appeared in the training data of the evaluated models.

We first seeded candidate repositories using a set of quality and feasibility filters intended to retain active projects with meaningful maintenance activity while excluding repositories unlikely to support dependency-update tasks. Specifically, repositories had to be non-forks, non-archived, contain dependency management files such as `package.json` or a JavaScript lockfile, and exhibit at least 20 pull requests per year. To further avoid inactive repositories and very large monorepos, we also required at least 100 stargazers and a repository size of at most 600 MB. This process yielded 1,690 unique repositories.

We then enriched these repositories with maintenance-oriented metadata by scanning pull requests within the same 24-month window. A pull request was considered a dependency-update candidate if it both appeared to describe a dependency update in its title (e.g., containing terms such as *bump*, *update*, or *upgrade*) and modified `package.json` or a recognized JavaScript lockfile. We also recorded whether repositories exposed CI configuration and evidence of tests, since prior work suggests that dependency maintenance is commonly validated through automated build and test workflows [68, 155]. This stage yielded 502 repositories with at

least one merged dependency-update pull request.

At the pull-request level, we collected the metadata needed to identify and analyze dependency-update tasks. This included repository and pull-request identifiers, base and head commits, changed dependency files, changed source files, extracted upgraded packages and their old and new specifications, authorship information, and complexity indicators such as file count and line churn. This stage produced 14,620 dependency-related pull requests across 502 repositories. We then removed noisy false positives by filtering out pull requests whose extracted dependency metadata was empty despite having been initially classified as dependency updates. After this filtering step, the final corpus contained 9,908 dependency-update pull requests from 423 repositories.

6.4.2 Task Construction and Labelling

From this filtered pull-request corpus, we constructed two benchmark task configurations that differ in scope. In the **configuration-only** setting, the model receives the requested dependency update and the base `package.json` and is asked to modify only the manifest. In the **configuration-code** setting, the model additionally receives related source or test files and is asked to make any code changes needed to accommodate the dependency update. The same dependency-update instances are used in both settings, allowing us to compare narrowly scoped manifest editing with the broader task of generating a maintenance patch. Pull requests were assessed for eligibility using their changed-file profiles and the extracted dependency metadata from the dataset construction stage.

6.4.3 Sampling Strategy

To ensure that benchmark instances corresponded to focused and attributable maintenance problems, we restricted candidate tasks to pull requests that upgraded exactly one dependency. This single-upgrade constraint helps ensure that observed model behavior can be tied to one intended dependency change rather than to a bundle of simultaneous updates.

We also excluded pull requests that were too complex for controlled patch-generation evaluation. A pull request was labeled as too complex if it modified more than 10 files or if its total line churn exceeded 300 lines, measured as additions plus deletions. These thresholds were chosen to exclude large maintenance changes that would be difficult to express as controlled prompting tasks and that would introduce ambiguity

about the minimal expected patch. As a result, the benchmark remains realistic while still focusing on dependency-update instances that are tractable for evaluation.

We then built a shared evaluation sample for configuration-only and configuration-code settings. This allows both settings to be tested on the same underlying PRs, making differences between manifest-only and context-augmented prompting directly comparable. In practice, this meant retaining only PRs that satisfied the manifest-level constraints for configuration-only setting and also contained appropriate source-file context for constructing T1 prompts (i.e., at least one modified source file whose base version explicitly mentions the updated dependency).

To avoid over-representing repositories with unusually high dependency-update activity, we adopted a repo-first sampling strategy with a per-repository cap. Concretely, repositories were sampled uniformly at random from the eligible pool for each task family, and up to two eligible pull requests were selected per repository. Where possible, we also maintained diversity between bot-authored and human-authored pull requests. This strategy improves repository diversity while preventing the benchmark from being dominated by a small number of highly active projects. The final sampled benchmark contains 56 shared configuration-only and configuration-code pull requests from 46 repositories.

6.4.4 Prompt Construction

Each benchmark task was instantiated as a chat-style prompt composed of a fixed system instruction and a task-specific user message. The system instruction is shared across all task variants and frames the model as a software maintenance assistant required to output only a unified diff patch. This choice was made to standardize the output format across models and ensure that completions can be evaluated as candidate code changes rather than free-form textual suggestions.

The user prompt was constructed from the task record produced during task construction. In all cases, the prompt included the intended dependency update in structured textual form, specifying the manifest path, dependency section, dependency name, and old and new dependency specifications. For configuration-only tasks, the prompt additionally included only the base `package.json`. For configuration-code tasks, the prompt included the base `package.json` together with the selected source and test files needed to support local code adaptation.

Prompt construction also varied by prompting strategy. **Natural prompts** describe the maintenance objective directly and allow the model to decide how to carry it out. **Surgical prompts** introduce stronger

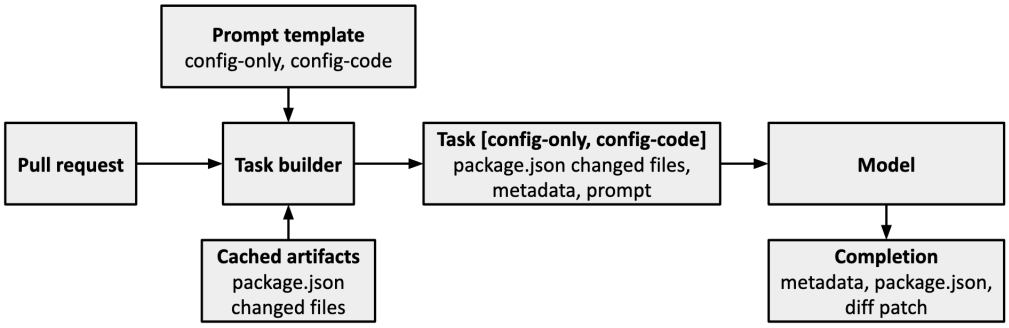


Figure 6.2: An overview of our end-to-end experimental pipeline.

constraints on scope and side effects, such as editing only the specified dependency entries, restricting edits to the provided files, and avoiding unrelated changes or unnecessary formatting modifications. In this way, all prompt variants preserve the same underlying maintenance task while differing in how tightly the expected behavior is specified. The complete system and user prompt templates are provided in the Appendix.

6.4.5 Experiment Setup

Figure 6.2 summarizes the end-to-end experimental pipeline. Starting from a sampled dependency-update pull request, we first materialize the relevant repository artifacts at the base revision, including package.json and, when needed, the changed source or test files. These artifacts are then combined with the structured dependency-update metadata to construct a benchmark task. Each task is instantiated under the natural and surgical prompting strategies and submitted to the target model for inference. The resulting completion is stored together with the prompt, response metadata, and extracted diff artifact.

We evaluated four instruction-tuned models from different contemporary model families: GPT-OSS-120B, DeepSeek-V3.1, Llama 3.3 70B Instruct Turbo, and Gemma-3n-E4B. The purpose of this model set is comparative coverage rather than exhaustiveness. By including models from distinct families under a shared prompting and decoding setup, we can study whether dependency-maintenance behavior is consistent across families or whether it depends strongly on model lineage. Table 6.1 summarizes the key characteristics of the selected models.

All models were executed under the same decoding configuration to reduce variance introduced by stochastic sampling and to make outputs directly comparable. Because the benchmark requires models to generate patch artifacts rather than open-ended text, we favored a near-deterministic setup with low-temperature

Table 6.1: Evaluated models and their main characteristics. All models are instruction-tuned.

Model	Provider	Release Year	Size	Knowledge Cutoff
GPT-OSS-120B	OpenAI	2025	117B total, 5.1B active	June 2024
DeepSeek-V3.1	DeepSeek	2025	685B MoE	January 2025
Llama-3.3-70B-Instruct	Meta	2024	70B	December 2023
Gemma-3n-E4B	Google	2025	E4B effective size	June 2024

decoding (0.01) and a fixed seed. In maintenance tasks, reproducibility and correctness are more important than diversity of completions.

Output completeness. The experimental design yields up to 448 outputs per task setting, corresponding to 56 dependency-update tasks evaluated with four models under two prompting strategies. We successfully obtained 446 configuration-only outputs and 429 configuration-code outputs. The remaining outputs were unavailable because the model either returned an empty response or the inference request failed. These missing outputs occurred for `Gemma-3n-E4B-it`, resulting in 110 configuration-only outputs and 93 configuration-code outputs for that model, compared with 112 outputs for each of the other models. All analyses use the successfully obtained outputs for the corresponding task setting.

6.5 RQ1: How do LLMs perform on real-world dependency-update tasks?

Motivation. Dependency updates are a common software-maintenance task. Projects update libraries to obtain security fixes, bug fixes, new features, and compatibility improvements. Correctly performing an update, however, requires more than identifying the target version. The model must modify the intended manifest entry, preserve unrelated content, and express the change as a patch that applies to the supplied project state. We therefore examine whether LLMs can generate dependency-update patches that are structurally usable, applicable to the base manifest, and semantically correct.

Approach. We evaluate configuration-only outputs using four manifest-level measures. A *usable patch* is a diff-like artifact that targets `package.json`. An *applicable patch* indicates that the generated manifest patch can be cleanly applied to the supplied base file and is in a valid JSON format. *Semantic success* indicates that the output updates the intended dependency to the requested target version without introducing unrelated dependency changes. *Patch success* requires both successful application and semantic correctness.

For patches that apply successfully, we compare the resulting manifest with a synthetic gold target

Table 6.2: Overall manifest-level performance on configuration-only dependency-update tasks ($N = 446$).

Metric	Definition	Result
Usable patch	The output contains a diff-like artifact that targets `package.json`.	441/446 (98.9%)
Applicable patch	The manifest patch applies cleanly to the supplied base file and produces valid JSON.	224/446 (50.2%)
Semantic success	The output updates the intended dependency to the requested target version without introducing unrelated dependency changes.	288/446 (64.6%)
Patch success	The manifest patch is both applicable and semantically correct.	210/446 (47.1%)

constructed from the base `package.json` and the requested dependency update. We use this target rather than the historical pull-request patch because our objective is to determine whether the model performs the required manifest operation, not whether it reproduces the developer's original implementation. When a patch does not apply but still contains a recoverable dependency edit, we evaluate the attempted manifest change directly from the parsed diff.

Results.

Models almost always produce a patch that targets the manifest, but only half of the generated patches apply successfully. Table 6.2 reports the overall results. Of the 446 outputs eligible for manifest-level evaluation, 441 (98.9%) contain a diff-like patch that targets `package.json`. However, only 224 outputs (50.2%) produce a patch that applies cleanly to the supplied base manifest. A larger proportion, 288 outputs (64.6%), express the intended dependency update, while 210 outputs (47.1%) both apply successfully and produce the correct manifest.

These results show that models are more successful at determining the required dependency change than at encoding it in a patch that matches the supplied file. Among the 224 patches that apply successfully, 210 produce the expected manifest, corresponding to an accuracy of 93.8% among applicable patches. Thus, once a generated patch matches the base manifest, it usually performs the correct update. The principal limitation is constructing an applicable patch rather than identifying the dependency and target version.

Performance varies substantially across models. Table 6.3 shows that `GPT-OSS-120B` performs best overall. It expresses the intended dependency update in 108 of 112 outputs (96.4%), and 98 outputs (87.5%) both apply successfully and produce the expected manifest. Its patch applicability and patch success rates are identical, meaning that every applicable patch generated by `GPT-OSS-120B` is also semantically correct.

`DeepSeek-V3.1` performs second best. It produces a semantically correct dependency update in 91 of 112 outputs (81.3%), while 85 outputs (75.9%) are both applicable and correct. Of its 89 applicable patches,

Table 6.3: Manifest-level performance by model on configuration-only dependency-update tasks.

Model	N	Usable patch	Applicable patch	Semantic success	Patch success
`DeepSeek-V3.1`	112	112/112 (100.0%)	89/112 (79.5%)	91/112 (81.3%)	85/112 (75.9%)
`Gemma-3n-E4B-it`	110	105/110 (95.5%)	8/110 (7.3%)	26/110 (23.6%)	2/110 (1.8%)
`Llama-3.3-70B`	112	112/112 (100.0%)	29/112 (25.9%)	63/112 (56.3%)	25/112 (22.3%)
`GPT-OSS-120B`	112	112/112 (100.0%)	98/112 (87.5%)	108/112 (96.4%)	98/112 (87.5%)

Table 6.4: Paired prompt-style comparisons for configuration-only manifest-level outcomes.

Metric	N pairs	Natural	Surgical	Diff.	Nat. only	Surg. only	Holm p
Usable patch	222	220/222 (99.1%)	219/222 (98.6%)	−0.5 pp	1	0	1.0000
Applicable patch	222	110/222 (49.5%)	114/222 (51.4%)	+1.8 pp	15	19	1.0000
Semantic success	222	142/222 (64.0%)	146/222 (65.8%)	+1.8 pp	12	16	1.0000
Patch success	222	102/222 (45.9%)	108/222 (48.6%)	+2.7 pp	13	19	1.0000

85 produce the expected manifest, showing that its applied changes are also usually correct.

The weaker models exhibit a larger separation between identifying the update and constructing the patch. `Llama-3.3-70B` performs the intended dependency update in 63 of 112 outputs (56.3%), but only 29 outputs (25.9%) apply successfully and 25 (22.3%) satisfy both requirements. It therefore often identifies the requested change without expressing it as a patch that matches the base manifest. `Gemma-3n-E4B-it` performs the intended update in only 26 of 110 outputs (23.6%), while eight patches (7.3%) apply successfully and only two outputs (1.8%) are both applicable and correct. Its difficulties therefore extend across both semantic correctness and patch construction.

Overall, the results reveal substantial variation across models. `GPT-OSS-120B` and `DeepSeek-V3.1` translate the requested dependency change into an applicable and correct patch much more reliably than `Llama-3.3-70B` and `Gemma-3n-E4B-it`.

Prompt wording does not significantly affect any evaluated outcome. Table 6.4 compares natural and surgical prompts on the 222 model–task pairs eligible under both conditions. Natural prompting produces a usable manifest patch in 220 cases (99.1%), compared with 219 cases (98.6%) under surgical prompting. Surgical prompting produces slightly higher rates of applicability (51.4% vs. 49.5%), semantic success (65.8% vs. 64.0%), and patch success (48.6% vs. 45.9%). These differences range from -0.5 to $+2.7$ percentage points.

Because the two prompt styles are evaluated on the same model–task instances, we compare each paired binary outcome using an exact two-sided McNemar test [163]. We apply Holm correction across the four evaluated outcomes. None of the differences is statistically significant after correction, with an adjusted

Table 6.5: Manifest-level failure profile by model for configuration-only dependency-update tasks. Counts and percentages are calculated over eligible outputs.

Failure category	Overall	DeepSeek-V3.1	Gemma-3n-E4B-it	Llama-3.3-70B	GPT-OSS-120B
Unusable patch	4/446 (0.9%)	0/112 (0.0%)	4/110 (3.6%)	0/112 (0.0%)	0/112 (0.0%)
Diff-like, no `package.json` target	1/446 (0.2%)	0/112 (0.0%)	1/110 (0.9%)	0/112 (0.0%)	0/112 (0.0%)
Patch application failed	211/446 (47.3%)	21/112 (18.8%)	93/110 (84.5%)	83/112 (74.1%)	14/112 (12.5%)
Patch applied but produced invalid JSON	6/446 (1.3%)	2/112 (1.8%)	4/110 (3.6%)	0/112 (0.0%)	0/112 (0.0%)
Applicable but semantically incorrect	14/446 (3.1%)	4/112 (3.6%)	6/110 (5.5%)	4/112 (3.6%)	0/112 (0.0%)

$p - value$ of 1.0000 for each metric. The additional constraints in the surgical prompt therefore do not reliably improve manifest-level performance. Compared with the large differences observed across models, prompt formulation has little effect on configuration-only dependency-update tasks.

Most unsuccessful outputs fail because their patches cannot be applied to the supplied manifest. Table 6.5 reports the broad failure outcomes across models. These categories are mutually exclusive and together account for the 236 outputs that do not achieve patch success.

Of all 446 eligible outputs, 211 (47.3%) contain a manifest patch that fails to apply cleanly to the base `package.json`. These cases account for 211 of the 236 unsuccessful outputs (89.4%). The remaining failure outcomes are comparatively rare. Four outputs (0.9%) do not contain a usable patch, while one output (0.2%) contains a diff-like artifact that does not target `package.json`. Six outputs (1.3%) apply but produce invalid JSON, and 14 (3.1%) apply successfully but produce a manifest that differs from the expected target.

The model-level failure profiles further explain the performance differences. Failed application affects 93 of 110 eligible `Gemma-3n-E4B-it` outputs (84.5%) and 83 of 112 `Llama-3.3-70B` outputs (74.1%). By comparison, it affects 21 of 112 `DeepSeek-V3.1` outputs (18.8%) and 14 of 112 `GPT-OSS-120B` outputs (12.5%). The dominant failure is therefore not an applicable patch that performs the wrong update, but a generated patch that does not match the manifest it is intended to modify.

Answer to RQ1. When asked to update only the project manifest, models produce a usable manifest patch in 98.9% of eligible outputs, but only 50.2% of patches apply cleanly to the supplied base file. Although 64.6% of outputs express the intended dependency update, only 47.1% both apply successfully and produce the correct manifest. Among applicable patches, 93.8% produce the expected manifest, showing that patch construction is a greater limitation than identifying the requested dependency and target version. Performance differs substantially across models, while prompt wording has no statistically significant effect on any evaluated outcome.

6.6 RQ2: What kinds of errors or omissions are common in LLM-generated updates?

Motivation. RQ1 determines whether a generated dependency update produces an applicable and semantically correct manifest patch. However, these outcomes do not explain why unsuccessful outputs fail. For example, a patch may be inapplicable because its diff structure is malformed, because its context does not match the base manifest, or because it attempts the wrong dependency edit. These failures reflect different limitations in patch construction, file grounding, and task understanding. RQ2 therefore examines the recurring failure patterns among configuration-only outputs that do not achieve patch success.

Approach. We analyze the 236 configuration-only outputs that did not achieve patch success in RQ1. We first use the broad RQ1 failure outcomes, such as the absence of a usable patch, an inapplicable patch, or an incorrect resulting manifest, to identify where each output fails. We then manually inspect representative cases by comparing the generated `patch.diff`, the base `package.json`, and the RQ1 evaluation metadata.

Based on this analysis, we develop a taxonomy of recurring failures involving malformed patch structure, mismatched or hallucinated file context, incorrect dependency edits, and invalid manifest reconstruction. We further identify failure subtypes that describe concrete manifestations of each category. We implement the taxonomy using deterministic heuristics that analyze the generated patch text, parsed hunks, base manifest, and RQ1 evaluation results. Each unsuccessful output is assigned one primary failure category and one corresponding failure substype.

Results.

Patch-grounding and construction problems account for most unsuccessful outputs. Table 6.6 reports the primary failure category assigned to each of the 236 outputs that did not achieve patch success. The most frequent category is *inconsistent hunk content*, which occurs in 79 outputs (33.5%). These patches contain duplicated, impossible, or hallucinated old-side content that does not match the base manifest. Another 59 outputs (25.0%) contain an *incorrect hunk location*, where the intended edit may exist but its surrounding context or claimed position does not align with the supplied file. A further 44 outputs (18.6%) contain a *malformed diff* whose structure prevents it from functioning as an applicable unified diff.

Together, these three patch-construction and grounding categories account for 182 of the 236 unsuccessful outputs (77.1%). By comparison, 43 outputs (18.2%) exhibit *wrong edit semantics*, such as modifying the wrong dependency, selecting the wrong target version, making no effective change, or introducing unrelated

Table 6.6: Failure taxonomy for configuration-only outputs that did not achieve patch success (n = 236).

Failure category	**Count**	**Share**
Inconsistent hunk content	79	33.5%
Incorrect hunk location	59	25.0%
Malformed diff	44	18.6%
Wrong edit semantics	43	18.2%
Invalid manifest serialization	6	2.5%
Non-patch output	4	1.7%
Wrong target artifact	1	0.4%
Total	**236**	**100.0%**

manifest edits. Invalid manifest serialization, non-patch output, and edits directed toward the wrong artifact are comparatively rare.

The two largest categories, *inconsistent hunk content* and *incorrect hunk location*, alone account for 138 failures (58.5%). This result shows that many models identify an update to attempt but express it using old-side patch content that does not correspond to the supplied `package.json`. The dominant problem is therefore not simply determining which dependency should change, but grounding that change in the exact state of the manifest.

Four concrete defects account for more than three-quarters of unsuccessful outputs. Table 6.7 reports the ten most frequent fine-grained failure subtypes. The most common is *context window not in base*, which occurs in 57 outputs (24.2%). In these cases, the generated hunk includes surrounding lines that do not occur together in the supplied manifest. The next most frequent failure subtypes are *duplicate/impossible old lines* (44 outputs; 18.6%), *missing hunk coordinates* (44; 18.6%), and *hallucinated old lines* (34; 14.4%).

Together, these four failure subtypes account for 179 of the 236 unsuccessful outputs (75.8%). They show that many patches contain invented, misplaced, duplicated, or structurally incomplete representations of the file they are intended to modify. A model may specify the correct dependency and target version but still fail to produce an applicable patch because its diff structure or old-side content does not match the supplied manifest.

Listings 6.3, 6.4, and 6.5 illustrate three common failure patterns. In Listing 6.3, the intended dependency update is correct, but the surrounding hunk context does not match the base manifest. In Listing 6.4, the dependency and target version are also correct, but the patch duplicates old-side content that does not exist twice in the file. In Listing 6.5, the output communicates the requested change to a human reader but omits

Table 6.7: Most frequent failure subtypes among configuration-only outputs that did not achieve patch success ($n = 236$).

Failure subtype	**Count**	**Share**
Context window not in base	57	24.2%
Duplicate/impossible old lines	44	18.6%
Missing hunk coordinates	44	18.6%
Hallucinated old lines	34	14.4%
Wrong dependency edit with bad context	17	7.2%
Target plus unrelated edits with bad context	8	3.4%
Applied but not gold manifest	5	2.1%
Manifest fragment instead of patch	4	1.7%
Wrong target version with bad context	4	1.7%
No-effect patch	3	1.3%

the hunk coordinates required for a valid unified diff. These examples show why semantic correctness alone is insufficient: identifying the intended update does not guarantee that the resulting patch can be applied.

Listing 6.3: Example of context window not in base. The target dependency exists in the base manifest, but the surrounding hunk context does not match the supplied file.

```
@@ -252,7 +252,7 @@
"@npmcli/fs": "5.0.0",
"@npmcli/git": "7.0.1",
"@npmcli/map-workspaces": "5.0.1",
-"@npmcli/package-json": "7.0.2",
+"@npmcli/package-json": "7.0.3",
```

Listing 6.4: Example of duplicate/impossible old lines. The patch duplicates old-side content in a way that does not occur in the base manifest.

```
"moment": "^2.30.1",
"mysql2": "^3.16.2",
-"mysql2": "^3.16.2",
+"mysql2": "^3.16.3",
"node-rsa": "^1.1.1",
```

Listing 6.5: Example of missing hunk coordinates. The output is patch-like but omits the structure required for a valid unified diff.

```
--- package.json ---

* uploader: "js-ipfs-fetch": "^5.0.0"

- uploader: "js-ipfs-fetch": "^5.1.0"
```

The taxonomy changes how the RQ1 results should be interpreted. If most unsuccessful outputs changed the wrong dependency or selected the wrong target version, the main limitation would be understanding the requested update. Instead, *wrong edit semantics* accounts for 43 of the 236 failures (18.2%), substantially fewer than the failures involving inconsistent content, incorrect locations, or malformed structure. For configuration-only tasks, the principal bottleneck is therefore expressing the intended change as a patch that matches the exact manifest being edited.

Different models exhibit distinct failure profiles. Table 6.8 reports the failure-category distribution within each model's unsuccessful outputs. `GPT-OSS-120B` has the narrowest profile. Of its 14 unsuccessful outputs, 13 (92.9%) contain malformed diff structure and only one (7.1%) performs the wrong manifest edit. Its relatively few failures therefore concern patch formatting rather than selecting the requested dependency change.

The failures of `DeepSeek-V3.1` and `Llama-3.3-70B` are concentrated around manifest grounding. For `DeepSeek-V3.1`, 11 of 27 unsuccessful outputs (40.7%) contain an incorrect hunk location and 9 (33.3%) contain inconsistent hunk content. Similarly, 39 of the 87 `Llama-3.3-70B` failures (44.8%) contain inconsistent hunk content and 36 (41.4%) contain an incorrect hunk location. These distributions explain why these models, particularly `Llama-3.3-70B`, may express the intended dependency change without producing an applicable patch: the attempted edit does not match the exact old-side content of the base manifest.

`Gemma-3n-E4B-it` has the broadest failure profile. Its 108 unsuccessful outputs span inconsistent hunk content (31; 28.7%), malformed diff structure (31; 28.7%), wrong edit semantics (25; 23.1%), incorrect hunk location (12; 11.1%), and smaller numbers of invalid manifest serialization, non-patch output, and wrong-target failures. Unlike the stronger models, its failures are distributed across patch construction, manifest grounding, and dependency-level correctness.

These results show that differences across models concern not only how frequently they fail, but also which capabilities limit their performance. `GPT-OSS-120B` fails infrequently and almost exclusively

Table 6.8: Failure taxonomy by model for configuration-only outputs that did not achieve patch success. Percentages are calculated within each model's unsuccessful outputs.

Failure category	**DeepSeek**	**Gemma**	**Llama**	**GPT-OSS**
Inconsistent hunk content	9 (33.3%)	31 (28.7%)	39 (44.8%)	0 (0.0%)
Incorrect hunk location	11 (40.7%)	12 (11.1%)	36 (41.4%)	0 (0.0%)
Malformed diff	0 (0.0%)	31 (28.7%)	0 (0.0%)	13 (92.9%)
Wrong edit semantics	5 (18.5%)	25 (23.1%)	12 (13.8%)	1 (7.1%)
Invalid manifest serialization	2 (7.4%)	4 (3.7%)	0 (0.0%)	0 (0.0%)
Non-patch output	0 (0.0%)	4 (3.7%)	0 (0.0%)	0 (0.0%)
Wrong target artifact	0 (0.0%)	1 (0.9%)	0 (0.0%)	0 (0.0%)
Total unsuccessful	**27**	**108**	**87**	**14**

Table 6.9: Prompt-style comparisons of unsuccessful configuration-only outputs. category-specific comparisons are exploratory; the overall McNemar and Bowker tests provide the primary inferential results.

Comparison	**Subset**	***N* pairs**	**Natural**	**Surgical**	***p***
Any unsuccessful output	All paired eligible outputs	222	120 (54.1%)	114 (51.4%)	0.3771
Failure category distribution	Both prompts unsuccessful	101	–	–	0.1977
Failure subtype distribution	Both prompts unsuccessful	101	–	–	0.0805
Inconsistent hunk content	Both prompts unsuccessful	101	32 (31.7%)	39 (38.6%)	0.2649
Incorrect hunk location	Both prompts unsuccessful	101	24 (23.8%)	26 (25.7%)	0.7744
Malformed diff	Both prompts unsuccessful	101	21 (20.8%)	14 (13.9%)	0.1435
Wrong edit semantics	Both prompts unsuccessful	101	18 (17.8%)	18 (17.8%)	1.0000
Invalid manifest serialization	Both prompts unsuccessful	101	4 (4.0%)	1 (1.0%)	0.3750
Non-patch output	Both prompts unsuccessful	101	2 (2.0%)	2 (2.0%)	1.0000
Wrong target artifact	Both prompts unsuccessful	101	0 (0.0%)	1 (1.0%)	1.0000

because of malformed diff structure. `DeepSeek-V3.1` and `Llama-3.3-70B` fail primarily when locating or reconstructing the existing manifest content. By contrast, `Gemma-3n-E4B-it` fails across several parts of the task.

Prompt wording does not significantly change the frequency or distribution of failures. Table 6.9 compares failure outcomes under natural and surgical prompting. Across the 222 paired eligible instances, 120 natural-prompt outputs (54.1%) and 114 surgical-prompt outputs (51.4%) do not achieve patch success. An exact paired McNemar test finds no statistically significant difference in the likelihood of an unsuccessful output ($p = 0.3771$).

Among the 101 model–task pairs that are unsuccessful under both prompt styles, Bowker's symmetry test finds no statistically significant change in the distribution of failure categories ($p = 0.1977$) or fine-grained failure subtypes ($p = 0.0805$). The category-specific comparisons similarly show only small shifts. Surgical prompting produces somewhat more inconsistent hunk-content failures and fewer malformed diffs, but none of the individual exploratory comparisons is statistically significant.

These findings help explain why the more constrained surgical prompt does not significantly improve RQ1 performance. The dominant failures do not arise from refusal, excessive explanation, or a general failure to recognize the requested output format. Instead, they involve exact hunk placement, old-side lines that do not occur in the manifest, hallucinated context, and missing diff structure. Additional instructions can clarify the intended scope, but they do not reliably improve the model's ability to ground a patch in the exact file being edited.

Answer to RQ2. RQ2 shows that unsuccessful configuration-only outputs are driven mainly by patch-grounding and construction problems. Inconsistent hunk content, incorrect hunk locations, and malformed diff structure account for 77.1% of failures, whereas wrong edit semantics accounts for 18.2%. Four recurring defects: context absent from the base manifest, duplicated or impossible old-side lines, missing hunk coordinates, and hallucinated old-side lines, account for 75.8% of unsuccessful outputs. Failures differ substantially across models, but prompt wording does not significantly affect either the likelihood of failure or the overall distribution of failure types. The main challenge is therefore expressing the intended dependency update as an applicable patch grounded in the exact manifest being edited.

6.7 RQ3: How do LLMs perform on manifest updates when source or test context is provided?

Motivation. RQ1 and RQ2 examine configuration-only dependency-update tasks, where the model receives the requested dependency change and the base project manifest. However, dependency updates may also involve related source or test files. In these broader tasks, the model must preserve the required manifest operation while considering whether changes to the additional files are needed. RQ3 therefore examines the manifest component of configuration-code outputs, where models receive related source or test files and are instructed to modify them when necessary.

Approach. We evaluate the manifest component of configuration-code outputs using the same four measures as in RQ1. A *usable patch* is a diff-like artifact that targets `package.json`. An *applicable patch* applies cleanly to the supplied base manifest and produces valid JSON. *Semantic success* indicates that the output updates the intended dependency to the requested target version without introducing unrelated dependency

Table 6.10: Overall manifest-level performance on configuration-code dependency-update tasks.

Scope	*N*	Usable patch	Applicable patch	Semantic success	Patch success
Overall	429	360/429 (83.9%)	162/429 (37.8%)	224/429 (52.2%)	145/429 (33.8%)

changes. *Patch success* requires both applicability and semantic correctness.

For each output, we extract the `package.json` portion of the generated patch and evaluate these four outcomes. We report the overall and model-level results and compare natural and surgical prompting using exact paired McNemar tests with Holm correction. We then apply the RQ2 taxonomy to outputs that do not achieve patch success. We also examine whether prompt style affects the likelihood or distribution of manifest-level failures. Source and test edits are not evaluated in this research question and are examined separately in the Discussion.

Results.

Manifest-level performance

Most configuration-code outputs contain a usable manifest patch, but only one-third achieve patch success. Table 6.10 reports the manifest-level results for the 429 eligible configuration-code outputs. Of these, 360 (83.9%) contain a diff-like patch that targets `package.json`. However, only 162 outputs (37.8%) contain an applicable manifest patch. Semantic success is higher, with 224 outputs (52.2%) performing the intended dependency update without introducing unrelated dependency changes. Overall, 145 outputs (33.8%) satisfy both applicability and semantic correctness.

The difference between semantic success and applicability shows that models often identify the intended dependency update without expressing it as a patch that matches the supplied manifest. Among the 162 applicable patches, 145 (89.5%) are also semantically correct. Thus, when a configuration-code output contains a manifest patch that applies successfully, it usually performs the intended update. The main difficulty remains constructing a patch that is grounded in the exact state of the supplied `package.json`.

Manifest-level performance varies substantially across models. Table 6.11 reports the results by model. `DeepSeek-V3.1` achieves the strongest overall performance. All 112 eligible outputs contain a usable manifest patch. Separately, 89 outputs (79.5%) contain an applicable patch and 89 (79.5%) achieve semantic success. Overall, 78 outputs (69.6%) satisfy both requirements and achieve patch success.

`GPT-OSS-120B` produces a usable manifest patch in 110 of 112 outputs (98.2%) and achieves semantic

Table 6.11: Manifest-level performance by model on configuration-code dependency-update tasks.

Model	N	Usable patch	Applicable patch	Semantic success	Patch success
`DeepSeek-V3.1`	112	112/112 (100.0%)	89/112 (79.5%)	89/112 (79.5%)	78/112 (69.6%)
`Gemma-3n-E4B-it`	93	26/93 (28.0%)	1/93 (1.1%)	6/93 (6.5%)	0/93 (0.0%)
`Llama-3.3-70B`	112	112/112 (100.0%)	31/112 (27.7%)	66/112 (58.9%)	26/112 (23.2%)
`GPT-OSS-120B`	112	110/112 (98.2%)	41/112 (36.6%)	63/112 (56.2%)	41/112 (36.6%)

success in 63 cases (56.2%). However, only 41 patches (36.6%) are applicable. All 41 applicable patches are also semantically correct, indicating that applicability is its main source of manifest-level failure.

`Llama-3.3-70B` produces a usable manifest patch in all 112 eligible outputs and achieves semantic success in 66 cases (58.9%). However, only 31 patches (27.7%) are applicable, and 26 outputs (23.2%) achieve patch success. It therefore frequently identifies the intended update without expressing it as an applicable patch.

`Gemma-3n-E4B-it` performs substantially worse. Only 26 of its 93 eligible outputs (28.0%) contain a usable manifest patch, six (6.5%) achieve semantic success, and one (1.1%) contains an applicable patch. None achieves patch success. Most of its outputs therefore fail to provide a usable `package.json` patch or direct the generated patch toward another file.

Overall, the model-level results reveal different limitations. `DeepSeek-V3.1` most reliably produces a correct and applicable manifest patch. `GPT-OSS-120B` and `Llama-3.3-70B` often identify the intended dependency change but struggle to express it as an applicable patch, while `Gemma-3n-E4B-it` frequently fails to produce a usable manifest edit at all.

Surgical prompting produces numerical improvements, but none is statistically significant. Table 6.12 compares natural and surgical prompting on the 209 model–task pairs eligible under both conditions. Both prompt styles produce a usable manifest patch in 179 cases (85.6%). Surgical prompting produces a higher applicable-patch rate than natural prompting (42.6% vs. 34.9%), as well as higher semantic-success (56.9% vs. 49.8%) and patch-success rates (38.3% vs. 31.1%).

We compare each paired binary outcome using an exact two-sided McNemar test and apply Holm correction across the four metrics. None of the differences is statistically significant after correction. The adjusted p-value is 1.0000 for usable patches and 0.1762 for applicable patches, semantic success, and patch success. The additional constraints in the surgical prompt therefore produce moderate numerical improvements but do not reliably improve the manifest operation.

Table 6.12: Paired prompt-style comparisons for configuration-code manifest-level outcomes. Here, we apply the correction across the four manifest-level outcomes.

Metric	N pairs	Natural	Surgical	Diff.	Nat. only	Surg. only	Holm p
Usable patch	209	179/209 (85.6%)	179/209 (85.6%)	+0.0 pp	6	6	1.0000
Applicable patch	209	73/209 (34.9%)	89/209 (42.6%)	+7.7 pp	20	36	0.1762
Semantic success	209	104/209 (49.8%)	119/209 (56.9%)	+7.2 pp	18	33	0.1762
Patch success	209	65/209 (31.1%)	80/209 (38.3%)	+7.2 pp	19	34	0.1762

Table 6.13: Failure taxonomy for configuration-code outputs that did not achieve manifest-level patch success ($n = 284$).

Failure category	Count	Share
Malformed diff	83	29.2%
Inconsistent hunk content	51	18.0%
Wrong target artifact	45	15.8%
Incorrect hunk location	38	13.4%
Wrong edit semantics	38	13.4%
Non-patch output	24	8.5%
Invalid manifest serialization	5	1.8%
Total	**284**	**100.0%**

Manifest-level failure taxonomy

Most unsuccessful outputs contain malformed, misgrounded, or misdirected manifest patches. We apply the RQ2 taxonomy to the 284 configuration-code outputs that do not achieve manifest-level patch success. Table 6.13 reports the primary failure category assigned to each output.

The most frequent category is *malformed diff*, which occurs in 83 outputs (29.2%). These outputs resemble patches but omit or corrupt structural elements needed for the manifest edit to be applied. Another 51 outputs (18.0%) contain *inconsistent hunk content*, such as duplicated, impossible, or hallucinated old-side lines. In 45 outputs (15.8%), the patch targets another file instead of `package.json`. A further 38 outputs (13.4%) contain an incorrect hunk location, and another 38 (13.4%) perform the wrong dependency edit or introduce unrelated manifest changes.

Malformed diffs, inconsistent hunk content, wrong target artifacts, and incorrect hunk locations together account for 217 of the 284 unsuccessful outputs (76.4%). These failures concern how the manifest patch is constructed, grounded, or directed. By comparison, 38 outputs (13.4%) perform an incorrect or unrelated dependency edit. The main source of failure is therefore not limited to selecting the wrong dependency change. Most unsuccessful outputs fail because the manifest patch is structurally incomplete, does not match

Table 6.14: Most frequent failure subtypes among configuration-code outputs that did not achieve manifest-level patch success ($n = 284$).

Failure label	Count	Share
Missing hunk coordinates	83	29.2%
Patch edits non-`package.json` file	41	14.4%
Context window not in base	38	13.4%
Duplicate/impossible old lines	27	9.5%
Non-diff text output	23	8.1%
Hallucinated old lines	23	8.1%
Applied but not gold manifest	10	3.5%
Wrong dependency edit with bad context	9	3.2%
Wrong target version with bad context	7	2.5%
Target plus unrelated edits with bad context	5	1.8%

the supplied file, or targets another provided artifact.

Missing patch structure and edits directed away from the manifest are the most common concrete failures. Table 6.14 reports the ten most frequent fine-grained failure subtypes. The largest is *missing hunk coordinates*, which occurs in all 83 malformed-diff cases and accounts for 29.2% of unsuccessful outputs. Another 41 outputs (14.4%) edit a non-`package.json` file instead of the manifest.

The remaining common subtypes mainly concern manifest grounding. A context window absent from the base manifest occurs in 38 outputs (13.4%), duplicated or impossible old-side lines occur in 27 (9.5%), and hallucinated old-side lines occur in 23 (8.1%). A further 23 outputs (8.1%) contain non-diff text rather than a usable patch.

These failures characterize the difficulties models encounter when producing the required manifest update under configuration-code prompts. Some outputs omit the `package.json` edit or direct the generated patch toward one of the provided source or test files. Others express the intended manifest change using incomplete diff syntax or old-side content that does not match the base manifest. Unsuccessful outputs are therefore driven mainly by malformed, misdirected, or insufficiently grounded manifest patches.

Prompt wording does not significantly affect the likelihood or distribution of manifest-level failures. Table 6.15 reports the prompt-style failure comparisons. Across the 209 paired eligible instances, an exact McNemar test finds no statistically significant difference in the likelihood of failing to achieve patch success ($p = 0.053$).

We then examine the 110 model–task pairs that are unsuccessful under both prompt styles. Bowker's symmetry test [164] finds no statistically significant difference in the distribution of broad failure categories

Table 6.15: Prompt-style comparisons of manifest-level failures in configuration-code outputs.

Comparison	n	**Test**	p**-value**
Failure presence	209	Exact McNemar	0.053
Failure-category distribution	110	Bowker symmetry	0.279
Failure-label distribution	110	Bowker symmetry	0.182

($p = 0.279$) or fine-grained failure subtypes ($p = 0.182$). Thus, although surgical prompting produces higher numerical manifest-level performance, it does not significantly change either the likelihood of failure or the kinds of errors produced.

Answer to RQ3. RQ3 shows that 83.9% of configuration-code outputs contain a usable manifest patch, but only 37.8% produce an applicable patch and 33.8% achieve patch success. Semantic success reaches 52.2%, indicating that models often identify the intended dependency update without expressing it as a patch that applies to the supplied manifest. Performance varies substantially across models: `DeepSeek-V3.1` achieves the highest patch-success rate, while `Gemma-3n-E4B-it` frequently omits or misdirects the required manifest edit. Surgical prompting produces moderate numerical improvements, but none is statistically significant after correction. Most unsuccessful outputs contain malformed diff structure, manifest content that does not match the supplied `package.json`, or edits directed toward another provided file.

6.8 Discussion

Our findings distinguish between identifying the intended dependency change, expressing the correct manifest operation, and constructing a patch that applies to the supplied project state. Success at one stage does not guarantee success at the others. RQ1 and RQ2 examine the manifest operation under configuration-only prompts, while RQ3 examines the same operation when related source or test files are also provided. In this section, we extend the configuration-code analysis to the generated source edits and complete patches, then discuss the implications for dependency-maintenance workflows.

Table 6.16: Structural source-edit and full-patch performance on configuration-code dependency-update tasks.

Component	Metric	Count / Eligible (n)	Rate
Source edit	Targets at least one provided source file	138 / 429	32.2%
Source edit	Changes at least one provided source file after application	34 / 429	7.9%
Source edit	Applicable in-scope source edit	32 / 429	7.5%
Full patch	Parseable diff artifact	405 / 429	94.4%
Full patch	Edits only files provided in the prompt	396 / 429	92.3%
Full patch	Applies across all edited files	152 / 429	35.4%
Full patch	Applies and produces at least one effective change	148 / 429	34.5%

6.8.1 From Manifest Updating to Maintenance-Patch Generation

RQ1 and RQ3 evaluate the same manifest-update operation under two task settings: configuration-only prompts provide only the manifest, whereas configuration-code prompts also provide related source or test files and ask the model to modify them when needed. To compare these settings fairly, we consider only the 429 model–task–prompt instances for which outputs are available in both settings. On this matched set, semantic success decreases from 66.0% under configuration-only prompting to 52.2% under configuration-code prompting, while manifest patch success decreases from 49.0% to 33.8%. When models receive additional source or test context and are also asked to consider related code changes, their performance on the manifest update decreases.

The manifest-level results do not, however, show whether models use the additional context productively. We therefore examine the source component and the complete configuration-code output. Source edits are evaluated structurally rather than semantically because several adaptations may correctly accommodate the same dependency version, and the historical pull-request patch represents only one possible implementation. Without consistent project-specific compilation, test, or behavioral evidence, successful application cannot establish that a generated source edit is correct. We consequently measure whether source edits target provided files, apply to their base versions, and produce an actual change.

Table 6.16 shows a substantial gap between producing source-file hunks and producing applicable source changes. Although 32.2% of outputs target at least one provided source file, only 7.5% produce an applicable in-scope source edit. Most outputs either contain no source edit or attempt an edit that does not apply to the supplied file. Related source context alone therefore rarely leads to a grounded source adaptation.

The full patch outputs exhibit a similar distinction. Most can be parsed as diff artifacts and remain within

Table 6.17: Model-level applicability of generated source edits and full patches on configuration-code dependency-update tasks.

Model	Eligible (n)	Applicable in-scope source edit	Applicable full patch
DeepSeek-V3.1	112	14/112 (12.5%)	86/112 (76.8%)
GPT-OSS-120B	112	10/112 (8.9%)	32/112 (28.6%)
Llama-3.3-70B	112	7/112 (6.3%)	27/112 (24.1%)
Gemma-3n-E4B-it	93	1/93 (1.1%)	7/93 (7.5%)

the allowed file scope, but only 35.4% apply successfully across all edited files. Structural compliance with the requested output format therefore does not imply that a patch matches the supplied project state. Moreover, only 7.9% of complete patches change a provided source file after application, meaning that most applicable full patches still do not perform source adaptation.

Model-level patterns also differ across task scopes. As shown in Table 6.3, GPT-OSS-120B achieves the highest manifest patch-success rate in the configuration-only setting. In contrast, Table 6.17 shows that DeepSeek-V3.1 achieves the highest rate of applicable in-scope source edits (12.5%) and applicable full patches (76.8%) under configuration-code prompts. Although these structural measures do not establish the semantic correctness of the generated source adaptations, they indicate that performance on manifest updating does not necessarily predict performance on broader maintenance-patch generation.

These findings complement prior work on repairing breaking dependency updates. Reyes et al. [74] and Fruntke and Krinke [73] evaluate source repair after an update has produced an observable compilation or build failure. Their approaches provide evidence such as failing lines, compiler diagnostics, API differences, or execution feedback. Our setting occurs earlier: models receive historically related source files but no concrete failure identifying what must change. The low source-edit success rate suggests that source adaptation should be guided by observable compatibility evidence rather than inferred from related files alone.

6.8.2 Implications for Dependency-Maintenance Workflows

The decline in manifest reliability and the low rate of applicable source edits indicate that dependency maintenance should not be treated as a single generation step. Additional context is useful only when it helps identify a required change. This is consistent with prior repository-level work showing that more context does not necessarily improve repair performance [165] and that multi-file changes benefit from incremental planning [157].

A dependency-maintenance workflow should instead separate four stages:

1. Apply the requested dependency change through a package-manager or manifest-aware operation.

2. Install the updated dependency and execute the repository's available build, test, type-checking, and linting commands.

3. When diagnostics or API-change information identify a compatibility problem, retrieve the affected files and ask the LLM for a localized source transformation.

4. Apply the transformation through a repository-aware editing operation, rerun the checks, and generate the final patch from the modified workspace.

This ordering preserves the required manifest update before introducing source-level reasoning. It also ensures that source context is selected according to an observed failure or identified API change rather than supplied solely because it was modified in the historical pull request. Manifest-only updates may terminate after installation and validation, while source adaptation is introduced only when the updated project provides evidence that it is needed.

The workflow should also separate maintenance reasoning from low-level patch construction. RQ2 shows that many unsuccessful outputs identify the intended update but fail because of malformed diff structure, missing hunk coordinates, or old-side content that does not match the base manifest. Similarly, 94.4% of configuration-code outputs are parseable, but only 35.4% apply across all edited files. Requiring the model to emit a raw unified diff therefore combines reasoning about the required change with reconstruction of exact file context already available to the editing environment.

Instead, an LLM can identify the dependency, target version, affected symbol, call site, or replacement, while manifest-aware and repository-aware tools perform the edits against the current workspace. The final diff can then be generated deterministically from the resulting file changes. This approach prevents patch-grounding failures rather than detecting them only after generation.

We also find that prompt constraints provide only limited support for this process. As shown in Table 6.18, surgical prompting significantly increases full-patch applicability from 31.1% to 41.6% after Holm correction ($p = 0.0373$). However, it does not significantly improve manifest semantic success, manifest patch success, or the production of applicable in-scope source edits. More detailed instructions may therefore help models construct an applicable complete patch, but they do not replace task decomposition, targeted context selection, structured editing, or executable validation.

Table 6.18: Paired prompt-style comparisons for configuration-code outputs ($n = 209$). Holm correction is applied across the six reported outcomes.

Outcome	Natural	Surgical	Raw p	Holm p
Manifest semantic success	104/209 (49.8%)	119/209 (56.9%)	0.0489	0.1955
Manifest patch success	65/209 (31.1%)	80/209 (38.3%)	0.0534	0.1955
Applicable full patch	65/209 (31.1%)	87/209 (41.6%)	0.0062	0.0373
In-scope full patch	190/209 (90.9%)	198/209 (94.7%)	0.0386	0.1929
Changes any provided source file	16/209 (7.7%)	18/209 (8.6%)	0.8238	1.0000
Applicable in-scope source edit	15/209 (7.2%)	17/209 (8.1%)	0.8238	1.0000

These conclusions remain relevant to agentic systems. Agents can apply the manifest update, invoke project commands, retrieve files identified by diagnostics, request localized changes, and revise unsuccessful attempts [158]. Such orchestration may recover from failures observed in our one-shot setting, but it does not remove the stages identified here. An agent must still determine when source adaptation is required, select relevant evidence, apply changes against the current repository state, and interpret validation results.

The appropriate scope of automation should therefore depend on the evidence available. Manifest-only updates can be handled through constrained, manifest-aware operations. Diagnosed and localized compatibility failures can support LLM-generated source edits within an iterative validation loop. Broader migrations with weak behavioral validation should remain developer-assisted. This staged approach uses LLMs for targeted maintenance reasoning while leaving file-state management, edit execution, and final patch construction to dedicated tools.

6.9 Related Work

We organize the related literature into three areas. First, we review research on dependency maintenance and its challenges. Second, we examine automation in dependency-update workflows. Third, we discuss the use of LLMs and agents for dependency maintenance.

6.9.1 Dependency Maintenance

Dependency maintenance covers the selection, monitoring, updating, and replacement of third-party packages throughout the evolution of a software project. This work becomes more difficult as package ecosystems grow and projects rely on increasingly large dependency networks. Decan et al. [2] studied the evolution of seven package ecosystems and found that dependencies and transitive dependency relationships

are widespread and continue to increase over time. Mens and Decan [10] synthesize this broader research into a catalogue of dependency-related challenges, including outdated and vulnerable dependencies, breaking changes, unsuitable versioning policies, deprecated or abandoned packages, and incompatible licenses. Their overview shows that dependency updating is one part of a broader and continuous maintenance problem. Our study focuses on a specific step within this process: producing the project changes required to carry out a selected dependency update.

Prior empirical work shows that projects frequently remain on outdated dependency versions. Kula et al. [14] found that many projects do not adopt available library releases promptly and that developers do not always respond to security advisories affecting their dependencies. Wang et al. [12] similarly observed widespread use of outdated third-party libraries in Java projects, including versions associated with security and compatibility risks. Delayed updates may leave known defects or vulnerabilities unresolved and can increase the gap that must be crossed during a later upgrade.

Dependency-update decisions are also shaped by practical project concerns. Pashchenko et al. [103] interviewed software practitioners to examine how functional and security considerations affect the selection, management, and updating of dependencies. Their study complements ecosystem-level measurements by showing that dependency management is not driven by security information alone. Developers must also consider compatibility, functionality, maintenance effort, and the information available about a package. These factors help explain why the availability of a newer or safer version does not necessarily lead to its immediate adoption.

Projects also use different strategies to control when dependency updates are accepted. Jafari et al. [11] studied dependency-update strategies in the npm ecosystem and found that package characteristics and release properties are associated with how projects constrain and update their dependencies. Their work examines how maintainers manage exposure to future versions through dependency specifications.

Adopting an update can itself introduce maintenance work. New dependency versions may modify APIs, remove functionality, change configuration requirements, or otherwise break existing client code. Reyes et al. [155] introduced BUMP, a benchmark of reproducible breaking dependency updates in which a dependency version change causes a previously successful Java project to fail during compilation or testing. Their work demonstrates that a small manifest change may lead to a broader task involving failure diagnosis, source adaptation, and validation. Our study examines an earlier and complementary stage of this process: whether LLMs can first produce a correct and applicable dependency-update patch, and whether providing related

source and test files helps or hinders that operation.

6.9.2 Automation in the Dependency-Update Workflow

Automation can support several stages of the dependency-update workflow. Package managers resolve and install declared versions, while dependency-management bots such as Dependabot, Renovate, and Greenkeeper monitor package releases or vulnerability reports and open pull requests for available updates. These pull requests commonly modify project manifests and lockfiles and trigger the repository's continuous integration checks. Maintainers must then review the proposed update, interpret the available validation results, and decide whether the change should be merged. Dependency bots therefore automate the discovery and initiation of updates, but do not necessarily complete the broader maintenance task.

Several studies have examined how dependency bots are used in practice. He et al. [68] studied Dependabot version updates through repository mining and a developer survey. They found that projects reduced their technical lag after adopting Dependabot and that developers were generally receptive to its pull requests. However, the study also identified limitations. Compatibility scores were available for only a small proportion of dependency-version pairs, and projects often adjusted update schedules, pull-request limits, and ignored dependencies to reduce notification volume. Cogo and Hassan [166] similarly examined how projects customize Dependabot and showed that configuration requires trade-offs between receiving timely updates and limiting disruption to developers. These studies focus on the adoption, configuration, and perceived usefulness of dependency bots. Our study instead examines the update artifact itself and determines whether a generated patch can be applied to the supplied project state.

Security updates are a major use case for dependency automation. Alfadel et al. [67] studied Dependabot security pull requests in JavaScript projects and found that many were merged shortly after being opened, showing that automated pull requests can help projects respond quickly to known vulnerabilities. Mohayeji et al. [63] further examined how projects resolve vulnerable dependencies using Dependabot and investigated how project characteristics, testing, and continuous integration relate to the handling of security updates. Rebatchi et al. [167] conducted a larger study across multiple ecosystems and similarly found that developers were responsive to security pull requests, particularly when automation and auto-merge were available. At the same time, some pull requests remained unresolved, failed project checks, or were automatically closed after being replaced by a newer update. These studies show that automated pull-request creation can shorten the time needed to identify and initiate a security update, but successful integration still depends on

project-specific validation and review.

Dependency bots can also introduce maintenance overhead. Rombaut et al. [168] analyzed issues created by Greenkeeper in npm projects and found that bot-generated reports represented a substantial part of the issue activity in projects that adopted it. Some reports were false alarms caused by failures unrelated to the dependency update, and automated attempts to resolve failures by pinning an earlier version were frequently unsuccessful. Developers therefore had to inspect the reports, determine whether the dependency caused the failure, and choose an appropriate response. This work shows that automating update detection can shift effort toward notification handling, failure diagnosis, and integration. Our study complements this perspective by examining an earlier source of overhead: generated patches that describe an update but cannot be replayed because their structure or file context is incorrect.

Automated validation is intended to reduce the risk of accepting an incompatible update, but its effectiveness depends on the available project checks. Hejderup and Gousios [169] examined whether test suites provide sufficient evidence for automatically accepting Java dependency updates. They found that tests exercise only part of the code that interacts with direct and transitive dependencies and can miss faults introduced by an update. Their results support combining tests with static or dynamic analysis rather than treating a passing test suite as complete evidence of compatibility. This limitation is particularly important for updates that change runtime behavior without causing an immediate compilation or test failure.

Other approaches automate the selection of dependency versions rather than only opening pull requests for the newest release. Dann et al. [170] proposed UpCy, which uses dependency-graph information and API incompatibility analysis to identify Maven updates that introduce fewer compatibility problems. Jaime et al. [171] model dependency updating as a multi-objective optimization problem that balances technical lag, compatibility, and update cost across the dependency graph. These approaches improve the decision of which versions or combinations of dependencies should be adopted. Our study begins after that decision has been made: the dependency, current specification, and target specification are already known. We evaluate whether LLMs can translate that selected update into a correct and applicable project patch.

Overall, existing automation supports release monitoring, vulnerability detection, version selection, pull-request creation, and project validation. However, these stages do not guarantee that an update will produce an applicable change or that any required source adaptation will be completed. This gap motivates the use of LLMs and agentic systems to support broader dependency-maintenance tasks.

6.9.3 LLMs and Agents for Dependency Maintenance

LLMs can extend dependency-update automation beyond release monitoring and version replacement. They can inspect source files, interpret API changes, and propose code adaptations when a new dependency version is incompatible with the client project. Most dependency-specific studies have therefore focused on repairing client code after a breaking update rather than on generating the dependency-update patch itself.

Reyes et al. [74] proposed BYAM, an LLM-based approach for repairing breaking dependency updates in Java projects. Their approach uses information from the build process, including compilation errors and failing lines, together with API differences between dependency versions. The best-performing configuration repaired 27% of complete builds and 78% of individual compilation errors. In these tasks, the dependency update has already been applied and has produced an observable failure. Our study examines an earlier stage of the workflow: whether an LLM can first generate a correct and applicable patch for the requested dependency update.

Fruntke and Krinke [73] similarly examined whether LLMs can repair Java projects after dependency updates introduce breaking changes. They compared single-step generation with an iterative approach that rebuilds the project and provides the resulting failures to the model. Their findings show the value of execution feedback for guiding later repair attempts. Our study complements this work by evaluating models before such feedback is available. In our maintenance-context setting, models receive source and test files associated with the update but no compiler or test failure identifying the required adaptation. This distinction allows us to examine whether source context alone helps models perform the update.

Recent work has also applied agents directly to dependency upgrades. Tawosi et al. [172] proposed a multi-agent framework that uses migration documentation to locate affected Java code and apply the changes required by a major dependency upgrade. The framework separates summarization, control, and code-modification responsibilities across agents and was evaluated using synthetic repositories based on an industrial use case. This work shows how agents can divide a dependency migration into specialized steps.

Xia et al. [173] proposed BDUpdater, an agent-based approach for breaking dependency updates in JavaScript projects. BDUpdater mines breaking-change information from upstream repositories, refines it into structured update knowledge, identifies affected client code, and provides the resulting information to an LLM that generates source adaptations. Their evaluation shows that fine-grained breaking-change records substantially improve client-code migration. This approach targets major updates with identified breaking

changes and supplies external knowledge about how client code should be adapted.

Broader repository-level systems provide further evidence that multi-file changes benefit from planning and tool support. CodePlan treats repository-level coding, including package migration, as a sequence of interdependent edits [157]. It uses dependency analysis to select edit locations and updates its plan as earlier changes affect other parts of the repository. RepairAgent follows a related approach for program repair by allowing an LLM to gather information, invoke development tools, generate changes, and validate its attempts [158]. These systems evaluate complete workflows in which orchestration can recover from unsuccessful model outputs. Our controlled setting instead isolates the generated patch and exposes failures that an agent would need to detect or prevent, including malformed diffs, incorrect file targets, and patch content that does not match the current repository state.

The selection of context is also important. Parasaram et al. [165] studied which bug-related facts should be provided to LLMs for program repair and found that adding more information does not consistently improve performance. Different tasks benefit from different combinations of source context, error messages, execution values, and issue information. Our paired comparison provides similar evidence in the context of dependency maintenance: adding source and test files from the historical update does not improve the core manifest operation and instead reduces both semantic and patch success. These findings suggest that maintenance context should be selected according to an observed failure or identified change rather than supplied solely because it is available.

Overall, prior work shows that LLMs can help repair dependency-induced failures when they receive compiler errors, API differences, migration documentation, or execution feedback. Agentic systems can further support this process by retrieving information, planning edits, applying changes, and repeating project checks. However, these systems still rely on LLMs to interpret the task, select changes, and generate or describe the required edits. Agentic orchestration may detect and recover from some model failures, but it does not remove the need to understand the capabilities and limitations of the underlying model. Evaluating LLMs directly therefore helps isolate the failure modes that agents must prevent, validate, or correct. Less attention has been given to the initial dependency update itself: whether a model can modify the correct manifest entry and produce a patch that applies to the supplied project state. Our study addresses this gap and examines how the reliability of this operation changes when the task expands from manifest editing to broader maintenance patch generation.

6.10 Threats to Validity

This section discusses the main threats to the internal and external validity of our study and the steps taken to reduce them.

6.10.1 Internal Validity

Internal validity considers potential experimenter bias and errors.

Data leakage. The repositories and pull requests used in this study are publicly available on GitHub and may have been included in the training data of the evaluated models. We mined pull requests from a recent 24-month window as of January 1, 2026, covering the period from January 1, 2024, to December 31, 2025. This window was selected to reduce the likelihood that the exact benchmark instances had appeared in model training data compared with older repository history. However, the window overlaps with the documented knowledge cutoffs of some evaluated models, which range from December 2023 to January 2025. We therefore cannot rule out that some repositories, pull requests, package releases, or similar dependency updates were observed during training. Knowledge cutoffs also do not provide complete information about the composition of a model's training data. To further reduce the risk, we provide only the base project files and the requested dependency change. We do not provide the historical patch, head revision, pull-request discussion, or developer solution. These measures reduce the likelihood of direct solution reproduction, but they cannot fully eliminate data leakage.

Identification of dependency-update pull requests. Our mining process identifies candidate dependency updates using pull-request titles, changed dependency files, and extracted version changes. This process may miss updates whose titles do not contain the expected terms. It may also include pull requests in which the dependency change is secondary to another maintenance task. Unusual manifest structures, file renames, or complex histories may also affect extraction. We reduce this risk by requiring a concrete modification to `package.json` or a recognized JavaScript lockfile and by removing pull requests for which no dependency update could be extracted. We also retain only tasks involving one identifiable dependency update, which makes the intended operation easier to attribute.

Construction of configuration-only and configuration-code tasks. Both task settings are constructed from the same underlying dependency-update pull requests. In the configuration-only setting, the model receives the requested dependency change and the base manifest and is instructed to modify only the manifest.

In the configuration-code setting, the model additionally receives source or test files changed in the historical pull request and is instructed to modify them when necessary. However, the presence of a source change in the historical pull request does not establish that the change was caused by the dependency update or required to complete it. We reduce this ambiguity by retaining only source or test files whose base version explicitly mentions the updated dependency. This requirement increases the likelihood that the supplied files are relevant, but it does not establish a causal relationship or prove that source adaptation is necessary. Our comparison therefore measures the effect of expanding the task with historically related source context and broader maintenance instructions, rather than the effect of providing a known required source adaptation.

Selection of maintenance context. The source and test files used in the configuration-code setting are selected from files changed in the same historical pull request as the dependency update. We assume that these files provide relevant maintenance context, but we cannot verify that their changes were caused by the dependency update or were necessary to complete it. A file may have been modified for an unrelated reason within the same pull request. We reduce this risk by retaining only files whose base version explicitly mentions the updated dependency and by providing only their base contents, without exposing the historical source edits. Because the relationship between the provided context and the dependency update cannot be established with certainty, we do not evaluate generated source edits against the historical implementation or claim semantic correctness. Instead, we evaluate whether source edits are structurally grounded in the supplied files, apply successfully, and produce an actual change.

Construction of the manifest gold target. Manifest-level semantic correctness is evaluated against a synthetic target created by applying the requested dependency change to the base `package.json`. This assumes that the expected operation is to change the specified dependency while preserving unrelated manifest content. The historical pull request may contain other valid changes, such as formatting updates, metadata changes, or lockfile modifications, that are not represented in this target. We use a synthetic target because the requested dependency and target specification define a precise manifest operation, while the historical patch represents only one possible implementation. We also report applicability separately from semantic correctness so that an applicable but incorrect patch is distinguished from a correct update expressed in a non-applicable patch.

Failure taxonomy. The failure taxonomy was developed through manual inspection of unsuccessful outputs and then implemented using deterministic heuristics. The initial analysis may reflect researcher judgment, and some failures could reasonably be assigned to more than one category. Selecting one primary

category may also hide secondary problems in the same output. We reduce this subjectivity by defining the categories using observable properties of the generated patch, applying the same rules to all outputs, and reporting more specific failure subtypes. We also provide representative examples to clarify the distinctions between categories.

Prompt design. LLM performance can be sensitive to prompt wording. Our natural and surgical prompts represent only two possible formulations, and alternative prompts may produce different results. We reduce this threat by keeping the requested dependency update, supplied files, system instruction, and output format constant across prompt styles. Both prompts are evaluated on paired instances, and their differences are tested statistically. Our results should nevertheless not be interpreted as the maximum performance obtainable through prompt engineering.

Inference variability. LLM outputs may vary across repeated executions. Hosted inference services may also introduce nondeterminism even when the same seed and decoding parameters are used. We reduce this variation by using a temperature of 0.01, a fixed seed, and the same decoding configuration across models. This setup reflects a maintenance setting in which reproducibility is preferred over response diversity. However, we generate only one completion for each model, task, and prompt condition. The study therefore does not measure variation across repeated generations or the possible benefit of generating and selecting among multiple candidates.

Hosted model implementations. The behavior of a hosted model may change because of provider-side updates, even when the public model name remains the same. Such changes could affect exact reproducibility. We reduce this risk by recording the model identifiers, prompts, generated outputs, and available response metadata. The results should be interpreted as measurements of the model deployments used during the experiment rather than all past or future deployments of the same model families.

6.10.2 External Validity

External validity concerns the generalizability of our findings.

Programming language and package ecosystem. Our benchmark contains JavaScript repositories using npm-style manifests. Dependency declarations, package-manager behavior, versioning practices, and project structures differ across ecosystems. The results may therefore not generalize directly to Maven, Gradle, PyPI, Cargo, NuGet, or other dependency-management systems. Replication across additional languages and ecosystems is still needed; we leave it to future work.

Repository population. The mined repositories are public, non-forked, non-archived GitHub projects with at least 100 stars, at least 20 pull requests per year, and a maximum size of 600 MB. These filters favor active and visible open-source projects. The results may not represent private repositories, small or inactive projects, very large monorepositories, or projects hosted outside GitHub. We use these filters to obtain repositories with sufficient maintenance history and available project artifacts. We also sample repositories before pull requests and limit the number of selected updates per repository to prevent a small number of highly active projects from dominating the benchmark.

Benchmark size. The final benchmark contains 56 dependency updates from 46 repositories. Although each task is evaluated across four models, two prompts, and two context conditions, the number of distinct maintenance tasks remains limited. Rare dependency-update patterns may therefore be absent. We reduce concentration by using a repository-first sampling procedure, selecting at most two tasks from each repository, and including both bot-authored and human-authored updates. The reported rates should still be interpreted in relation to the size and composition of this benchmark.

Task selection and complexity. We restrict the benchmark to pull requests that update exactly one dependency, modify at most 10 files, and contain at most 300 changed lines. These constraints produce focused tasks for which the intended dependency update can be clearly identified. However, they exclude multi-dependency updates, large migrations, broad refactorings, and complex monorepository changes. Such tasks may be more difficult and may produce failure modes not represented in our results. Our conclusions therefore apply mainly to focused dependency updates rather than all forms of dependency migration.

Evaluation of source adaptations. A dependency update may have several valid source-level adaptations. The historical developer patch represents one solution, but it is not necessarily the only correct solution. An exact comparison against that patch could reject valid alternatives. We therefore evaluate whether generated source edits target the provided files, apply to their base versions, and produce an actual change. This avoids treating the historical implementation as the only acceptable answer, but it also prevents us from determining whether an applicable source change is semantically correct. Future work should combine structural evaluation with compilation, tests, type checking, static analysis, or other project-specific validation.

Model coverage. We evaluate four instruction-tuned models from different model families and sizes. This selection is not exhaustive and does not include every proprietary, open-source, or code-specialized model. Newer models may perform differently, and the relative ranking of the evaluated models may change over time. Our broader conclusions focus on recurring patch-construction and grounding problems rather

than on a fixed model ranking.

Single-step generation. Our experiment evaluates a single model response without tool use, execution feedback, or iterative correction. Coding agents may detect that a patch failed, retrieve additional context, run project commands, and ask the model to revise its output. The reported results should therefore not be interpreted as the performance of complete agentic maintenance systems. The controlled design instead isolates the initial model-generated artifact and identifies failures that an agent or editing environment would need to prevent, detect, or correct.

Temporal validity. Models, package ecosystems, and repository-level development tools evolve quickly. Later models may be better at grounded editing and patch construction, while future coding agents may prevent some of the failures observed in this study. We report the evaluated model versions, release years, and documented knowledge cutoffs to place the results in their temporal context. The benchmark, evaluation measures, and failure taxonomy can be reused to assess whether later systems address these limitations.

6.11 Chapter Summary

In this study, we examined whether LLMs can generate correct and applicable patches for real-world dependency updates. We mined 9,908 dependency-update pull requests from 423 JavaScript repositories and constructed a paired benchmark of 56 updates from 46 repositories. We evaluated four instruction-tuned models under configuration-only tasks, where the model was asked to update the project manifest, and configuration-code tasks, where it additionally received related source or test files and was instructed to modify them when needed.

When asked to update only the manifest, models produced a usable manifest patch in 98.9% of eligible outputs. However, only 50.2% produced an applicable manifest patch, 64.6% performed the intended dependency update, and 47.1% produced a patch that was both applicable and semantically correct. Among applicable patches, 93.8% produced the expected manifest. The principal limitation was therefore not identifying the requested dependency and target version, but constructing a patch grounded in the exact state of the file being edited. The failure analysis supports this conclusion: inconsistent hunk content, incorrect hunk locations, and malformed diff structure accounted for 77.1% of unsuccessful outputs, whereas wrong edit semantics accounted for 18.2%.

Under configuration-code prompts, 83.9% of outputs contained a usable manifest patch, but only 37.8%

produced an applicable patch, 52.2% achieved semantic success, and 33.8% achieved manifest patch success. Restricting the configuration-only comparison to the same 429 eligible outputs, semantic success decreased from 66.0% to 52.2%, while manifest patch success decreased from 49.0% to 33.8%. Thus, providing historically related source or test context did not improve the required manifest operation and instead made it less reliable.

The additional configuration-code analyses show that models also struggle to convert the supplied context into structurally applicable source and multi-file changes. Only 7.5% of outputs produced an applicable in-scope source edit, while 35.4% produced a complete patch that applied across all edited files. These measures assess structural applicability rather than behavioral correctness and therefore do not establish that the generated source adaptations were necessary or semantically valid. Prompt formulation did not significantly affect manifest-level outcomes or source-edit measures, although surgical prompting increased full-patch applicability from 31.1% to 41.6% on paired configuration-code outputs.

Overall, dependency maintenance should not be treated as a single unconstrained generation task. The manifest update should first be performed through a package-manager or manifest-aware operation. The updated project should then be installed and checked, and source adaptation should be attempted only when compiler errors, test failures, API differences, or other evidence indicates that it is required. LLMs can support maintenance reasoning and localized code transformation, while repository-aware tools should manage current file state, apply structured edits, execute validation, and generate the final patch. Future work should evaluate this staged approach across additional package ecosystems and within agentic workflows that use project execution, targeted context retrieval, iterative correction, and behavioral validation.

Chapter 7

Conclusion and Future Work

In this chapter, we summarize the conclusions and findings from each chapter and discuss future directions for research.

7.1 Conclusion

Modern software systems rely extensively on third-party packages, but this reliance makes dependency management a persistent challenge across development, production, and maintenance. Developers must determine which dependencies are relevant to deployed software, evaluate packages introduced during development, and update them reliably as projects and ecosystems evolve. In this thesis, we empirically study these challenges and examine how emerging LLM-based development practices affect dependency-management workflows. We first investigate which dependencies installed in JavaScript projects are ultimately included in production and assess the limitations of conventional dependency classifications. We then study the packages introduced through LLM-generated Python code and develop the Software Librarian to provide additional information supporting their validation and integration. Finally, we evaluate whether LLMs can generate reliable dependency-maintenance patches for existing JavaScript projects. Together, these studies show that dependency management requires context beyond dependency configuration files, import statements, and update requests, and that reliable support should combine LLM capabilities with production-aware, ecosystem-aware, and repository-aware tooling. In the following sections, we discuss the main findings and contributions of each chapter.

Chapter 3: Identifying Dependencies in Production

We conduct an empirical study of 100 JavaScript projects to determine which installed dependencies are ultimately included in production releases. We analyze the projects' direct and transitive dependencies, compare their declared runtime and development classifications with their observed production use from project sourcemaps, and examine the production relevance of dependency-related security alerts. We find that less than 1% of the installed dependencies are included in production. Moreover, 59% of dependencies declared for runtime use are absent from production, while 28.2% of dependencies declared for development are nevertheless included. Most security alerts also target dependencies that do not reach production. These findings show that manifest classifications and installation trees do not reliably represent the operational relevance of dependencies and that production use can provide important context for prioritizing dependency-management activities.

Chapter 4: LLMs Introducing Dependencies

We evaluate six proprietary and open-source LLMs using 112 real-world Python programming questions collected from Stack Overflow. We analyze the libraries imported in the generated solutions, characterize their popularity, maintenance, licensing, and dependency properties, and assess whether the resulting imports can be used without additional intervention. We find that the models predominantly introduce third-party libraries and generally favour mature, popular, and permissively licensed packages. However, 4.6% of the imported libraries cannot be resolved automatically because of mismatches between import names and installable package distributions, and only two of the six models provide installation guidance. These findings show that LLMs often recommend plausible libraries but do not consistently provide the context required to integrate the resulting dependencies reliably.

Chapter 5: Tool Support for LLM Dependency Integration

We develop the Software Librarian, a Visual Studio Code extension that analyzes Python packages introduced through GitHub Copilot code completions. The tool distinguishes standard and third-party packages, maps aliases and importable resources to their corresponding distributions, identifies potential placeholders, and retrieves information related to licensing, dependencies, documentation, deprecation, and package health. We evaluate its package-identification and classification capabilities using a curated dataset

of 100 packages, including standard packages, third-party packages, aliases, importable resources, and placeholders. The Software Librarian correctly classifies 99 of the 100 packages, with the only error involving an importable subpackage classified as an alias. The tool demonstrates how ecosystem-aware analysis can complement LLM-generated code by validating package identities and providing current information that may be absent from the generated response.

Chapter 6: LLMs in Dependency Maintenance

We mine 9,908 dependency-update pull requests from 423 JavaScript repositories and construct a paired benchmark of 56 updates from 46 repositories. Using this benchmark, we evaluate four instruction-tuned LLMs under two task settings and two prompting strategies to determine whether they can generate applicable and semantically correct dependency-update patches. For manifest-only updates, the models produce a usable patch in 98.9% of outputs, but only 50.2% of the patches apply to the supplied project state and 47.1% are both applicable and semantically correct. Among applicable patches, 93.8% perform the expected update, indicating that patch construction and repository grounding are greater limitations than identifying the requested dependency change. When related source or test files are provided, manifest level success decreases to 33.8%, only 7.5% of outputs contain an applicable in-scope source edit, and 35.4% produce a complete patch that applies across all edited files. These results show that LLMs can often reason about the intended update but remain unreliable at constructing and grounding repository-level changes, motivating staged workflows that combine LLM reasoning with structured editing and validation tools.

7.2 Future Work

In this section, we present an overview of the key directions for future work.

7.2.1 Cross-Ecosystem Characterization and Detection of Dependency Bloat

Chapter 3 focuses on JavaScript projects and relies on source maps to identify which installed dependencies are included in production bundles. However, dependency bloat is not unique to the npm ecosystem, and related forms of unnecessary dependency inclusion have also been observed in other ecosystems such as Maven [49, 48]. As highlighted in Chapter 3, source maps are specific to certain web-development workflows and are not generated by many projects, leaving most software ecosystems without an equivalent mechanism

for distinguishing installed dependencies from those that contribute to production software. Future work should characterize dependency bloat beyond npm and investigate ecosystem-independent detection techniques using manifests, lockfiles, build artifacts, binaries, containers, static analysis, and runtime information. Such work would provide a broader understanding of the prevalence and causes of dependency bloat and support tools that help developers identify and prioritize potentially unnecessary dependencies. A logical next step to this is to then evaluate whether this production-aware information improves real maintenance decisions. For example, future studies could measure whether it reduces alert volume, helps developers identify high-priority updates, or changes how teams respond to vulnerable dependencies. Such work should also account for dynamic loading, environment-specific builds, and dependencies that are present in production but reachable only under particular conditions.

7.2.2 Studying LLM-Introduced Dependencies in Real Development Workflows

The evaluation of LLM-generated imports in Chapter 4 was conducted using controlled prompts derived from Stack Overflow problems. Stack Overflow provides a reproducible source of real-world programming tasks and enables consistent comparisons across models, but these prompts represent isolated problem-solving scenarios rather than complete development workflows. In practice, developers increasingly interact with LLM-based assistants directly through IDEs, code completions, and iterative conversations, where recommendations may be influenced by repository context, previously generated code, and subsequent user feedback. Future work should therefore study how developers encounter and adopt LLM-recommended dependencies in real projects. Repository mining, IDE instrumentation, and developer studies could be used to examine which recommendations are accepted, how often they require correction, and whether they introduce packages that would not otherwise have been selected. Longitudinal studies should also investigate what happens after adoption, including whether the recommended package remains in the project, creates compatibility or licensing concerns, is maintained over time, or is eventually replaced. Extending the evaluation to additional programming languages and package ecosystems would further help determine which integration challenges are specific to Python and which reflect broader characteristics of LLM-assisted development.

7.2.3 Extending Ecosystem-Aware Support for Package Integration

The Software Librarian presented in Chapter 5 provides an initial form of package validation and metadata retrieval, but its analysis can be expanded. Future versions should incorporate the project's current environment, lockfiles, interpreter version, installed packages, and organizational policies. This would allow the tool to evaluate not only whether a package exists, but also whether it is compatible with the specific project and permitted under its licensing or security requirements such as internal package repositories. Package provenance, release activity, vulnerability information, dependency health, and maintenance status should be combined into explanations that help developers interpret the retrieved information rather than presenting isolated metrics. A user study would be necessary to determine which signals developers find useful, whether the tool reduces integration effort, and whether it improves their ability to detect unsuitable recommendations.

7.2.4 Human Evaluation of Dependency-Management Support

Ultimately, the value of dependency-management support depends on how it affects developers' work. Future research should evaluate whether production information, package-validation tools, and LLM-assisted maintenance reduce review effort and improve decision quality. Relevant measures include time required to assess a dependency, number of corrections made to generated changes, confidence in accepting an update, and the ability to recognize incomplete or unsafe recommendations. Particular attention should be given to trust. A system that produces mostly plausible outputs may encourage developers to accept incorrect recommendations without sufficient review, while a system that generates excessive warnings may be ignored. Future tools should therefore communicate uncertainty, identify the source of package information, and clearly distinguish model-generated reasoning from validated ecosystem or repository facts. Studying these interactions would help ensure that dependency-management automation supports informed decisions rather than simply shifting hidden verification work to developers.

7.2.5 Agentic Dependency-Maintenance Workflows

Chapter 6 evaluates LLMs through prompt-response interactions, which remain a common way for developers to use AI-assisted programming tools. However, software-development workflows are increasingly incorporating agentic systems that can carry out multi-step tasks on the developer's behalf, including inspecting

repositories, writing and modifying code, invoking tools, and running builds or tests. Although these systems are built on LLMs, their ability to act directly on software projects introduces different risks and capabilities from those observed in single-generation settings. Future work should therefore characterize how reliably software-engineering agents introduce, update, and adapt dependencies, including whether they modify the correct artifacts, preserve repository consistency, and recover from unsuccessful changes. It should also examine how agent architecture affects dependency-maintenance performance. For example, agents may differ in how they plan changes, retrieve documentation, allocate responsibilities between the LLM and deterministic tools, validate intermediate outputs, and respond to execution feedback. Comparing these architectures would help determine whether staged workflows in this context improve dependency maintenance and which components are most important for producing reliable repository-level changes.

Bibliography

[1] Victor R. Basili, Lionel C. Briand, and Walclio L. Melo. How reuse influences productivity in object-oriented systems. *Communications of the ACM*, 39:104–116, 10 1996. doi:10.1145/236156.236184.

[2] Alexandre Decan, Tom Mens, and Philippe Grosjean. An empirical comparison of dependency network evolution in seven software packaging ecosystems. *Empirical Software Engineering*, 24(1):381–416, 2019.

[3] Emerson Murphy-Hill, Ciera Jaspan, Caitlin Sadowski, David Shepherd, Michael Phillips, Collin Winter, Andrea Knight, Edward Smith, and Matt Jorde. What predicts software developers' productivity? *IEEE Transactions on Software Engineering*, pages 1–1, 2019. doi:10.1109/tse.2019.2900308. URL https://storage.googleapis.com/pub-tools-public-publication-data/pdf/54eb17429fe4bce386a7db598d656b6a4a1bc749.pdf.

[4] Eight key findings illustrating how to make open source work even better for developers, 2019. URL https://cdn2.hubspot.net/hubfs/4008838/Resources/The-2019-Tidelift-managed-open-source-survey-results.pdf.

[5] Nicole Forsgren, Bas Alberts, Kevin Backhouse, Grey Baker, Greg Cecarelli, Derek Jedamski, Scot Kelly, and Clair Sullivan. 2020 state of the octoverse: Securing the world's software. *arXiv preprint arXiv:2110.10246*, 2021.

[6] Fred Bals. 2024 Open Source Security and Risk Analysis Report, 2 2024. URL https://www.blackduck.com/blog/open-source-trends-ossra-report.html.

[7] Victor Wen and Zedong Peng. Advanced vulnerability scanning for open source software to minimize

false positives. In *2024 IEEE International Conference on Information Reuse and Integration for Data Science (IRI)*, pages 156–157. IEEE, 2024.

[8] npm, Inc. npm: The javascript package registry. https://www.npmjs.com/. Accessed: 2026-07-02.

[9] Python Software Foundation. PyPI: The python package index. https://pypi.org/. Accessed: 2026-07-02.

[10] Tom Mens and Alexandre Decan. An overview and catalogue of dependency challenges in open source software package registries. In *The 23rd Belgium-Netherlands Software Evolution Workshop (BENEVOL 2024)*, 2024.

[11] Abbas Javan Jafari, Diego Elias Costa, Emad Shihab, and Rabe Abdalkareem. Dependency update strategies and package characteristics. *ACM Trans. Softw. Eng. Methodol.*, 32(6), sep 2023. ISSN 1049-331X. doi:10.1145/3603110. URL https://doi.org/10.1145/3603110.

[12] Ying Wang, Bihuan Chen, Kaifeng Huang, Bowen Shi, Congying Xu, Xin Peng, Yijian Wu, and Yang Liu. An empirical study of usages, updates and risks of third-party libraries in java projects. In *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 35–45. IEEE, 2020.

[13] Abbas Javan Jafari, Diego Elias Costa, Rabe Abdalkareem, Emad Shihab, and Nikolaos Tsantalis. Dependency smells in javascript projects. *IEEE Transactions on Software Engineering*, 48(10): 3790–3807, 2021.

[14] Raula Gaikovina Kula, Daniel M German, Ali Ouni, Takashi Ishio, and Katsuro Inoue. Do developers update their library dependencies? an empirical study on the impact of security advisories on library migration. *Empirical Software Engineering*, 23:384–417, 2018.

[15] Mahmoud Alfadel, Diego Elias Costa, and Emad Shihab. Empirical analysis of security vulnerabilities in python packages. *Empirical Software Engineering*, 28(3):59, 2023.

[16] Ahmed Zerouali, Tom Mens, Alexandre Decan, and Coen De Roover. On the impact of security vulnerabilities in the npm and rubygems dependency networks. *Empirical Software Engineering*, 27 (5):107, 2022.

[17] Jasmine Latendresse, Suhaib Mujahid, Diego Elias Costa, and Emad Shihab. Not All Dependencies are Equal: An Empirical Study on Production Dependencies in NPM. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE'22)*, 2022.

[18] Ipek Ozkaya. A paradigm shift in automating software engineering tasks: Bots. *IEEE Software*, 39(5): 4–8, 2022.

[19] GitHub, Inc. About Dependabot version updates. https://docs.github.com/en/code-security/concepts/supply-chain-security/about-dependabot-version-updates. Accessed: 2026-07-02.

[20] Mend.io. Renovate documentation. https://docs.renovatebot.com/. Accessed: 2026-07-02.

[21] Michael E Ellis, K Mike Casey, and Geoffrey Hill. Chatgpt and python programming homework. *Decision Sciences Journal of Innovative Education*, 22(2):74–87, 2024.

[22] Timm Teubner, Christoph M Flath, Christof Weinhardt, Wil van der Aalst, and Oliver Hinz. Welcome to the era of chatgpt et al. the prospects of large language models. *Business & Information Systems Engineering*, 65(2):95–101, 2023.

[23] Moataz Chouchen, Narjes Bessghaier, Mahi Begoug, Ali Ouni, Eman Alomar, and Mohamed Wiem Mkaouer. How do software developers use chatgpt? an exploratory study on github pull requests. In *Proceedings of the 21st International Conference on Mining Software Repositories*, pages 212–216, 2024.

[24] Md Asraful Haque. Llms: A game-changer for software engineers? *BenchCouncil Transactions on Benchmarks, Standards and Evaluations*, 5(1):100204, 2025.

[25] Jasmine Latendresse, SayedHassan Khatoonabadi, and Emad Shihab. How robust are llm-generated library imports? an empirical study using stack overflow. *arXiv preprint arXiv:2507.10818*, 2025.

[26] Bhanu Prakash Vangala, Ali Adibifar, Ashish Gehani, and Tanu Malik. Ai-generated code is not reproducible (yet): an empirical study of dependency gaps in llm-based coding agents. *arXiv preprint arXiv:2512.22387*, 2025.

[27] Jasmine Latendresse, Naoures Day, SayedHassan Khatoonabadi, and Emad Shihab. The software

librarian: Python package insights for copilot. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, pages 17–20. IEEE, 2025.

[28] Jasmine Latendresse and Emad Shihab. Evaluating large language models for dependency-maintenance patch generation. *ACM Transactions on Software Engineering and Methodology*, jun 2026. Manuscript submitted for publication.

[29] Slinger Jansen, Anthony Finkelstein, and Sjaak Brinkkemper. A sense of community: A research agenda for software ecosystems. In *2009 31st International Conference on Software Engineering-Companion Volume*, pages 187–190. IEEE, 2009.

[30] npm. About npm. https://docs.npmjs.com/about-npm/. Accessed: 2026-07-03.

[31] Mahmoud Alfadel, Diego Elias Costa, Emad Shihab, and Bram Adams. On the discoverability of npm vulnerabilities in node.js projects. *ACM Transactions on Software Engineering and Methodology*, 32 (4):1–27, 2023. doi:10.1145/3571848.

[32] Alexandre Decan, Tom Mens, and Eleni Constantinou. On the evolution of technical lag in the npm package dependency network. In *2018 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 404–414. IEEE, 2018.

[33] Ahmed Zerouali, Eleni Constantinou, Tom Mens, Gregorio Robles, and Jesús González-Barahona. An empirical analysis of technical lag in npm package dependencies. In *International Conference on Software Reuse*, pages 95–110. Springer, 2018.

[34] Tobias Lauinger, Abdelberi Chaabane, and Christo Wilson. Thou shalt not depend on me: A look at javascript libraries in the wild. *Queue*, 16(1):62–82, 2018.

[35] Ahmed Zerouali, Tom Mens, and Coen De Roover. On the usage of javascript, python and ruby packages in docker hub images. *Science of Computer Programming*, 207:102653, 2021.

[36] Ahmed Zerouali, Ruben Opdebeeck, and Coen De Roover. Helm charts for kubernetes applications: Evolution, outdatedness and security risks. In *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*, pages 523–533. IEEE, 2023.

[37] Aline Brito, Marco Tulio Valente, Laerte Xavier, and Andre Hora. You broke my code: understanding the motivations for breaking changes in apis. *Empirical Software Engineering*, 25:1458–1492, 2020.

[38] Chris Bogart, Christian Kästner, James Herbsleb, and Ferdian Thung. When and how to make breaking changes: Policies and practices in 18 open source software ecosystems. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 30(4):1–56, 2021.

[39] Courtney Miller, Mahmoud Jahanshahi, Audris Mockus, Bogdan Vasilescu, and Christian Kästner. Understanding the response to open-source dependency abandonment in the npm ecosystem. In *Intl Conf. Software Engineering (ICSE), IEEE/ACM*, 2025.

[40] Kaylea Champion and Benjamin Mako Hill. Underproduction: An approach for measuring risk in open source software. In *2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 388–399. IEEE, 2021.

[41] Théo Zimmermann and Jean-Rémy Falleri. A grounded theory of community package maintenance organizations. *Empirical Software Engineering*, 28(4):101, 2023.

[42] Bodin Chinthanet, Raula Gaikovina Kula, Shane McIntosh, Takashi Ishio, Akinori Ihara, and Kenichi Matsumoto. Lags in the release, adoption, and propagation of npm vulnerability fixes. *Empirical Software Engineering*, 26:1–28, 2021.

[43] Chengwei Liu, Sen Chen, Lingling Fan, Bihuan Chen, Yang Liu, and Xin Peng. Demystifying the vulnerability propagation and its evolution via dependency trees in the npm ecosystem. In *Proceedings of the 44th international conference on software engineering*, pages 672–684, 2022.

[44] Marc Ohm, Arnold Sykosch, and Michael Meier. Towards detection of software supply chain attacks by forensic artifacts. In *Proceedings of the 15th international conference on availability, reliability and security*, pages 1–6, 2020.

[45] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. Backstabbers knife collection: A review of open source software supply chain attacks. In *Detection of Intrusions and Malware, and Vulnerability Assessment: 17th International Conference, DIMVA 2020, Lisbon, Portugal, June 24–26, 2020, Proceedings 17*, pages 23–43. Springer, 2020.

[46] William Enck and Laurie Williams. Top five challenges in software supply chain security: Observations from 30 industry and government organizations. *IEEE Security & Privacy*, 20(2):96–100, 2022.

[47] Alex Birsan. Dependency confusion: How i hacked into apple, microsoft and dozens of other companies. *URL: medium. com/@ alex. birsan/dependency-confusion-4a5d60fec610*, 2021.

[48] César Soto-Valero, Nicolas Harrand, Martin Monperrus, and Benoit Baudry. A comprehensive study of bloated dependencies in the maven ecosystem. *Empirical Software Engineering*, 26(3):45, 2021.

[49] César Soto-Valero, Thomas Durieux, and Benoit Baudry. A longitudinal analysis of bloated java dependencies. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 1021–1031, 2021.

[50] Jeremy Rack and Cristian-Alexandru Staicu. Jack-in-the-box: An empirical study of javascript bundling on the web and its security implications. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 3198–3212, 2023.

[51] Yuxin Liu, Deepika Tiwari, Cristian Bogdan, and Benoit Baudry. Detecting and removing bloated dependencies in commonjs packages. *Journal of Systems and Software*, 230:112509, 2025.

[52] Georgios-Petros Drosos, Thodoris Sotiropoulos, Diomidis Spinellis, and Dimitris Mitropoulos. Bloat beneath pythons scales: A fine-grained inter-project dependency analysis. *Proceedings of the ACM on Software Engineering*, 1(FSE):2584–2607, 2024.

[53] Nimmi Rashinika Weeraddana, Mahmoud Alfadel, and Shane McIntosh. Dependency-induced waste in continuous integration: An empirical study of unused dependencies in the npm ecosystem. *Proceedings of the ACM on Software Engineering*, 1(FSE):2632–2655, 2024.

[54] Ferdian Thung, David Lo, and Julia Lawall. Automated library recommendation. In *2013 20th Working conference on reverse engineering (WCRE)*, pages 182–191. IEEE, 2013.

[55] Ali Ouni, Raula Gaikovina Kula, Marouane Kessentini, Takashi Ishio, Daniel M German, and Katsuro Inoue. Search-based software library recommendation using multi-objective optimization. *Information and Software Technology*, 83:55–75, 2017.

[56] Mohamed Aymen Saied, Ali Ouni, Houari Sahraoui, Raula Gaikovina Kula, Katsuro Inoue, and David Lo. Automated inference of software library usage patterns. *arXiv preprint arXiv:1612.01626*, 2016.

[57] Phuong T Nguyen, Juri Di Rocco, Davide Di Ruscio, and Massimiliano Di Penta. Crossrec: Supporting software developers by recommending third-party libraries. *Journal of Systems and Software*, 161: 110460, 2020.

[58] Chunyang Chen and Zhenchang Xing. Similartech: automatically recommend analogical libraries across different programming languages. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering*, pages 834–839, 2016.

[59] Md Mahir Asef Kabir, Ying Wang, Danfeng Yao, and Na Meng. How do developers follow security-relevant best practices when using npm packages? In *2022 IEEE Secure development conference (SecDev)*, pages 77–83. IEEE, 2022.

[60] Sabato Nocera, Simone Romano, Massimiliano Di Penta, Rita Francese, and Giuseppe Scanniello. Software bill of materials adoption: A mining study from github. In *2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 39–49. IEEE, 2023.

[61] Sihan Xu, Ya Gao, Lingling Fan, Zheli Liu, Yang Liu, and Hua Ji. Lidetector: License incompatibility detection for open source software. *ACM Transactions on Software Engineering and Methodology*, 32 (1):1–28, 2023.

[62] Kawsar Ahmed Bhuiyan, Mohamed Bilel Besbes, Rachna Raj, Adam Al Assil, and Diego Elias Costa. Beyond compliance: A large scale study on the completeness and consistency of the github sboms. *arXiv preprint arXiv:2607.04614*, 2026.

[63] Hamid Mohayeji, Andrei Agaronian, Eleni Constantinou, Nicola Zannone, and Alexander Serebrenik. Investigating the resolution of vulnerable dependencies with dependabot security updates. In *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*, pages 234–246. IEEE, 2023.

[64] Alexandre Decan and Tom Mens. What do package dependencies tell us about semantic versioning? *IEEE Transactions on Software Engineering*, 47(6):1226–1240, 2019.

[65] Zhaoxu Zhang, Hengcheng Zhu, Ming Wen, Yida Tao, Yepang Liu, and Yingfei Xiong. How do python framework apis evolve? an exploratory study. In *2020 ieee 27th international conference on software analysis, evolution and reengineering (saner)*, pages 81–92. IEEE, 2020.

[66] Xingliang Du and Jun Ma. Aexpy: Detecting api breaking changes in python packages. In *2022 IEEE 33rd International Symposium on Software Reliability Engineering (ISSRE)*, pages 470–481. IEEE, 2022.

[67] Mahmoud Alfadel, Diego Elias Costa, Emad Shihab, and Mouafak Mkhallalati. On the use of dependabot security pull requests. In *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*, pages 254–265, 2021. doi:10.1109/MSR52588.2021.00037.

[68] Runzhi He, Hao He, Yuxia Zhang, and Minghui Zhou. Automating dependency updates in practice: An exploratory study on github dependabot. *IEEE Transactions on Software Engineering*, 49(8): 4004–4022, 2023.

[69] Bar Lanyado. Can you trust ChatGPTs package recommendations?, 3 2023. URL https://vulcan.io/blog/ai-hallucinations-package-risk.

[70] Aylton Almeida, Laerte Xavier, and Marco Tulio Valente. Automatic library migration using large language models: First results. In *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, pages 427–433. Association for Computing Machinery, 2024. doi:10.1145/3674805.3690746.

[71] Md Mohayeminul Islam, Ajay Kumar Jha, May Mahmoud, Ildar Akhmetov, and Sarah Nadi. An empirical study of python library migration using large language models. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering*, pages 867–879. IEEE, 2025. doi:10.1109/ASE63991.2025.00077.

[72] Celal Ziftci, Stoyan Nikolov, Anna Sjövall, Bo Kim, Daniele Codecasa, and Max Kim. Migrating code at scale with llms at google. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, FSE Companion '25, pages 162–173. Association for Computing Machinery, 2025. doi:10.1145/3696630.3728542.

[73] Lukas Fruntke and Jens Krinke. Automatically fixing dependency breaking changes. *Proceedings of the ACM on Software Engineering*, 2(FSE):2146–2168, 2025.

[74] Frank Reyes, May Mahmoud, Federico Bono, Sarah Nadi, Benoit Baudry, and Martin Monperrus. Byam: Fixing breaking dependency updates with large language models. *Empirical Software Engineering*, 31 (4):113, 2026.

[75] Huanting Wang, Jingzhi Gong, Huawei Zhang, Jie Xu, and Zheng Wang. AI agentic programming: A survey of techniques, challenges, and opportunities. *arXiv preprint arXiv:2508.11126*, 2025. doi:10.48550/arXiv.2508.11126.

[76] Miku Watanabe, Hao Li, Yutaro Kashiwa, Brittany Reid, Hajimu Iida, and Ahmed E. Hassan. On the use of agentic coding: An empirical study of pull requests on GitHub. *ACM Transactions on Software Engineering and Methodology*, 2026. doi:10.1145/3798166. Just Accepted.

[77] Rachna Raj and Diego Elias Costa. The role of library versions in developer-chatgpt conversations. In *Proceedings of the 21st International Conference on Mining Software Repositories*, pages 172–176, 2024.

[78] Ruiyin Li, Peng Liang, Yifei Wang, Yangxiao Cai, Weisong Sun, and Zengyang Li. Unveiling the role of chatgpt in software development: Insights from developer-chatgpt interactions on github. *ACM Transactions on Software Engineering and Methodology*, 2025.

[79] Eman Abdullah AlOmar, Anushkrishna Venkatakrishnan, Mohamed Wiem Mkaouer, Christian Newman, and Ali Ouni. How to refactor this code? an exploratory study on developer-chatgpt refactoring conversations. In *Proceedings of the 21st International Conference on Mining Software Repositories*, pages 202–206, 2024.

[80] Securing the world's software, 2020. URL https://octoverse.github.com/static/github-octoverse-2020-security-report.pdf.

[81] Stefan Wagner and Emerson Murphy-Hill. *Factors That Influence Productivity: A Checklist*, pages 69–84. 05 2019. ISBN 978-1-4842-4220-9. doi:10.1007/978-1-4842-4221-6_8.

[82] 2019 state of the software supply chain, 2019. URL https://www.sonatype.com/hubfs/SSC/2019%20SSC/SON_SSSC-Report-2019_jun16-DRAFT.pdf.

[83] Josh Fruhlinger. Equifax data breach faq: What happened, who was affected, what was the impact?, 02 2020. URL https://www.csoonline.com/article/3444488/equifax-data-breach-faq-what-happened-who-was-affected-what-was-the-impact.html.

[84] Ivan Pashchenko, Henrik Plate, Serena Ponta, Antonino Sabetta, and Fabio Massacci. Vulnerable open source dependencies: counting those that matter. pages 1–10, 10 2018. doi:10.1145/3239235.3268920.

[85] Ivan Pashchenko, Henrik Plate, Serena Elisa Ponta, Antonino Sabetta, and Fabio Massacci. Vuln4real: A methodology for counting actually vulnerable dependencies. *IEEE Transactions on Software Engineering*, 48(5):1592–1609, 2020.

[86] Stack Overflow. Stack overflow developer survey 2021, 2021. URL https://insights.stackoverflow.com/survey/2021. (Accessed on 05/06/2022).

[87] Create react app, 2021. URL https://create-react-app.dev/.

[88] The complete guide to software composition analysis - fossa, 2022. URL https://fossa.com/complete-guide-software-composition-analysis.

[89] webpack, 2019. URL https://webpack.js.org/.

[90] Baishakhi Ray, Daryl Posnett, Vladimir Filkov, and Premkumar Devanbu. A large scale study of programming languages and code quality in github. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, FSE 2014, page 155165, New York, NY, USA, 2014. Association for Computing Machinery. ISBN 9781450330565. doi:10.1145/2635868.2635922. URL https://doi.org/10.1145/2635868.2635922.

[91] Emitza Guzman, David Azócar, and Yang Li. Sentiment analysis of commit comments in github: An empirical study. In *Proceedings of the 11th Working Conference on Mining Software Repositories*, MSR 2014, page 352355, New York, NY, USA, 2014. Association for Computing Machinery. ISBN 9781450328630. doi:10.1145/2597073.2597118. URL https://doi.org/10.1145/2597073.2597118.

[92] Rodrigo Zapata, Raula Kula, Bodin Chinthanet, Takashi Ishio, Kenichi Matsumoto, and Akinori Ihara. Towards smoother library migrations: A look at vulnerable dependency migrations at function level for npm javascript packages. pages 559–563, 09 2018. doi:10.1109/ICSME.2018.00067.

[93] rollup.js, 2021. URL https://rollupjs.org/guide/en/.

[94] unisil. Source map parser, 07 2021. URL https://github.com/unisil/source-map-parser.

[95] Nasif Imtiaz, Seaver Thorn, and Laurie Williams. A comparative study of vulnerability reporting by software composition analysis tools. *Proceedings of the 15th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, 10 2021. doi:10.1145/3475716.3475769. URL https://arxiv.org/pdf/2108.12078.pdf.

[96] Md Atique, Reza Chowdhury, Rabe Abdalkareem, and Emad Shihab. On the untriviality of trivial packages: An empirical study of npm javascript packages. *Journal of IEEE Transactions on Software Engineering*, 01, 2019. URL http://das.encs.concordia.ca/uploads/atique_tse2021.pdf.

[97] Rabe Abdalkareem, Vinicius Oda, Suhaib Mujahid, and Emad Shihab. On the impact of using trivial packages: an empirical case study on npm and pypi. *Empirical Software Engineering*, 25:1168–1204, 01 2020. doi:10.1007/s10664-019-09792-9.

[98] Xiaowei Chen, Rabe Abdalkareem, Suhaib Mujahid, Emad Shihab, and Xin Xia. Helping or not helping? why and how trivial packages impact the npm ecosystem. *Empirical Software Engineering*, 26, 03 2021. doi:10.1007/s10664-020-09904-w.

[99] Joel Cox, Eric Bouwers, Marko van Eekelen, and Joost Visser. Measuring dependency freshness in software systems. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, volume 2, pages 109–118, 2015. doi:10.1109/ICSE.2015.140.

[100] Tobias Lauinger, Abdelberi Chaabane, Sajjad Arshad, William Robertson, Christo Wilson, and Engin Kirda. Thou shalt not depend on me: Analysing the use of outdated JavaScript libraries on the web. In *Proceedings 2017 Network and Distributed System Security Symposium*. Internet Society, 2017. doi:10.14722/ndss.2017.23414. URL https://doi.org/10.14722%2Fndss.2017.23414.

[101] Do ”dependencies” and ”devdependencies” matter when using webpack?, 03 2020. URL https://jsramblings.com/do-dependencies-devdependencies-matter-when-using-webpack/.

[102] Help, ‘npm audit‘ says i have a vulnerability in react-scripts! issue #11174 facebook/create-react-app, 2021. URL https://github.com/facebook/create-react-app/issues/11174.

[103] Ivan Pashchenko, Duc-Ly Vu, and Fabio Massacci. A qualitative study of dependency management and its security implications. In *Proceedings of the 2020 ACM SIGSAC conference on computer and communications security*, pages 1513–1531, 2020.

[104] npm-deps-parser, 09 2020. URL https://github.com/nVisium/npm-deps-parser.

[105] Github advisory database, 2022. URL https://github.com/advisories.

[106] Haroen Viaene. feat(dependencies): update algoliasearch-helper. https://github.com/algolia/instantsearch.js/pull/4936, October 2021. (Accessed on 05/04/2022).

[107] Chris Bogart, Christian Kstner, James Herbsleb, and Ferdian Thung. When and how to make breaking changes. *ACM Transactions on Software Engineering and Methodology*, 30:1–56, 07 2021. doi:10.1145/3447245.

[108] Snyk — developer security — develop fast. stay secure., 2022. URL https://snyk.io/.

[109] Software bill of materials — cisa, 2022. URL https://www.cisa.gov/sbom.

[110] Rabe Abdalkareem, Olivier Nourry, Sultan Wehaibi, Suhaib Mujahid, and Emad Shihab. Why do developers use trivial packages? an empirical case study on npm. *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, 08 2017. doi:10.1145/3106237.3106267.

[111] Diego Elias Costa, Suhaib Mujahid, Rabe Abdalkareem, and Emad Shihab. Breaking type-safety in go: An empirical study on the usage of the unsafe package. *IEEE Transactions on Software Engineering*, pages 1–1, 2021. doi:10.1109/TSE.2021.3057720.

[112] Suhaib Mujahid, Diego Elias Costa, Rabe Abdalkareem, Emad Shihab, Mohamed Aymen Saied, and Bram Adams. Toward using package centrality trend to identify packages in decline. *IEEE Transactions on Engineering Management*, pages 1–15, 2021. doi:10.1109/tem.2021.3122012. URL http://das.encs.concordia.ca/uploads/Mujahid_TEM2021.pdf.

[113] Stan Zajdel, Diego Elias Costa, and Hafedh Mili. Open source software: An approach to controlling usage and risk in application ecosystems. In *Proceedings of the 26TH ACM International Systems and Software Product Line Conference*. arXiv, 2022. doi:10.48550/ARXIV.2206.10358.

[114] Jailton Coelho, Marco Tulio Valente, Luciano Milen, and Luciana L. Silva. Is this GitHub project maintained? measuring the level of maintenance activity of open-source projects. *Information and Software Technology*, 122:106274, jun 2020. doi:10.1016/j.infsof.2020.106274. URL https://doi.org/10.1016%2Fj.infsof.2020.106274.

[115] Riivo Kikas, Georgios Gousios, Marlon Dumas, and Dietmar Pfahl. Structure and evolution of package dependency networks. In *Proceedings of the 14th International Conference on Mining Software Repositories*, MSR '17, page 102112. IEEE Press, 2017. ISBN 9781538615447. doi:10.1109/MSR.2017.55. URL https://doi.org/10.1109/MSR.2017.55.

[116] J. I. Hejderup. In dependencies we trust: How vulnerable are dependencies in software modules? *repository.tudelft.nl*, 2015. URL https://repository.tudelft.nl/islandora/object/uuid:3a15293b-16f6-4e9d-b6a2-f02cd52f1a9e?collection=education.

[117] Jeff Williams and Arshan Dabirsiaghi. The unfortunate reality of insecure libraries. *Asp. Secur. Inc*, pages 1–26, 2012.

[118] Henrik Plate, Serena Ponta, and Antonino Sabetta. Impact assessment for vulnerabilities in open-source software libraries. pages 411–420, 09 2015. doi:10.1109/ICSM.2015.7332492.

[119] Adriana Sejfia and Max Schäfer. Practical automated detection of malicious npm packages. *arXiv preprint arXiv:2202.13953*, 2022.

[120] Suhaib Mujahid. *Effective Dependency Management for the JavaScript Software Ecosystem*. PhD thesis, Concordia University, 2021.

[121] Suhaib Mujahid, Rabe Abdalkareem, and Emad Shihab. What are the characteristics of highly-selected packages? a case study on the npm ecosystem. *Journal of Systems and Software*, 198:111588, 2023.

[122] Oyvind Hauge, Thomas Osterlie, Carl-Fredrik Sorensen, and Marinela Gerea. An empirical study on selection of open source software-preliminary results. In *2009 ICSE Workshop on Emerging Trends in Free/Libre/Open Source Software Research and Development*, pages 42–47. IEEE, 2009.

[123] Nicole Haenni, Mircea Lungu, Niko Schwarz, and Oscar Nierstrasz. Categorizing developer information needs in software ecosystems. In *Proceedings of the 2013 international workshop on ecosystem architectures*, pages 1–5, 2013.

[124] Enrique Larios Vargas, Maurício Aniche, Christoph Treude, Magiel Bruntink, and Georgios Gousios. Selecting third-party libraries: the practitioners perspective. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2020, page 245256, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450370431. doi:10.1145/3368089.3409711. URL https://doi.org/10.1145/3368089.3409711.

[125] Gianluca Mezzetti, Anders Møller, and Martin Toldam Torp. Type regression testing to detect breaking changes in node. js libraries. In *32nd european conference on object-oriented programming (ECOOP 2018)*, pages 7–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2018.

[126] Anders Møller and Martin Toldam Torp. Model-based testing of breaking changes in node. js libraries. In *Proceedings of the 2019 27th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*, pages 409–419, 2019.

[127] Yulu Cao, Lin Chen, Wanwangying Ma, Yanhui Li, Yuming Zhou, and Linzhang Wang. Towards better dependency management: A first look at dependency smells in python projects. *IEEE Transactions on Software Engineering*, 49(4):1741–1765, 2022.

[128] César Soto-Valero, Thomas Durieux, Nicolas Harrand, and Benoit Baudry. Coverage-based debloating for java bytecode. *ACM Transactions on Software Engineering and Methodology*, 32(2):1–34, 2023.

[129] Zhouyang Jia, Shanshan Li, Tingting Yu, Chen Zeng, Erci Xu, Xiaodong Liu, Ji Wang, and Xiangke Liao. Depowl: detecting dependency bugs to prevent compatibility failures. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, pages 86–98. IEEE, 2021.

[130] Jenny T Liang, Chenyang Yang, and Brad A Myers. A large-scale survey on the usability of ai programming assistants: Successes and challenges. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, pages 1–13, 2024.

[131] Steven I Ross, Fernando Martinez, Stephanie Houde, Michael Muller, and Justin D Weisz. The programmers assistant: Conversational interaction with a large language model for software development. In *Proceedings of the 28th International Conference on Intelligent User Interfaces*, pages 491–514, 2023.

[132] Luc Bryan Heitz, Joun Chamas, and Christopher Scherb. Evaluation of the programming skills of large language models. *arXiv preprint arXiv:2405.14388*, 2024.

[133] Stack Overflow - where developers learn, share, & build careers. URL https://stackoverflow.com/.

[134] Nidhish Shah, Zulkuf Genc, and Dogu Araci. Stackeval: Benchmarking llms in coding assistance. *Advances in Neural Information Processing Systems*, 37:36976–36994, 2024.

[135] Jacob Haimes, Cenny Wenner, Kunvar Thaman, Vassil Tashev, Clement Neo, Esben Kran, and Jason Schreiber. Benchmark inflation: Revealing llm performance gaps using retro-holdouts, 2024. *URL https://arxiv. org/abs/2410.09247*.

[136] Alexandre Matton, Tom Sherborne, Dennis Aumiller, Elena Tommasone, Milad Alizadeh, Jingyi He, Raymond Ma, Maxime Voisin, Ellen Gilsenan-McMahon, and Matthias Gallé. On leakage of code generation evaluation datasets. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 13215–13223, 2024.

[137] AL Sayeth Saabith, T Vinothraj, and M Fareez. Popular python libraries and their application domains. *International Journal of Advance Engineering and Research Development*, 7(11), 2020.

[138] KR Srinath. Python–the fastest growing programming language. *International Research Journal of Engineering and Technology*, 4(12):354–357, 2017.

[139] Victor Adamson and Johan Bägerfeldt. Assessing the effectiveness of chatgpt in generating python code, 2023.

[140] Tianyang Zhong, Zhengliang Liu, Yi Pan, Yutong Zhang, Yifan Zhou, Shizhe Liang, Zihao Wu, Yanjun Lyu, Peng Shu, Xiaowei Yu, et al. Evaluation of openai o1: Opportunities and challenges of agi. *arXiv preprint arXiv:2409.18486*, 2024.

[141] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[142] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao,

Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[143] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[144] Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, and Daxin Jiang. Wizardlm: Empowering large language models to follow complex instructions. *arXiv preprint arXiv:2304.12244*, 2023.

[145] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.

[146] Russ Cox. Surviving software dependencies. *Communications of the ACM*, 62(9):36–43, 2019.

[147] Yoonsik Cheon et al. Llms as code generators for model-driven development. In *Proceedings of the 20th International Conference on Software Technologies (ICSOFT)*, 2025.

[148] Luke Twist et al. A study of LLMs' preferences for libraries and programming languages. *arXiv preprint arXiv:2503.17181*, 2025.

[149] Gang Fan, Chengpeng Wang, Rongxin Wu, Xiao Xiao, Qingkai Shi, and Charles Zhang. Escaping dependency hell: finding build dependency errors with the unified dependency graph. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 463–474, 2020.

[150] Yudai Tanabe, Tomoyuki Aotani, and Hidehiko Masuhara. A context-oriented programming approach to dependency hell. In *Proceedings of the 10th ACM International Workshop on Context-Oriented Programming: Advanced Modularity for Run-time Composition*, pages 8–14, 2018.

[151] Steven Raemaekers, Arie van Deursen, and Joost Visser. Exploring risks in the usage of third-party libraries. In *of the BElgian-NEtherlands software eVOLution seminar*, volume 31, 2011.

[152] Li Zhong and Zilong Wang. Can chatgpt replace stackoverflow? a study on robustness and reliability of large language model code generation, 2024.

[153] Kailun Jin, Chung-Yu Wang, Hung Viet Pham, and Hadi Hemmati. Can chatgpt support developers? an empirical evaluation of large language models for code generation. *arXiv preprint arXiv:2402.11702*, 2024.

[154] Anwoy Chatterjee, HSVNS Kowndinya Renduchintala, Sumit Bhatia, and Tanmoy Chakraborty. Posix: A prompt sensitivity index for large language models. *arXiv preprint arXiv:2410.02185*, 2024.

[155] Frank Reyes, Yogya Gamage, Gabriel Skoglund, Benoit Baudry, and Martin Monperrus. Bump: A benchmark of reproducible breaking dependency updates. In *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 159–170, 2024. doi:10.1109/SANER60148.2024.00024.

[156] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, volume 2024, pages 54107–54157, 2024.

[157] Ramakrishna Bairi, Atharv Sonwane, Aditya Kanade, Vageesh D C, Arun Iyer, Suresh Parthasarathy, Sriram Rajamani, Balasubramanyan Ashok, and Shashank Shet. Codeplan: Repository-level coding using llms and planning. *Proceedings of the ACM on Software Engineering*, 1(FSE):675–698, 2024.

[158] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. Repairagent: An autonomous, llm-based agent for program repair. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pages 2188–2200. IEEE, 2025.

[159] npm. Specifying dependencies and `devDependencies` in a `package.json` file, aug 2024. URL https://docs.npmjs.com/specifying-dependencies-and-devdependencies-in-a-package-json-file/.

[160] package-lock.json — npm Docs. URL https://docs.npmjs.com/cli/v11/configuring-npm/package-lock-json,.

[161] GNU Project. Unified format, 2025. URL https://www.gnu.org/software/diffutils/manual/html_node/Unified-Format.html. GNU Diffutils Manual.

[162] Git Project. `git-apply` documentation, 2025. URL https://git-scm.com/docs/git-apply. Git Reference Manual.

[163] Peter A Lachenbruch. Mcnemar test. *Wiley StatsRef: Statistics Reference Online*, 2014.

[164] Albert H Bowker. A test for symmetry in contingency tables. *Journal of the american statistical association*, 43(244):572–574, 1948.

[165] Nikhil Parasaram, Huijie Yan, Boyu Yang, Zineb Flahy, Abriele Qudsi, Damian Ziaber, Earl T Barr, and Sergey Mechtaev. The fact selection problem in llm-based program repair. in 2025 ieee/acm 47th international conference on software engineering (icse), 2025.

[166] Filipe R. Cogo and Ahmed E. Hassan. Understanding the customization of dependency bots: The case of Dependabot. *IEEE Software*, 39(5):44–49, 2022. doi:10.1109/MS.2022.3179484.

[167] Hocine Rebatchi, T'egawend'e F. Bissyand'e, and Naouel Moha. Dependabot and security pull requests: Large empirical study. *Empirical Software Engineering*, 29(5):128, 2024. doi:10.1007/s10664-024-10523-y.

[168] Benjamin Rombaut, Filipe R Cogo, Bram Adams, and Ahmed E Hassan. Theres no such thing as a free lunch: Lessons learned from exploring the overhead introduced by the greenkeeper dependency bot in npm. *ACM Transactions on Software Engineering and Methodology*, 32(1):1–40, 2023.

[169] Joseph Hejderup and Georgios Gousios. Can we trust tests to automate dependency updates? a case study of Java projects. *Journal of Systems and Software*, 183:111097, 2022. doi:10.1016/j.jss.2021.111097.

[170] Andreas Dann, Ben Hermann, and Eric Bodden. UpCy: Safely updating outdated dependencies. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering*, pages 233–244. IEEE, 2023. doi:10.1109/ICSE48619.2023.00031.

[171] Damien Jaime, Pascal Poizat, Joyce El Haddad, and Thomas Degueule. Balancing the quality and cost of updating dependencies. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, pages 1834–1845. ACM, 2024. doi:10.1145/3691620.3695595.

[172] Vali Tawosi, Salwa Alamir, Xiaomo Liu, and Manuela Veloso. LLM agents for automated dependency

upgrades. In *2025 IEEE/ACM 40th International Conference on Automated Software Engineering Workshops (ASEW)*, pages 34–37. IEEE, 2025. doi:10.1109/ASEW67777.2025.00016.

[173] Yifan Xia, Chengwei Liu, Zifan Xie, Lyuye Zhang, Peiyu Liu, Kangjie Lu, Yang Liu, Wenhai Wang, and Shouling Ji. Break to adapt: Knowledge-based updates of breaking dependencies in JavaScript. *Proceedings of the ACM on Software Engineering*, 3(FSE), jul 2026. doi:10.1145/3808116.

Appendices

This appendix provides supplementary material for the study ”Evaluating Large Language Models for Dependency-Maintenance Patch Generation”. It includes the exact prompt templates used in the experiment and the model-level statistical analyses supporting the results presented in the corresponding thesis chapter.

A.1 Prompt Templates

This section presents the exact prompt templates used in the experiment. Text enclosed in braces denotes a task-specific placeholder that was replaced during prompt construction. The same system instruction was used for all task and prompting variants.

A.1.1 Shared System Instruction

```
You are a software maintenance assistant. Produce a unified diff patch. Output ONLY the
diff.
```

A.1.2 Configuration-Only Prompts

Natural Prompt.

```
Task: Update dependency versions in a JavaScript project.

Update instructions (apply exactly):
{UPDATE_LINES}

Return a unified diff patch that applies to the file(s) below.
Output ONLY the diff (no explanations, no markdown).
```

```
Files (base version):
--- {PACKAGE_JSON_PATH} ---
{PACKAGE_JSON_CONTENT}
```

Surgical Prompt.

```
Task: Apply a dependency version update (config-only).

Rules (must follow):

* ONLY edit the specific dependency entries listed under
  "Update instructions".
* Do NOT change any other keys/values in package.json
  (including scripts, engines, metadata, formatting, or
  ordering).
* Do NOT add or remove dependencies.
* Do NOT touch any other files.
* Output ONLY a unified diff patch. No explanations.

Update instructions (apply exactly):
{UPDATE_LINES}

Files (base version):
--- {PACKAGE_JSON_PATH} ---
{PACKAGE_JSON_CONTENT}
```

A.1.3 Configuration-Code Prompts

Natural Prompt.

```
Task: Apply a dependency update and update local code as needed.

Update instructions (apply exactly in package.json):
{UPDATE_LINES}
```

```
Make any minimal code changes needed in the provided files to
accommodate the dependency update.
Return a unified diff patch that applies to the files below.
Output ONLY the diff (no explanations, no markdown).

Files (base version):
{FILES_SECTION}
```

Surgical Prompt.

```
Task: Apply a dependency update and fix only the local API usage
needed for this update.

Rules (must follow):

* Apply ONLY the dependency changes listed under
  "Update instructions" in package.json.
* Edit ONLY the files provided below. Do NOT create new files.
* Make the smallest changes necessary; do not refactor unrelated
  code.
* Do not change formatting unless required for correctness.
* Output ONLY a unified diff patch. No explanations.

Update instructions (apply exactly):
{UPDATE_LINES}

Files (base version):
{FILES_SECTION}
```

A.1.4 Template Placeholders

The templates use the following task-specific placeholders:

{**UPDATE_LINES**} The update instruction generated from the mined pull request, including the manifest path, dependency section, dependency name, current specification, and target specification.

{**PACKAGE_JSON_PATH**} The repository-relative path of the project manifest.

{**PACKAGE_JSON_CONTENT**} The complete contents of the base-version `package.json`.

{**FILES_SECTION**} The base-version manifest and selected source or test files, with each file preceded by its repository-relative path.

A.2 Model-Level Prompt-Style Analyses

This section reports the model-level prompt-style comparisons supporting the aggregate analyses presented in the main text. All comparisons use model–task instances for which outputs are available under both natural and surgical prompting.

In Tables A.1 and A.3, Holm correction is applied separately within each model across the outcomes reported for that model. For the failure analyses in Tables A.2 and A.4, the failure-presence comparison includes all paired eligible instances, whereas the Bowker symmetry tests include only pairs that are unsuccessful under both prompt styles. The failure-distribution comparisons are exploratory and report unadjusted p-values.

Table A.1: Model-level paired prompt-style comparisons for configuration-only manifest outcomes.

Model	**Outcome**	**Natural**	**Surgical**	**Paired n**	**Raw p**	**Holm p**
DeepSeek-V3.1	Usable patch	56/56	56/56	56	1.0000	1.0000
DeepSeek-V3.1	Applicable patch	45/56	44/56	56	1.0000	1.0000
DeepSeek-V3.1	Semantic success	46/56	45/56	56	1.0000	1.0000
DeepSeek-V3.1	Patch success	42/56	43/56	56	1.0000	1.0000
Gemma-3n-E4B-it	Usable patch	52/54	51/54	54	1.0000	1.0000
Gemma-3n-E4B-it	Applicable patch	4/54	4/54	54	1.0000	1.0000
Gemma-3n-E4B-it	Semantic success	12/54	14/54	54	0.7539	1.0000
Gemma-3n-E4B-it	Patch success	1/54	1/54	54	1.0000	1.0000
Llama-3.3-70B	Usable patch	56/56	56/56	56	1.0000	1.0000
Llama-3.3-70B	Applicable patch	14/56	15/56	56	1.0000	1.0000
Llama-3.3-70B	Semantic success	30/56	33/56	56	0.4531	1.0000
Llama-3.3-70B	Patch success	12/56	13/56	56	1.0000	1.0000
GPT-OSS-120B	Usable patch	56/56	56/56	56	1.0000	1.0000
GPT-OSS-120B	Applicable patch	47/56	51/56	56	0.3438	1.0000
GPT-OSS-120B	Semantic success	54/56	54/56	56	1.0000	1.0000
GPT-OSS-120B	Patch success	47/56	51/56	56	0.3438	1.0000

Table A.2: Model-level prompt-style comparisons of configuration-only failures.

Model	**Comparison**	n	**Test**	p**-value**
DeepSeek-V3.1	Failure presence	56	Exact McNemar	1.0000
DeepSeek-V3.1	Failure-category distribution	7	Bowker symmetry	0.4060
DeepSeek-V3.1	Failure-subtype distribution	7	Bowker symmetry	0.4060
Gemma-3n-E4B-it	Failure presence	54	Exact McNemar	1.0000
Gemma-3n-E4B-it	Failure-category distribution	52	Bowker symmetry	0.0719
Gemma-3n-E4B-it	Failure-subtype distribution	52	Bowker symmetry	0.1777
Llama-3.3-70B	Failure presence	56	Exact McNemar	1.0000
Llama-3.3-70B	Failure-category distribution	40	Bowker symmetry	0.0216
Llama-3.3-70B	Failure-subtype distribution	40	Bowker symmetry	0.0620
GPT-OSS-120B	Failure presence	56	Exact McNemar	0.3438
GPT-OSS-120B	Failure-category distribution	2	Bowker symmetry	1.0000
GPT-OSS-120B	Failure-subtype distribution	2	Bowker symmetry	1.0000

Table A.3: Model-level paired prompt-style comparisons for configuration-code manifest, source-edit, and full-patch outcomes.

Model	Outcome	Natural	Surgical	Paired n	Raw p	Holm p
DeepSeek-V3.1	Manifest semantic success	42/56	47/56	56	0.2668	1.0000
DeepSeek-V3.1	Manifest patch success	37/56	41/56	56	0.4545	1.0000
DeepSeek-V3.1	Applicable full patch	41/56	45/56	56	0.4807	1.0000
DeepSeek-V3.1	In-scope full patch	56/56	56/56	56	1.0000	1.0000
DeepSeek-V3.1	Changes any provided source file	6/56	9/56	56	0.5078	1.0000
DeepSeek-V3.1	Applicable in-scope source edit	6/56	8/56	56	0.7266	1.0000
Gemma-3n-E4B-it	Manifest semantic success	1/41	4/41	41	0.3750	1.0000
Gemma-3n-E4B-it	Manifest patch success	0/41	0/41	41	1.0000	1.0000
Gemma-3n-E4B-it	Applicable full patch	3/41	4/41	41	1.0000	1.0000
Gemma-3n-E4B-it	In-scope full patch	22/41	30/41	41	0.0386	0.2314
Gemma-3n-E4B-it	Changes any provided source file	1/41	0/41	41	1.0000	1.0000
Gemma-3n-E4B-it	Applicable in-scope source edit	1/41	0/41	41	1.0000	1.0000
Llama-3.3-70B	Manifest semantic success	33/56	33/56	56	1.0000	1.0000
Llama-3.3-70B	Manifest patch success	15/56	11/56	56	0.2188	1.0000
Llama-3.3-70B	Applicable full patch	13/56	14/56	56	1.0000	1.0000
Llama-3.3-70B	In-scope full patch	56/56	56/56	56	1.0000	1.0000
Llama-3.3-70B	Changes any provided source file	5/56	2/56	56	0.2500	1.0000
Llama-3.3-70B	Applicable in-scope source edit	5/56	2/56	56	0.2500	1.0000
GPT-OSS-120B	Manifest semantic success	28/56	35/56	56	0.1671	0.6683
GPT-OSS-120B	Manifest patch success	13/56	28/56	56	0.0107	0.0534
GPT-OSS-120B	Applicable full patch	8/56	24/56	56	0.0025	0.0150
GPT-OSS-120B	In-scope full patch	56/56	56/56	56	1.0000	1.0000
GPT-OSS-120B	Changes any provided source file	4/56	7/56	56	0.4531	0.9063
GPT-OSS-120B	Applicable in-scope source edit	3/56	7/56	56	0.2891	0.8672

Table A.4: Model-level prompt-style comparisons of configuration-code manifest-level failures.

Model	**Comparison**	n	**Test**	p**-value**
DeepSeek-V3.1	Failure presence	56	Exact McNemar	0.4545
DeepSeek-V3.1	Failure-category distribution	9	Bowker symmetry	0.5724
DeepSeek-V3.1	Failure-subtype distribution	9	Bowker symmetry	0.4159
Gemma-3n-E4B-it	Failure presence	41	Exact McNemar	1.0000
Gemma-3n-E4B-it	Failure-category distribution	41	Bowker symmetry	0.2056
Gemma-3n-E4B-it	Failure-subtype distribution	41	Bowker symmetry	0.2491
Llama-3.3-70B	Failure presence	56	Exact McNemar	0.2188
Llama-3.3-70B	Failure-category distribution	40	Bowker symmetry	0.2474
Llama-3.3-70B	Failure-subtype distribution	40	Bowker symmetry	0.2296
GPT-OSS-120B	Failure presence	56	Exact McNemar	0.0107
GPT-OSS-120B	Failure-category distribution	20	Bowker symmetry	0.3679
GPT-OSS-120B	Failure-subtype distribution	20	Bowker symmetry	0.3679